
Criterio 3: Documentación y crítica por archivo de código

Documentación técnica de auditoría · Documento 1.0 · Un documento por criterio de la rúbrica

Consultor Estadístico Senior · Auditoría de Proyectos de Datos

Universidad Santo Tomás · Ustadistica · 2026-I

Repositorio auditado: `Observatorio_Ministerio_de_Ciencias_Grupo8`

Commit auditado: 1528939

Versión 1.0 · 15 de septiembre de 2026

Documento generado de forma reproducible a partir de la auditoría del repositorio. La construcción de este documento está documentada en `documentacion_auditoria`.

Índice

1. Contexto y guía de lectura	3
1.1. Qué es este documento	3
1.2. Glosario (solo los términos que usa este documento)	3
2. Documentación y crítica por archivo de código	4
2.0.1. src/ingesta/_init_.py	4
2.0.2. src/ingesta/minciencias.py	7
2.0.3. src/ingesta/produccion.py	8
2.0.4. src/Transformacion.py	10
2.0.5. src/modelo/_init_.py	12
2.0.6. src/modelo/dimensional.py	13
2.0.7. src/analisis/_init_.py	15
2.0.8. src/analisis/calidad.py	17
2.0.9. src/analisis/diversidad.py	19
2.0.10. src/analisis/genero.py	21
2.0.11. src/analisis/longitudinal.py	23
2.0.12. src/analisis/produccion.py	24
2.0.13. src/analisis/redes.py	27
2.0.14. src/analisis/territorial.py	29
2.0.15. scripts/generar_manual.py	31
2.0.16. scripts/sprint2_duckdb.py	35
2.0.17. scripts/sprint2_genero_ocde.py	38
2.0.18. scripts/sprint2_matrices_transicion.py	40
2.0.19. scripts/sprint2_panel_longitudinal.py	42
2.0.20. scripts/sprint2_territorial.py	44
2.0.21. scripts/sprint3_grafo_interactivo.py	46
2.0.22. scripts/sprint3_redes.py	48
2.0.23. scripts/sprint4_diversidad.py	51
2.0.24. scripts/sprint5_duckdb.py	54
2.0.25. scripts/sprint5_produccion.py	56
2.0.26. scripts/sprint6_geografia_institucional.py	59
2.0.27. scripts/sprint6_ocde_composicion.py	62
2.0.28. scripts/sprint6_sankey_categoria.py	64
2.0.29. scripts/sprint6_sankey_territorial.py	67
2.0.30. scripts/sprint6_tabla_maestra_ies.py	69
2.0.31. scripts/sprint6_validacion_calidad.py	71
2.0.32. app/streamlit_app.py	74
2.0.33. streamlit_app.py	75
2.0.34. docs/manual.ipynb	78
2.0.35. notebooks/01_eda.ipynb	80
2.0.36. tests/_init_.py	82
2.0.37. tests/test_ingesta.py	83

2.0.38. notebooks/tarea_analisis_bases_de_datos/2019_codigo.py	84
2.0.39. notebooks/tarea_analisis_bases_de_datos/datos_2021.ipynb	86
2.0.40. notebooks/tarea_analisis_bases_de_datos/inv_17_codigo_graficas.R	88
2.0.41. notebooks/tarea_dimensiones_y_hechos/codigo_dimensiones_C.R	89
2.0.42. notebooks/tarea_dimensiones_y_hechos/dimensiones.ipynb	91
2.0.43. notebooks/tarea_dimensiones_y_hechos/tabla_hechos_codigo.R	93
2.0.44. notebooks/tarea_gran_tabla/analisis2.ipynb	94
2.0.45. notebooks/tarea_gran_tabla/analisis_de_clusters.R	96
2.0.46. notebooks/tarea_gran_tabla/analisis_multivariado_base_gran_table.R	98
2.0.47. notebooks/tarea_gran_tabla/codigo_analisis_univariado.py	100
2.0.48. notebooks/tarea_gran_tabla/codigo_creacion_gran_tabla_a_partir_del_	
modelo_estrella.py	102
2.0.49. notebooks/tarea_join/codigo_join.py	104
2.0.50. notebooks/tarea_tabla/arreglar_base_consultoria_7.R	105
2.0.51. src/__init__.py	107
3. Relación con los demás criterios	108
4. Conclusiones y recomendaciones del criterio	108

1 Contexto y guía de lectura

1.1 Qué es este documento

Este documento desarrolla el **criterio 3 de la rúbrica**: *Documentación y crítica por archivo de código*. El repositorio auditado es el Observatorio de investigadores reconocidos del Ministerio de Ciencia, Tecnología e Innovación de Colombia (MinCiencias), que consolida y analiza **6 convocatorias (2013–2021)** del padrón oficial de investigadores y su producción científica. Esta serie cubre los **6 criterios** de la rúbrica en documentos separados, más el documento maestro que los integra.

En resumen: Este criterio evalúa la documentación de los archivos de código: qué hace cada uno, cómo está construido, si está bien hecho y si al ejecutarse produce lo que se buscaba. **Conclusión:** el código está bien construido en general (modular y documentado); 10 de los 16 scripts principales se ejecutan correctamente en un clon del repositorio y los 6 restantes fallan por causas identificadas (datos no versionados y una dependencia no declarada).

Nota metodológica

Cómo leer este documento: cada PDF de esta serie desarrolla *un* criterio de la rúbrica. Los demás criterios se tratan a fondo en sus propios documentos y en el **documento maestro** (*maestro_auditoria.pdf*). Si este es tu primer acercamiento, lee primero la portada y este contexto; al final encontrarás las conclusiones del criterio.

1.2 Glosario (solo los términos que usa este documento)

Esta tabla incluye únicamente los términos técnicos que aparecen en este documento, explicados en lenguaje sencillo:

Tabla 1. Glosario de términos presentes en este documento

Término	Qué significa
HHI (Herfindahl-Hirschman)	Índice que mide si algo está concentrado o repartido. Va de 0 (todo repartido) a 1 (todo en un solo lugar). En el informe se usa para ver si los investigadores se concentran en pocos departamentos.
Matriz de transición	Tabla que muestra con qué probabilidad un investigador pasa de una categoría (p. ej. Junior) a otra (p. ej. Asociado) entre dos convocatorias. Permite ver si el sistema 'sube', 'mantiene' o 'pierde' investigadores.
Tasa de retención	Porcentaje de investigadores que siguen presentes en la siguiente convocatoria. Una retención baja puede indicar abandono o cambio de reglas.
Representación relativa	Cuántas veces aparece un grupo (p. ej. afrocolombianos) en el padrón frente a lo que correspondería según su peso en la población colombiana (DANE). Un valor de 0.3 indica que aparece 3 veces menos de lo esperado.

Término	Qué significa
k-prototypes	Algoritmo de agrupamiento (clustering) que agrupa personas con perfiles similares cuando los datos mezclan números (edad) y categorías (área, género).
MCA / FAMD	Técnicas para resumir tablas con muchas categorías en pocos ejes visualizables, de modo que se puedan ver patrones (p. ej. qué áreas se asocian con qué géneros).
Modelo dimensional (esquema estrella)	Forma de organizar los datos en tablas de 'hechos' (métricas: quién, cuándo, qué categoría) y 'dimensiones' (descripciones: área, región, institución), para hacer consultas rápidas y reportes.
DuckDB	Base de datos analítica ligera que se ejecuta dentro del propio proyecto (sin instalar un servidor). Se usa para almacenar el modelo dimensional.
Mojibake	Texto ilegible por un problema de codificación (p. ej. 'FÃsica' en lugar de 'Física'). Detectado en el CSV del repositorio.
Contrato de esquema	Validación automática que garantiza que un archivo de datos tenga las columnas, tipos y valores esperados antes de analizarlo.
Pipeline	Cadena de procesos que lleva los datos desde la fuente hasta el análisis (ingesta, limpieza, transformación, modelado). También llamada 'cadena de procesamiento' o 'flujo de datos'.
Markovianidad	Propiedad de un proceso en el que la probabilidad de pasar al siguiente estado solo depende del estado actual (y no de toda la historia). Se usa para validar si las matrices de transición de categorías son adecuadas.
Modelos de supervivencia (Kaplan-Meier / Cox)	Técnicas estadísticas para analizar 'tiempo hasta que ocurre un evento' (p. ej. cuánto tarda un investigador en salir del sistema), incluyendo datos incompletos.
Intervalo de confianza (IC) y bootstrap	El intervalo de confianza indica el rango plausible de una estimación (p. ej. 0.15-0.19). El bootstrap es un método que lo calcula remuestreando los datos muchas veces, sin fórmulas complejas.
CI/CD y Docker	Automatización de pruebas y despliegue (CI/CD) y empaquetado de la aplicación en un contenedor (Docker) para que funcione igual en cualquier máquina.
Sprint	Ciclo corto de trabajo (en este proyecto, de 2 semanas) con objetivos concretos.

2 Documentación y crítica por archivo de código

Cobertura: 51 de 51 archivos de código documentados en esta versión.

2.0.1 src/ingesta/__init__.py

Rol: ingestas · **Líneas:** 53

Descripción funcional Punto de entrada de carga de datos del paquete ingesta. Expone `cargar_consolidado()` y `cargar_produccion()`, con prioridad de rutas (CSV crudo Socrata -> XLSX local para investigadores; CSV crudo obligatorio para producción), normaliza nombres de columnas a MAYÚSCULAS y reporta conteos cargados.

Análisis de la lógica Define rutas absolutas `ROOT/datos/raw` y `ROOT/datos/tarea_join`. `cargar_consolidado()` prefiere `datos/raw/investigadores_consolidado.csv` (Socrata) y cae a `datos/tarea_join/investigadores_consolidado.xlsx`; `cargar_produccion()` exige `datos/raw/produccion_grupos.csv` y lanza `FileNotFoundError` si falta, sugiriendo `python -m src.ingesta.produccion`. Es el contrato de entrada que usan todos los scripts (`from ingesta import cargar_consolidado, cargar_produccion`).

Lo que está bien construido

- Mecanismo de fallback `raw -> tarea_join` permite ejecutar análisis de investigadores sin datos crudos.
- Uppercase centralizado de columnas: garantiza el contrato MAYÚSCULAS que exigen `analisis/*.py` y `Transformacion.py`.
- Mensajes de error accionables que indican el comando de ingesta a ejecutar.

Puntos de mejora (si aplican)

- `cargar_consolidado()` no valida el shape esperado (77.237 x 30) ni los tipos: con `low_memory=True`, `id_persona_pr` con ceros a la izquierda puede leerse de forma inconsistente (`int` vs `str`).
- `cargar_produccion()` solo acepta el CSV crudo: no hay fallback ni lectura desde el modelo DuckDB; bloquea todo el análisis de producción en el clon.
- Depende de `openpyxl` para `read_excel`, declarado solo en `requirements.txt` (no en `pyproject.toml`): una instalación vía `poetry` pura fallaría.
- El comando sugerido `python -m src.ingesta.minciencias` no pagina (hace una sola llamada con `limit=100.000`) y puede truncar silenciosamente los 77.237 registros si Socrata limita la respuesta por petición (contraste con `ingesta/produccion.py` que sí pagina).

Sugerencias concretas

- Añadir validación de shape y conteo contra `datos/catalogo.yaml` (`tamano_registros`).
- Especificar `dtypes` en `read_csv` (`id_persona_pr` como `str`) para preservar ceros a la izquierda y garantizar el cruce con producción.
- Declarar `openpyxl` y `pyyaml` en `pyproject.toml`.
- Añadir fallback para `cargar_produccion` desde `parquet` o desde el modelo DuckDB si el CSV crudo no está.

Ejecutabilidad cargar_consolidado(): SÍ en el clon — cae al XLSX datos/tarea_join/investigadores_consolidado.xlsx (existe, 8 MB) porque datos/raw/investigadores_consolidado.csv NO existe (solo .gitkeep). cargar_produccion(): NO — datos/raw/produccion_grupos.csv no existe (gitignored; requiere descarga de ~1,2 GB). El módulo en sí es importable sin datos.

¿Produce lo que se buscaba? No produce archivos; es capa de lectura. Sus salidas (DataFrames) alimentan transformar() y analisis.*. Cumple el contrato documentado en README (ingesta/ cargar_consolidado + cargar_produccion).

Riesgo / Limitación

Problemas detectados:

- src/ingesta/_init_.py:44-48: cargar_produccion() lanza FileNotFoundError sin alternativa; en el clon (sin datos/raw) todos los scripts sprint5/sprint6 de producción mueren en el arranque.

Evidencia en src/ingesta/_init_.py (líneas 44-48):

```
1 if not _PROD.exists():
2     raise FileNotFoundError(
3         "Dataset de producción no encontrado. "
4         "Ejecute: python -m src.ingesta.produccion"
5     )
```

- src/ingesta/_init_.py:26: read_csv(low_memory=False) + read_excel sin dtype explícito -> ID_PERSONA_PR puede inferirse como int en un lado y str en otro, rompiendo el cruce id_persona_pd <-> id_persona_pr.

Evidencia en src/ingesta/_init_.py (línea 26):

```
1 df = pd.read_csv(_RAW, low_memory=False)
```

- src/ingesta/_init_.py:18-36: no verifica duplicados por (ID_PERSONA_PR, ID_CONVOCATORIA) pese a que el README define esa granularidad para el dataset.

Evidencia en src/ingesta/_init_.py (líneas 18-36):

```
1 def cargar_consolidado() -> pd.DataFrame:
2     """Carga el dataset consolidado de investigadores (77 237
3         registros, 30 cols).
4
5         Prioridad: datos/raw/investigadores_consolidado.csv (
6             Socrata)
7             datos/tarea_join/investigadores_consolidado.
8                 xlsx (Excel local)
9
10    """
11     [...]
```

Valoración global mejorar — diseño sólido de fallback, pero requiere validación de tipos/duplicados y dependencias declaradas (openpyxl/pyyaml) para que la

instalación reproducible funcione.

2.0.2 src/ingesta/minciencias.py

Rol: ingestá · **Líneas:** 70

Descripción funcional Descarga el dataset consolidado de investigadores reconocidos de MinCiencias desde datos.gov.co (API Socrata, sodapy; recurso bqtm-4y2h) y lo guarda en datos/raw/investigadores_consolidado.csv. Actualiza el campo ultimo_pull del catálogo YAML. CLI con `-limit` (muestra de pruebas) y `-token` (app token Socrata).

Análisis de la lógica `descargar()` crea el cliente Socrata, trae hasta `limit` (por defecto 100.000) registros y construye un `DataFrame`. `guardar()` persiste a CSV UTF-8 en datos/raw. `actualizar_catalogo()` reescribe el YAML con la fecha de hoy. `main()` orquesta y, si no hay límite, actualiza el catálogo. Logging con formato estándar.

Lo que está bien construido

- CLI limpia y documentada (`-limit` para pruebas evita descargas completas).
- Separación `descargar/guardar/actualizar_catalogo` permite reutilización y tests.
- Maneja app token Socrata para evitar throttling.

Puntos de mejora (si aplican)

- Sin paginación para datasets grandes (el límite por defecto 100.000 cubre el padrón de 77k, pero es frágil si el dataset crece).
- Escribe en datos/raw (gitignored): el artefacto no se versiona ni se valida.
- `actualizar_catalogo` reescribe el YAML completo con `yaml.dump`, perdiendo comentarios y formato original del catálogo.
- Sin control de calidad post-descarga (conteo de filas, codificación UTF-8) — el mojibake del CSV canónico no se detecta aquí.

Sugerencias concretas

- Añadir paginación genérica (`offset`) para datasets >100k.
- Validar el resultado (filas esperadas, codificación) tras la descarga.
- Actualizar el catálogo conservando comentarios o con un campo separado.

Ejecutabilidad Parcial: requiere red y API Socrata (no ejecutable offline); en la auditoría no se ejecutó porque los datos ya están en el clon (el XLSX versionado proviene de esta fuente). datos/raw está gitignored, por lo que el resultado no es visible en el repositorio.

¿Produce lo que se buscaba? Sí, en su momento: los 77.237 registros (según README/catálogo) se descargaron de esta fuente; el XLSX consolidado versionado es el derivado. No verificable en clon por gitignore.

Riesgo / Limitación

Problemas detectados:

- yaml.dump reescribe el catálogo y puede alterar su estructura/orden (riesgo de formato).

Valoración global mantener — simple y funcional; mejorar la validación de codificación y el manejo del catálogo.

2.0.3 src/ingesta/produccion.py

Rol: ingest · **Líneas:** 106

Descripción funcional Descarga paginada del dataset de Producción de Grupos (Socrata 33dq-ab5a, 3.166.629 filas) a datos/raw/produccion_grupos.csv, con modo `-limit` para pruebas y actualización de metadatos en datos/catalogo.yaml. Es la única ingest del proyecto con paginación real.

Análisis de la lógica Usa `sodapy.Socrata` con `timeout=300`; `descargar()` pagina con `$offset/$limit` de 50.000 y `order='id'` hasta que un chunk venga vacío o con menos de `PAGE_SIZE`; con `-limit` hace una sola llamada. `guardar()` escribe CSV UTF-8; `actualizar_catalogo()` actualiza `ultimo_pull` y `tamano_registros` de la fuente 33dq-ab5a en `catalogo.yaml`; `main()` orquesta y omite el catálogo en modo prueba.

Lo que está bien construido

- Paginación robusta con orden estable (`:id`) y condición de corte doble (chunk vacío o `<PAGE_SIZE`).
- CLI argparse con modo `limit` para pruebas sin descargar 1,2 GB.
- Cierra el cliente Socrata explícitamente y loguea progreso por página.
- Actualiza el catálogo de metadatos automáticamente tras la descarga completa.

Puntos de mejora (si aplican)

- Importa yaml en el módulo pero PyYAML NO está declarado en pyproject.toml ni requirements.txt (sodapy no lo trae) -> ImportError en un entorno limpio.
- Descarga todas las columnas sin proyección (select), incluyendo títulos largos (nme_producto_pd) -> payload y tiempo innecesarios en 3,2 M de filas.
- Sin reintentos/backoff ante errores de red ni validación del conteo final contra el catálogo (3.166.629).
- yaml.dump reescribe catalogo.yaml reformateándolo (indentación y comillas), generando diffs ruidosos en un archivo versionado con formato manual.
- if limit: con limit=0 ejecutaría la descarga completa (valor falsy no contemplado).

Sugerencias concretas

- Añadir select de columnas analíticas (id_persona_pd, id_convocatoria, ano_convo, nme_tipo_medicion_pd, cod_grupo_gr, etc.).
- Declarar pyyaml en pyproject.toml y requirements.txt.
- Verificar len(df) contra catalogo['tamano_registros'] y reintentar páginas fallidas con backoff exponencial.
- Guardar en parquet con tipos explícitos (id_persona_pd como str de 10 caracteres).

Ejecutabilidad Ejecutable con red: `python -m src.ingesta.produccion` (completa) o `-limit N`. Requiere sodapy, pandas y PyYAML (este último no declarado en el proyecto). Escribe en `datos/raw/` (gitignored; en el clon el directorio existe con `.gitkeep` y es escribible localmente, pero la salida NO está versionada ni presente en el clon).

¿Produce lo que se buscaba? Produce `datos/raw/produccion_grupos.csv` (gitignored, ~1,2 GB), el input que exige `cargar_produccion()`. No genera evidencias/ ni artifacts/ directamente; coincide con lo documentado en README (ingesta por API Socrata paginada, Sprint 5).

Riesgo / Limitación**Problemas detectados:**

- `src/ingesta/produccion.py:24`: `import yaml` no cubierto por `pyproject.toml` -> falla en instalación reproducible (probablemente ejecutado ad-hoc con PyYAML instalado manualmente).

Evidencia en src/ingesta/produccion.py (línea 24):

```
1 import yaml
```

- `src/ingesta/produccion.py:47-53`: en modo `limit` `main()` igualmente guarda el DataFrame como CSV 'completo' en SALIDA: si alguien corre `-limit` sin querer, sobrescribe el archivo crudo con una muestra.

Evidencia en src/ingesta/produccion.py (líneas 47-53):

```

1     if limit:
2         registros = client.get(SOCRATA_ID, limit=limit)
3         client.close()
4         df = pd.DataFrame.from_records(registros)
5         log.info("Descargados (modo limit=%d): %d filas, %d
6                 columnas",
7                 limit, len(df), len(df.columns))
8     [...]

```

- `src/ingesta/produccion.py:59-65`: si la última página devuelve exactamente `PAGE_SIZE` y la siguiente viene vacía, el loop hace una petición extra vacía (ineficiencia menor, no error).

Evidencia en `src/ingesta/produccion.py` (líneas 59-65):

```

1     chunk = client.get(SOCRATA_ID, limit=PAGE_SIZE,
2                       offset=offset, order=":id")
3     if not chunk:
4         break
5     paginas.append(pd.DataFrame.from_records(chunk))
6     offset += len(chunk)
7     if len(chunk) < PAGE_SIZE:
8     [...]

```

Valoración global mantener/mejorar — única ingesta con paginación correcta del proyecto; corregir la dependencia PyYAML declarada y el sobrescrito en modo `limit` para producción segura.

2.0.4 `src/Transformacion.py`

Rol: `transformacion` · **Líneas:** 222

Descripción funcional Núcleo de limpieza y normalización del dataset consolidado de investigadores (77.237 registros). Aplica limpieza de texto (`strip` + mayúsculas), parseo robusto del año de convocatoria (fecha, serial de Excel o regex de 4 dígitos), estandarización de género y tratamiento de faltantes (mediana para numéricas, 'NO REPORTADO' para categóricas). Entrada: DataFrame crudo de `ingesta.cargar_consolidado()`; salida: DataFrame transformado listo para análisis.

Análisis de la lógica Flujo en `transformar()`: `limpiar_texto` -> `parsear_ano_convocatoria` -> `estandarizar_genero` -> `tratar_faltantes`. Usa constantes `COLS_NUMERICAS` y `COLS_CATEGORICAS`; funciones puras que copian el df. En `__main__` inserta `ROOT/src` en `sys.path` y llama `cargar_consolidado()` + `transformar()`. Se conecta con el resto del pipeline porque todos los scripts `sprint*` ejecutan `df = transformar(cargar_consolidado())` antes de llamar a `analisis.*`.

Lo que está bien construido

- Funciones puras con docstrings claros y operaciones idempotentes (strip + upper).
- Parseo de año multiestrategia (fecha, serial Excel, regex) — robusto frente a formatos heterogéneos de ANO_CONVO.
- Centraliza la lógica de limpieza en un solo módulo consumido por todos los scripts.

Puntos de mejora (si aplican)

- Imputa con mediana global sin crear columna indicadora de imputación: para un observatorio de calidad de datos, destruye la señal de 'missingness' (el propio proyecto reporta faltantes como hallazgo).
- No corrige la anomalía documentada de la convocatoria 833 (ANO_CONVO=2019-12-06, año nominal 2018): ANO_CONVO_INT quedará en 2019 para esa convocatoria.
- Imputar mediana en columnas ordinales (ORDEN_CLAS_PR, NRO_ORDEN_FORM_PR) puede producir valores no enteros que contaminan agrupaciones posteriores.
- Docstring dice 'Grupo 7' y 'generado por ingesta.py' mientras el repo es Grupo 8 y el módulo de carga es ingesta/__init__.py (desactualización cosmética).

Sugerencias concretas

- Añadir columna flag (ej. *_imputado) por variable imputada y comparar análisis con/sin imputación.
- Aplicar corrección de año nominal para la conv. 833 según datos/catalogo.yaml (campo ano_convo_campo).
- Validar shape/nulos contra catálogo (77.237 x 30) y loguear filas descartadas.
- Escribir tests para parsear_ano_convocatoria con casos: '2013', serial 41739, '2019-12-06', NaN.

Ejecutabilidad Sí, ejecutable tal cual en el clon para el dataset de investigadores: __main__ carga vía ingesta.cargar_consolidado(), que cae al fallback datos/tarea_join/investigadores_consolidado.xlsx (existe, 8 MB) porque datos/raw/ solo tiene .gitkeep (sin CSV crudo). Requiere pandas, numpy (declarados) y openpyxl (solo en requirements.txt, no en pyproject.toml). NO depende de datos/raw ni de .duckdb.

¿Produce lo que se buscaba? No escribe archivos (solo devuelve DataFrame y print de shape); el volcado a CSV lo hacen los scripts sprint*. Su output alimenta todos los análisis del README; consistente con lo documentado.

Riesgo / Limitación**Problemas detectados:**

- Transformacion.py:107-109: la regex `r'\{\}b(20[0-9]{2})\{\}b'` solo captura años 2000-2099; correcto para el dominio, pero un año fuera de rango quedaría como NaT sin aviso.

Evidencia en Transformacion.py (líneas 107-109):

```
1      # Intento 3: año de 4 dígitos embebido en el texto
2      anio_regex = s.str.extract(r"\b(20[0-9]{2})\b")[0]
3      fecha_regex = pd.to_datetime(anio_regex, format="%Y",
                                   errors="coerce")
```

- Transformacion.py:99-100: `pd.to_datetime(s, dayfirst=True)` sobre un texto numérico tipo '640' (id de convocatoria) produciría NaT; depende del contenido real de ANO_CONVO, que el catálogo documenta como año o timestamp.

Evidencia en Transformacion.py (líneas 99-100):

```
1      fecha = pd.to_datetime(s, dayfirst=True, errors="coerce")
```

- Transformacion.py:140-141: si una columna numérica fuera 100 % NaN, `mediana=NaN` y `fillna(NaN)` no corrige nada (caso límite no manejado).

Evidencia en Transformacion.py (líneas 140-141):

```
1      mediana = df[col].median()
2      df[col] = df[col].fillna(mediana)
```

Valoración global mejorar — núcleo correcto y bien estructurado; debe mejorarse la trazabilidad de imputaciones y la corrección del año nominal 2018 antes de seguir usándolo en un informe de calidad de datos.

2.0.5 src/modelo/_init_.py

Rol: modelado · **Líneas:** 2

Descripción funcional Marcador de paquete Python para `src/modelo/` (módulo del modelo dimensional).

Análisis de la lógica Sin lógica: archivo de marcador.

Lo que está bien construido

- Correcto por convención.

Puntos de mejora No se identifican debilidades significativas: el archivo cumple su función de forma clara y consistente.

Ejecutabilidad N/A.

¿Produce lo que se buscaba? N/A.

Valoración global mantener — estándar de Python.

2.0.6 `src/modelo/dimensional.py`

Rol: modelado · **Líneas:** 286

Descripción funcional Construye el modelo dimensional (Issue #14) en DuckDB: 7 dimensiones (convocatoria, investigador, categoría, área, territorio, formación, institución) + `fact_clasificacion` (investigador x convocatoria) + tabla puente N:N `bridge_hecho_institucion`. Entrada: DataFrame transformado; salida: `datos/processed/observatorio.duckdb` (gitignored).

Análisis de la lógica Construye cada dimensión con `drop_duplicates` sobre su clave natural y SK solo para territorio/institucion; `construir_fact_clasificacion()` une la SK de territorio por columnas geográficas e inserta `sk_hecho`; `construir_bridge_institucion()` descompone `inst_filia` con un dict `sk` y un loop de filas. `cargar_en_duckdb()` crea el archivo (`mkdir parents`) y hace `DROP + CREATE TABLE AS SELECT` por tabla; `resumen_modelo()` lista tablas y conteos. Importa `normalizar_categoria/ORDEN_CATEGORIAS` desde `analisis.longitudinal` (requiere `ROOT/src` en `sys.path`).

Lo que está bien construido

- Esquema claro y documentado (asociación dimensión -> hecho, más bridge para la relación N:N de instituciones).
- SK para territorio e institución y ordenamiento determinista antes de `drop_duplicates` en varias dimensiones.
- `DB_PATH` centralizado y carga idempotente (`DROP + CREATE`).
- La tabla puente resuelve correctamente la granularidad N:N que una dimensión plana no podría representar.

Puntos de mejora (si aplican)

- LA 'PRIMERA APARICIÓN' NO ES CRONOLÓGICA: `construir_dim_investigador()` ordena solo por `ID_PERSONA_PR` (no por `ANO_CONVO_INT`) y luego `keep='first'`; el snapshot de atributos no corresponde realmente a la primera convocatoria del investigador.

- El fact referencia claves naturales (id_convocatoria, id_persona_pr, id_clas_pr, id_area_con_pr, id_niv_formacion_pr) sin SK: las dimensiones (salvo territorio/institucion) carecen de clave sustituta y no hay FKs/constraints — es más un conjunto de tablas de referencia que un modelo estrella estricto.
- `_drop_and_create()` es código muerto (definido y nunca llamado), sugiriendo una intención de DDL con constraints que no se implementó.
- `construir_bridge_institucion()` itera las ~77.237 filas con `iterrows`: lento y sin vectorización.
- El archivo `.duckdb` es `gitignored` y NO existe en el clon: el modelo no es reproducible sin volver a ejecutar (requiere `duckdb >= 1.0` y el XLSX).

Sugerencias concretas

- Ordenar por `[ANO_CONVO_INT, ID_PERSONA_PR]` en `construir_dim_investigador()` para que 'primera aparición' sea real.
- Añadir SK (surrogate) a todas las dimensiones y referenciarlas en el fact; definir PK/FK con `ALTER TABLE` o `CREATE TABLE` con constraints.
- Vectorizar el bridge (`explode + map`) en lugar de `iterrows`.
- Documentar el modelo como 'tablas de referencia + hecho' si no se van a imponer FKs, para no sobrevender el término 'estrella'.

Ejecutabilidad Sí para la parte de investigadores en el clon: requiere `duckdb`, `pandas` y el XLSX de `tarea.join` (presente); escribe `datos/processed/observatorio.duckdb` (`gitignored`; en el clon solo hay `.gitkeep`, el archivo NO está). No requiere `datos/raw`. NOTA: para incorporar producción (`sprint5_duckdb.py`) sí haría falta el CSV crudo.

¿Produce lo que se buscaba? Sí (con datos): genera las tablas documentadas y `evidencias/duckdb_resumen_modelo.txt` existe como evidencia de ejecución del equipo; el README reporta el modelo estrella como completado (Issue #14). El `.duckdb` final no está versionado por diseño (`.gitignore: *.duckdb`).

Riesgo / Limitación

Problemas detectados:

- `src/modelo/dimensional.py:69-74`: sort solo por `ID_PERSONA_PR` -> 'keep first' no es la primera aparición temporal; los atributos del snapshot (género, región) pueden pertenecer a una convocatoria posterior.

Evidencia en `src/modelo/dimensional.py` (líneas 69-74):

```

1     dim = (
2         df[cols]
3         .sort_values("ID_PERSONA_PR")
4         .drop_duplicates(subset=["ID_PERSONA_PR"], keep="
           first")

```

```

5         .reset_index(drop=True)
6     )

```

- src/modelo/dimensional.py:41-44: `_drop_and_create` nunca se usa (código muerto).

Evidencia en src/modelo/dimensional.py (líneas 41-44):

```

1 def _drop_and_create(con: duckdb.DuckDBPyConnection, ddl: str
2     ) -> None:
3     table = ddl.split("(")[0].split()[-1]
4     con.execute(f"DROP TABLE IF EXISTS {table}")
5     con.execute(ddl)

```

- src/modelo/dimensional.py:192-197: el merge por columnas geográficas con NaN (código DANE ausente) no empareja en pandas (NaN != NaN) -> fact con sk_territorio NULL para filas sin geografía, sin contarlas ni advertirlo.

Evidencia en src/modelo/dimensional.py (líneas 192-197):

```

1 fact = fact.merge(
2     dim_territorio[["sk_territorio", *geo_cols]],
3     on=geo_cols,
4     how="left",
5 )
6 fact = fact.drop(columns=geo_cols)

```

- src/modelo/dimensional.py:52-56: `construir_dim_convocatoria` conserva `ANO_CONVO_INT` = 2019 para la convocatoria id 20 (año nominal 2018), propagando la anomalía de datos documentada al modelo.

Evidencia en src/modelo/dimensional.py (líneas 52-56):

```

1 cols = ["ID_CONVOCATORIA", "NME_CONVOCATORIA", "
2     ANO_CONVO_INT", "ANO_CONVO_FECHA"]
3 dim = df[cols].drop_duplicates(subset=["ID_CONVOCATORIA"]
4     ).copy()
5 dim = dim.sort_values("ANO_CONVO_INT").reset_index(drop=
6     True)
7 dim.columns = dim.columns.str.lower()
8 return dim

```

Valoración global mejorar — modelo útil y correcto en granularidad, pero necesita SK/constraints para ser un 'modelo estrella' real y corregir el snapshot cronológico del investigador antes de citarlo como modelo dimensional canónico.

2.0.7 src/analysis/__init__.py

Rol: orquestacion · **Líneas:** 80

Descripción funcional Fachada del paquete analisis: re-exporta las funciones públicas de genero, territorial, redes, diversidad, longitudinal y produccion, y declara `__all__`. Define la API estable que consumen `scripts/sprint*/*.py` y `streamlit_app.py`.

Análisis de la lógica Importa explícitamente cada función/constante de los 6 submódulos y las lista en `__all__`. No contiene lógica de negocio; su función es el contrato de importación (`from analisis import pct_femenino_por_area, hhi, construir_grafo, comparar_dane, ...`). Cualquier script que importe analisis dispara la importación de `networkx` (vía `redes`) y de `longitudinal` en el arranque.

Lo que está bien construido

- API pública explícita y bien documentada para todos los análisis del proyecto.
- `__all__` completo y ordenado facilita autocompletado e inspección.
- Permite a `scripts` y `dashboard` importar sin conocer la estructura interna del paquete.

Puntos de mejora (si aplican)

- Importación eager de los 6 submódulos: importar `'analisis'` carga `networkx` y `longitudinal` (peso y acoplamiento); si una dependencia opcional falta, todo el paquete falla al importar.
- Duplicación de símbolos entre el bloque de imports y `__all__` (mantenimiento manual: añadir una función exige tocar dos lugares).
- No hay validación de esquema en la frontera: las funciones asumen columnas MAYÚSCULAS y `ANO_CONVO_INT` sin verificar (contrato implícito con `Transformacion.py`).

Sugerencias concretas

- Convertir `redes` (y `pyvis`) en import perezoso o submódulo opcional para no penalizar al `dashboard`.
- Generar `__all__` a partir de una sola lista de nombres para evitar duplicación.
- Añadir una función `validate_esquema(df)` que verifique columnas requeridas antes de los análisis.

Ejecutabilidad Sí, importable en el clon si las dependencias están instaladas (`pandas`, `numpy`, `networkx`; `pyvis` solo dentro de `generar_html_pyvis`). No lee archivos: la carga la hace ingesta. El fallo de cargar `produccion` en el clon no afecta el import, pero sí a las funciones de `.produccion` al ejecutarlas.

¿Produce lo que se buscaba? No produce outputs; habilita el resto del pipeline. Coincide con la estructura documentada en el README (`src/analisis` como `'lógica reutilizable'`).

Riesgo / Limitación**Problemas detectados:**

- `src/analisis/__init__.py:4-36`: importa funciones que no usa directamente; si `analisis/longitudinal.py` fallara al importar (dependencias), derriba también `genero/territorial`, que son independientes (acoplamiento innecesario).

Evidencia en `src/analisis/__init__.py` (líneas 4-36):

```

1  from .genero import pct_femenino_por_area,
    tabla_pivot_pct_femenino, brecha_genero,
    evolucion_pct_femenino
2  from .territorial import hhi, hhi_por_convocatoria,
    top_territorios, tabla_cuotas
3  from .redes import (
4      construir_pares, construir_grafo, metricas_grafo,
5      tabla_nodos, tabla_aristas, metricas_por_convocatoria,
6      subgrafo_grado_minimo, generar_html_pyvis,
7      normalizar_institucion,
8      [...]
```

Valoración global mantener — fachada correcta; refactorizar a imports perezosos o dividir el paquete para reducir el acoplamiento del dashboard.

2.0.8 `src/analisis/calidad.py`

Rol: analisis · **Líneas:** 105

Descripción funcional Validación documentada de la calidad de los datos del padrón, con foco en atípicos de edad (`EDAD_ANOS_PR`). Documenta explícitamente la decisión metodológica (eliminación frente a imputación por mediana del grupo) para que cualquier auditor pueda reproducirla y discutirla. Expone `detectar_atipicos.edad`, `resumen_tratamiento`, `filtrar` (método elegido) e `imputar_mediana` (alternativa comparativa).

Análisis de la lógica Umbral `UMBRALEDAD_MAX=100`. `detectar_atipicos.edad` devuelve los registros con `edad>100` (mínimas columnas de identificación). `resumen_tratamiento` produce el reporte antes/después con justificación. `filtrar` aplica eliminación (decisión por defecto). `imputar_mediana` reemplaza los atípicos por la mediana del grupo (gran área, clasificación, género) como alternativa reproducible. El docstring documenta el razonamiento con la cifra de su momento ('diez registros' sobre el total entonces disponible, 0.013%); el conteo vigente en el README y en las evidencias versionadas (git HEAD) es de 22 casos sobre 77.237 (0.028%), sesgo despreciable, eliminación preferida por trazabilidad.

Lo que está bien construido

- Decisión metodológica documentada y justificada en el propio código (transparencia auditable).
- Ofrece comparación explícita entre eliminación e imputación (análisis de sensibilidad).
- Funciones puras y pequeñas, sin efectos laterales.
- El filtro se aplica solo donde el análisis depende de la edad.

Puntos de mejora (si aplican)

- Docstring dice 'diez registros' (10) mientras el conteo real documentado en README y evidencias es 22 — inconsistencia documental verificada.
- Umbral fijo (100) sin parametrización.
- `imputar_mediana` usa `apply` fila a fila (lento con 77k filas).
- El resto de anomalías (NO DISPONIBLE 5.2 %, NO REPORTADO 2.7 %, conv. 833 etiquetada 2019) se documenta en el docstring pero no se instrumenta como funciones.

Sugerencias concretas

- Corregir el docstring a 22 casos y parametrizar el umbral.
- Vectorizar `imputar_mediana` (`groupby transform` en vez de `apply`).
- Instrumentar también las otras dimensiones de calidad (completitud, consistencia, unicidad) como funciones.

Ejecutabilidad Sí, verificada empíricamente: `sprint6_validacion_calidad.py` (que lo usa) ejecutado en clon fresco con `EXIT=0` en 61.6 s. Requiere el XLSX consolidado (presente).

¿Produce lo que se buscaba? Sí: `evidencias/calidad_atipicos_edad.csv`, `calidad_comparacion.csv` y `calidad_resumen.json` existen. Matiz verificable: la versión versionada en git (HEAD) reporta 22 atípicos sobre 77.237 (0.0285 %); el working tree regenerado por la propia ejecución de la auditoría sobre el XLSX parcial (3 convocatorias, 50.891 filas) reporta 19 (0.0373 %). El tratamiento documentado (eliminación) es el mismo en ambos casos.

Riesgo / Limitación**Problemas detectados:**

- Docstring (línea ~10): 'diez registros' vs 22 reales — inconsistencia que puede confundir al lector del informe.

Valoración global mejorar — lógica correcta y bien documentada; corregir la inconsistencia documental y ampliar la instrumentación de calidad.

2.0.9 src/analisis/diversidad.py

Rol: analisis · **Líneas:** 162

Descripción funcional Análisis de variables de diversidad (Issue #20): cobertura temporal de ID_VICTIMA_CONFLICTO, TXT_GRUPO_ETNICO y TXT_POBLACION_DISCA, distribución por categoría, comparación de subrepresentación frente a DANE 2018/RUV 2021, cruce interseccional género x etnia y categorías por minoría. Entrada: DataFrame transformado (mayúsculas aplicadas por src/Transformacion.py); salida: DataFrames exportados a evidencias/diversidad_*.csv.

Análisis de la lógica Define DANE_REFERENCIA (constantes poblacionales) y VALORES_NULOS por variable. cobertura_por_convocatoria() calcula % NO REGISTRA/NO DISPONIBLE y cobertura por año. comparar_dane() (diseñada para df_2021) mapea nombres MinCiencias ->DANE con regex y calcula la razón de subrepresentación. interseccional_genero_etnia() y categoria_por_minoria() cruzan con género y categoría académica. El script sprint4_diversidad.py filtra df_2021 antes de llamar a comparar_dane.

Lo que está bien construido

- Hallazgo central documentado en el propio módulo: las variables solo se capturaron desde 2021 — honestidad metodológica.
- Razón de subrepresentación (DANE/MinCiencias) correcta y con manejo de división por cero (None).
- Constantes de referencia (DANE 2018 / RUV 2021) centralizadas y comentadas.

Puntos de mejora (si aplican)

- Contrato frágil: comparar_dane, interseccional_genero_etnia y categoria_por_minoria asumen que el caller ya filtró la convocatoria 2021; si se pasa el df completo, los resultados mezclan años sin advertencia.
- ACOPLAMIENTO ROTO CON Transformacion.py: éste convierte NaN categórico en 'NO REPORTADO', pero diversidad espera 'NO REGISTRA'/'NO DISPONIBLE'; los faltantes reales etiquetados 'NO REPORTADO' no se cuentan como nulos ->cobertura inflada.
- Bug metodológico en comparar_dane(): el comentario (línea 78) dice excluir 'NINGUN GRUPO ETNICO' del denominador, pero el código (línea 79) solo excluye 'NO DISPONIBLE'; 'NINGUN GRUPO ETNICO' queda en el denominador y diluye la subrepresentación de las minorías (verificado: comentario vs código).
- Comparación 'Sí' con tilde (línea 121): funciona con los datos actuales porque transformar() pone en mayúsculas 'Sí' ->'SÍ', pero es frágil ante variantes sin tilde ('Si'/'SI') — el catálogo documenta 'Si'.

- `interseccional_genero_etnia()` puede lanzar `KeyError` 'FEMENINO' solo si NINGÚN grupo étnico tiene mujeres (con `unstack(fill_value=0)` la columna existe si al menos un grupo tiene mujeres); caso límite, crash en runtime.

Sugerencias concretas

- Que las funciones filtren internamente por `ANO_CONVO_INT == 2021` (parametrizado) para hacer la API autónoma.
- Alinear los valores nulos entre `Transformacion` y `diversidad` (constante compartida) o contar también 'NO REPORTADO' como faltante.
- Excluir de verdad 'NINGUN GRUPO ETNICO' del denominador (alinear comentario y código) y documentar el impacto en las razones de subrepresentación.
- Normalizar 'SÍ'/'SI'/'Si' con `strip + upper` antes de comparar (o eliminar acentos).
- Usar `.reindex(columns=['FEMENINO', 'MASCULINO'], fill_value=0)` tras el `unstack` en `interseccional_genero_etnia`.

Ejecutabilidad Sí en el clon: solo `DataFrame` transformado (XLSX presente); las funciones de 2021 requieren que el caller filtre (`sprint4_diversidad.py` lo hace). Sin datos/raw ni `.duckdb`.

¿Produce lo que se buscaba? Sí: `evidencias/diversidad_cobertura_por_convocatoria.csv`, `diversidad_comparacion_dane.csv`, `diversidad_distribucion_*.csv`, `diversidad_interseccional_genero_etnia.csv` y `diversidad_categorias_por_etnia.csv` existen, junto con `artifacts/sprint4_diversidad/fig01-03.png` — consistente con el README (Issue #20, hallazgos 4-6).

Riesgo / Limitación

Problemas detectados:

- `src/analisis/diversidad.py:78-79`: contradicción comentario-código sobre 'NINGUN GRUPO ETNICO' en el denominador de `comparar_dane()` — diluye la subrepresentación de minorías (verificado por lectura directa).

Evidencia en `src/analisis/diversidad.py` (líneas 78-79):

```
1      # Etnia: excluir "NINGUN GRUPO ETNICO" del denominador
      para ver minorias
2      etnia = df_2021[df_2021["TXT_GRUPO_ETNICO"] != "NO
      DISPONIBLE"]
```

- `src/analisis/diversidad.py:145`: `d['FEMENINO']` puede lanzar `KeyError` solo si NINGÚN grupo étnico tiene mujeres (con `unstack(fill_value=0)` la columna no se crea en ese caso) — crash en caso límite (reproducido).

Evidencia en `src/analisis/diversidad.py` (línea 145):

```
1      .assign(pct_femenino=lambda d: (d["FEMENINO"] / d["
      total"] * 100).round(1))
```

- `src/analisis/diversidad.py:44-46`: los NaN convertidos a 'NO REPORTADO' por `Transforma-`

cion.py no se cuentan como nulos ->pct_no_registra subestimado y cobertura sobreestimada para registros con faltante real.

Evidencia en src/analisis/diversidad.py (líneas 44-46):

```

1      pct_nulo = (grupo[col] == val_nulo).mean() * 100
2      fila[f"{col}_pct_no_registra"] = round(pct_nulo,
3      2)
      fila[f"{col}_pct_cobertura"] = round(100 -
      pct_nulo, 2)

```

- src/analisis/diversidad.py:121: comparación 'Sí' sensible a acentos; no es un bug activo (los datos traen 'Sí' y transformar() lo pone en mayúsculas), pero fallaría si la fuente usara 'Si' sin tilde.

Evidencia en src/analisis/diversidad.py (línea 121):

```

1      n_vict = (vict["ID_VICTIMA_CONFLICTO"] == "Sí").sum()

```

Valoración global mejorar — hallazgos valiosos y bien enmarcados; prioridad alta: corregir el denominador de comparar_dane(), alinear valores nulos con Transformacion.py y robustecer comparaciones de cadenas.

2.0.10 src/analisis/genero.py

Rol: analisis · **Líneas:** 85

Descripción funcional Análisis de género por área OCDE (Issue #22): porcentaje femenino, tablas pivote y brecha de género por convocatoria a 3 niveles de área. Entrada: DataFrame transformado con NME_GENERO_PR estandarizado y ANO_CONVO_INT; salida: DataFrames tidy/pivot listos para exportar a evidencias/genero_*.csv y graficar.

Análisis de la lógica pct_femenino_por_area() filtra género en {FEMENINO, MASCULINO} (excluye NO REPORTADO del denominador, decisión documentada), agrupa por (año, área), hace value_counts().unstack() y calcula n_femenino, n_masculino, n_total y pct. tabla_pivot_pct_femenino() pivotea área x año con promedio y ordena; brecha_genero() deriva pct_masculino = 1 - pct_femenino y la brecha; evolucion_pct_femenino() reordena. El script sprint2_genero_ocde.py llama a estas funciones y exporta los CSVs.

Lo que está bien construido

- Decisión metodológica explícita y correcta: excluir NO REPORTADO del denominador evita sesgos de imputación.
- Funciones puras, parametrizables (col.area, col.year) y reutilizables en scripts y dashboard.

- Documenta el contrato de entrada (género estandarizado, ANO_CONVO_INT disponible).

Puntos de mejora (si aplican)

- `evolucion_pct_femenino()` es redundante (mismo DataFrame ordenado); el propio docstring lo admite.
- No aplica `dropna` sobre `col_area`: un área con NaN se pierde silenciosamente del `groupby` (aceptable pero implícito).
- `pct_femenino = n_fem/n_total` puede dividir por cero si un grupo tiene 0 registros reportados ->NaN propagado a pivote y brecha sin aviso.
- `brecha` usa `pct_masculino = 1 - pct_femenino`, válido solo dentro del universo 'reportado'; la columna `n_masculino` calculada no se aprovecha.

Sugerencias concretas

- Eliminar `evolucion_pct_femenino` o fusionarla con un parámetro `sort`.
- Añadir manejo explícito de grupos vacíos (`n_total=0` ->`pct=NaN` con advertencia).
- Escribir tests con df pequeño (área 100 % hombres, área con solo NO REPORTADO).

Ejecutabilidad Sí en el clon: solo requiere el DataFrame transformado, que se obtiene de `cargar_consolidado()` con fallback al XLSX presente. Sin dependencias de datos/raw ni `.duckdb`.

¿Produce lo que se buscaba? Sí: `evidencias/genero_pct_femenino_por_area.csv`, `genero_pct_femenino_por_gran_area.csv`, `genero_brecha_por_gran_area.csv` y `genero_tabla_pivot_gran_area.csv` existen en el clon, junto con `artifacts/sprint2_genero_ocde/fig01-04.png` — consistente con el README (Issue #22).

Riesgo / Limitación

Problemas detectados:

- `src/analisis/genero.py:32-33`: `conteos.get('FEMENINO', 0)` funciona porque `DataFrame.get` acepta default escalar, pero si un grupo queda sin filas reportadas, `n_total=0` y `pct_femenino=NaN` (división por cero) sin advertencia — caso límite real si una convocatoria tuviera 100 % NO REPORTADO.

Evidencia en `src/analisis/genero.py` (líneas 32-33):

```
1 resultado["n_femenino"] = conteos.get("FEMENINO", 0)
2 resultado["n_masculino"] = conteos.get("MASCULINO", 0)
```

- `src/analisis/genero.py:67`: `pct_masculino = 1 - pct_femenino` asume universo binario; correcto tras el filtro, pero la columna `n_masculino` ya existe y no se usa (inconsistencia menor).

Evidencia en `src/analisis/genero.py` (línea 67):

```
1 base["pct_masculino"] = 1 - base["pct_femenino"]
```

Valoración global mantener — análisis correcto y bien documentado; mejoras menores de robustez (grupos vacíos) y limpieza de redundancia.

2.0.11 `src/analisis/longitudinal.py`

Rol: analisis · **Líneas:** 207

Descripción funcional Núcleo del análisis longitudinal (Issue #11): construye el panel investigador×convocatoria a partir de ID_PERSONA_PR, compara trayectorias entre periodos consecutivos (sube/baja/se mantiene/desaparece), calcula matrices de transición de conteos y probabilidades, y tasas de retención con entrada de nuevos investigadores. Cubre las 6 convocatorias históricas (2013-2021).

Análisis de la lógica ORDEN_CATEGORIAS define la escala ordinal (Junior=1 ... Emérito=4) y normalizar_categoria mapea el texto libre a las 4 etiquetas canónicas. construir_panel deduplica (ID, año) conservando la categoría más alta. comparar_periodo hace left join desde t0 hacia t1 (los que desaparecen quedan representados) y clasifica el resultado. matriz_transicion calcula conteos (crosstab) y probabilidades normalizadas por fila, con opción de incluir 'Desaparece' como estado final. tasa_retencion_por_periodo computa retención (presentes en t0 que reaparecen) y nuevos en t1.

Lo que está bien construido

- Implementación estadística correcta: crosstab normalizado por fila = matriz de Markov empírica; left join conserva la atrición.
- Escala ordinal explícita para clasificar sube/baja/se mantiene.
- Estado 'Desaparece' tratado como opción configurable.
- Docstrings con interpretación metodológica (caveat eméritos vitalicios documentado en README/sprint6).

Puntos de mejora (si aplican)

- No se testea la propiedad markoviana ni la homogeneidad temporal; con intervalos irregulares (2013, 2014, 2015, 2017, 2019, 2021) las tasas no son directamente comparables entre periodos.
- comparar_periodo usa apply con closure fila a fila (lento con 77k filas).

- La 'desaparición' incluye a los eméritos vitalicios (diseño del sistema), que sesgan la interpretación si no se aísla (documentado en sprint6, no en el código).
- No hay análisis de supervivencia (Kaplan-Meier/Cox) para el tiempo hasta la salida.

Sugerencias concretas

- Añadir test de markovianidad y matrices condicionadas a historia.
- Vectorizar la clasificación del tracking (np.select en vez de apply).
- Complementar con modelos de supervivencia y reportar IC bootstrap de las tasas.

Ejecutabilidad Sí, verificada empíricamente: usado por `sprint2_panel_longitudinal.py` (EXIT=0, 108 s) y `sprint2_matrices_transicion.py` (EXIT=0, 74 s) en clon fresco. Requiere XLSX consolidado (presente).

¿Produce lo que se buscaba? Sí: matrices de transición (`evidencias/matriz_*.csv`), paneles y tasas versionados coinciden con la lógica; los regenerados con el clon (3 convocatorias) cubren solo 2 periodos — limitación de datos, no de código.

Riesgo / Limitación

Problemas detectados:

- Ninguno crítico; riesgo metodológico: interpretar las transiciones como cadena de Markov sin validar la propiedad (caveat recomendado).

Valoración global mantener con mejoras — lógica sólida para el alcance descriptivo; la ampliación inferencial (markovianidad, supervivencia) es la recomendación de mayor valor.

2.0.12 `src/analisis/produccion.py`

Rol: analisis · **Líneas:** 340

Descripción funcional Análisis cruzado producción <-> investigadores (Sprint 5): cobertura de reconocimiento, productividad por categoría/género/área/territorio, mix de tipologías y reconocidos sin producción. Entrada: `prod` (3.166.629 filas, Socrata 33dq-ab5a) e `inv` (consolidado transformado), cruzados por `ID_PERSONA_PD` <-> `ID_PERSONA_PR` (+ `ID_CONVOCATORIA`); salida: DataFrames exportados a `evidencias/produccion_*.csv` y consumidos por `sprint5-produccion.py`.

Análisis de la lógica `normalizar_produccion()` pone columnas en MAYÚSCULAS, convierte IDs a Int64 y extrae `ANO_CONVO_INT`. `cobertura_reconocidos()` marca `es_reconocido` por `isin` global y agrega por año; `cobertura_autores_unicos()` devuelve un dict de solapamiento. `productividad_por_categoria()/por_genero_area()` agregan `n_productos` por (persona, convocatoria) y hacen `left merge` con `inv`. `productividad_territorial()` suma por departamento con `per cápita` y `ratio de concentración`. `mix_tipologias()` y `tipologias_por_categoria()` agregan por `NME_TIPO_MEDICION_PD`. `reconocidos_sin_produccion()` cruza conjuntos de autores por `ID_CONVOCATORIA`.

Lo que está bien construido

- Responde de forma completa las 6 preguntas críticas del Sprint 5 con funciones bien nombradas y docstrings que explican la interpretación de cada métrica.
- Manejo correcto de tipos para el cruce (`to_numeric -> Int64` en ambos lados).
- Coberturas y brechas con denominadores explícitos (`pct_con_al_menos_un_producto`, `razon_fm`, `ratio_concentracion`).
- Evidencias producidas y versionadas: 7 CSVs + resumen JSON en `evidencias/`.

Puntos de mejora (si aplican)

- Requiere `datos/raw/produccion_grupos.csv` (~1,2 GB, gitignored): en el clon ninguna función de este módulo es ejecutable hasta descargar el archivo.
- `normalizar_produccion()` solo usa `pd.to_datetime` para `ANO_CONVO`: si el CSV exporta años como enteros (2013), `to_datetime` los interpreta como nanosegundos -> año 1970; no replica la estrategia multi-formato de `Transformacion.py` (riesgo latente según el formato real del export).
- `reconocidos_sin_produccion()` hace `groupby(...).apply(set)` sobre 3,2 M filas: lento y con alto uso de memoria.
- Los merges asumen granularidad única (persona, convocatoria) en `inv`; duplicados inflarían `n_productos` (sin verificación previa).
- `brecha_productividad_genero()` lanza `KeyError` si un área no tiene mujeres (columna ausente tras el pivot).

Sugerencias concretas

- Replicar el parseo multiestrategia de año de `Transformacion.py` en `normalizar_produccion()` o unificarlo en un helper compartido.
- Vectorizar `reconocidos_sin_produccion()` con `merge` sobre autores únicos por convocatoria.
- Verificar unicidad de (`ID_PERSONA_PR`, `ID_CONVOCATORIA`) en `inv` antes de `merge`.
- Reindexar columnas en `brecha_productividad_genero` para áreas sin un género.

Ejecutabilidad NO en el clon: cargar_produccion() falla porque datos/raw/produccion_grupos.csv no existe (gitignored; requiere python -m src.ingesta.produccion, ~1,2 GB y red). Las funciones son puras y ejecutables si se les pasan DataFrames, pero el flujo documentado exige el CSV crudo. No depende del .duckdb.

¿Produce lo que se buscaba? Sí (con datos): evidencias/produccion_cobertura_por_convocatoria.csv, produccion_productividad_por_categoria.csv, produccion_brecha_genero.csv, produccion_concentracion_territorial.csv, produccion_mix_tipologias.csv, produccion_tipologias_por_categoria.csv, produccion_reconocidos_sin_produccion.csv + produccion_resumen_cobertura.json + 6 figuras en artifacts/sprint5_produccion/ — todos existen en el clon y coinciden con el README (Sprint 5, hallazgos 8-10).

Riesgo / Limitación

Problemas detectados:

- src/analisis/produccion.py:43: pd.to_datetime(ANO_CONVO) sin manejo de ints/seriales -> año 1970 si el campo es numérico (inconsistente con Transformacion.py:99-116).

Evidencia en src/analisis/produccion.py (línea 43):

```
1 df["ANO_CONVO_INT"] = pd.to_datetime(df["ANO_CONVO"],
    errors="coerce").dt.year.astype("Int64")
```

Evidencia en Transformacion.py (líneas 99-116):

```
1 fecha = pd.to_datetime(s, dayfirst=True, errors="coerce")
2
3 # Intento 2: serial numérico de Excel
4 s_num = pd.to_numeric(s, errors="coerce")
5 fecha_serial = pd.to_datetime(
6     s_num, unit="D", origin="1899-12-30", errors="coerce"
7 [...])
```

- src/analisis/produccion.py:199: pivot.rename no falla si falta 'FEMENINO', pero pivot['prom_femenino'] sí lanza KeyError en áreas sin mujeres.

Evidencia en src/analisis/produccion.py (línea 199):

```
1 pivot["razon_f_m"] = (pivot["prom_femenino"] / pivot["
    prom_masculino"]).round(3)
```

- src/analisis/produccion.py:89: división por len(autores_unicos) -> ZeroDivisionError si prod está vacío (caso límite).

Evidencia en src/analisis/produccion.py (línea 89):

```
1 "pct_autores_reconocidos": round(len(overlap) / len(
    autores_unicos) * 100, 2),
```

- src/analisis/produccion.py:63-65: isin usa IDs normalizados a Int64; si id_persona_pr llega con ceros a la izquierda desde el XLSX (leído como str) y prod como int, el cruce pierde matches (depende de la inferencia de tipos).

Evidencia en src/analisis/produccion.py (líneas 63-65):

```

1     ids_reconocidos = set(inv["ID_PERSONA_PR"].dropna().
                        astype("Int64"))
2     p = prod.copy()
3     p["es_reconocido"] = p["ID_PERSONA_PD"].isin(
                        ids_reconocidos)

```

Valoración global mantener/mejorar — módulo metodológicamente sólido y bien documentado; priorizar la robustez de tipos/años y la reproducibilidad del input crudo (o versionar un parquet de producción) para que el cruce sea reproducible sin 1,2 GB.

2.0.13 src/analisis/redes.py

Rol: analisis · **Líneas:** 225

Descripción funcional Análisis de redes de co-filiación institucional (Issues #15/#16): normaliza nombres de instituciones, extrae pares de instituciones de INST_FILIA (separador '|'), construye grafos NetworkX, calcula métricas (grado, betweenness, densidad, componentes) y genera HTML interactivo con Pyvis. Entrada: DataFrame transformado con INST_FILIA; salida: DataFrames de pares/nodos/aristas (evidencias/redes_*.csv) y HTML en hallazgos/.

Análisis de la lógica normalizar_institucion() aplica heurística de paréntesis (si el paréntesis contiene una institución madre, lo usa; si no, lo elimina), elimina sufijos SEDE/SECCIONAL y normaliza mayúsculas. construir_pares() filtra filas con '|', divide en 2 partes (n=1), normaliza, elimina self-loops y ordena pares para que (A,B)==(B,A). construir_grafo() agrega pesos por grupo (a,b). metricas_grafo(), tabla_nodos(), tabla_aristas(), metricas_por_convocatoria() y subgrafo_grado_minimo() componen el resto. generar_html_pyvis() pinta nodos por grado ponderado y betweenness.

Lo que está bien construido

- Normalización institucional con reglas documentadas y heurística de paréntesis — valor real frente a las 2.762 cadenas crudas de inst_filia.
- Eliminación de self-loops tras normalizar y orden canónico de pares: evita aristas duplicadas (A,B)==(B,A).
- Módulo completo (construcción, métricas, subgrafos, visualización) con lazy import de pyvis.
- Artefactos versionados en evidencias/redes_*.csv y artifacts/sprint3_redes/.

Puntos de mejora (si aplican)

- **CRÍTICO:** `construir_pares()` divide con `n=1`, capturando solo el PRIMER par de investigadores con 3+ instituciones; se pierden las combinaciones restantes (A,C), (B,C) -> el grafo subestima la co-filiación.
- El grafo global (`anio=None`) pondera por filas investigador x convocatoria: un investigador con el mismo par en 2 convocatorias cuenta doble, inflando el peso de aristas 'investigadores compartidos'.
- `import networkx` en mitad del archivo (línea 49) y un `import networkx as nx_local` sin uso dentro de `generar_html_pyvis` — estilo e higiene.
- Si `INST_FILIA` termina en `'|'` (cadena vacía tras `split`), se crea un nodo con nombre `"` en el grafo (no se filtra la cadena vacía, a diferencia de `modelo/dimensional.py`).
- El bloque `'if G.has_edge'` en `construir_grafo` es código muerto (cada grupo (a,b) es único en el `groupby`).

Sugerencias concretas

- Usar `itertools.combinations` sobre las instituciones separadas para generar todos los pares de investigadores multi-filiados.
- Agregar peso por investigadores únicos (`drop_duplicates` por `ID_PERSONA.PR` antes de contar) en el grafo global.
- Filtrar cadenas vacías y NaN después del `split` (`strip + dropna + != ""`).
- Mover los imports al inicio del módulo; eliminar `nx_local` no usado.

Ejecutabilidad Sí en el clon (solo DataFrame transformado, XLSX presente); requiere `networkx` y, para generar `.html_pyvis`, `pyvis` (ambos declarados en `pyproject.toml`). Sin datos/raw ni `.duckdb`.

¿Produce lo que se buscaba? Sí: `evidencias/redes_pares_cofiliacion.csv`, `redes_nodos_global.csv`, `redes_aristas_global.csv` y `redes_metricas_por_convocatoria.csv` existen, junto con `artifacts/sprint3_redes/fig01-03.png`; los HTML Pyvis los genera `sprint3_grafo_interactivo.py`. Coincide con el README (Issues #15/#16).

Riesgo / Limitación**Problemas detectados:**

- `src/analisis/redes.py:62-64`: `str.split('|', n=1)` descarta afiliaciones 3+; red incompleta (subestimación de aristas y nodos).

Evidencia en `src/analisis/redes.py` (líneas 62-64):

```

1 multi[["inst_a", "inst_b"]] = (
2     multi["INST_FILIA"]
3     .str.split("|", n=1, expand=True)
```

- `src/analisis/redes.py:74-79`: `sorted()` sobre pares que contienen " (de 'A | ') crea un nodo vacío en el grafo; con tipos no comparables podría lanzar `TypeError`.

Evidencia en `src/analisis/redes.py` (líneas 74-79):

```

1      multi = multi[multi["inst_a"] != multi["inst_b"]].copy()
2      # Normalizar orden para que (A,B) == (B,A)
3      multi[["inst_a", "inst_b"]] = pd.DataFrame(
4          [sorted([a, b]) for a, b in zip(multi["inst_a"],
5              multi["inst_b"])],
6          index=multi.index,
7      )

```

- `src/analisis/redes.py:93-98`: el peso del grafo global cuenta filas investigador x convocatoria, no investigadores únicos: 'investigadores compartidos' queda inflado entre años.

Evidencia en `src/analisis/redes.py` (líneas 93-98):

```

1      for (a, b), grp in pares.groupby(["inst_a", "inst_b"]):
2          peso = len(grp)
3          if G.has_edge(a, b):
4              G[a][b]["weight"] += peso
5          else:
6              G.add_edge(a, b, weight=peso)

```

- `src/analisis/redes.py:16-48`: `normalizar_institucion` devuelve el valor original (posiblemente sin estandarizar) en el caso base.

Evidencia en `src/analisis/redes.py` (líneas 16-48):

```

1  def normalizar_institucion(nombre) -> str:
2      """
3      Colapsa variantes (sedes, parentesis, mayusculas) a una
4      sola entidad.
5
6      Reglas:
7      - Si hay parentesis y contiene una institucion (
8          Universidad/Instituto/etc),
9      [...]

```

Valoración global refactorizar — análisis con buen valor analítico, pero la pérdida de pares multi-filiación y el conteo duplicado de pesos comprometen las métricas publicadas (`n_aristas`, peso de aristas); corregir antes de citar resultados de red.

2.0.14 `src/analisis/territorial.py`

Rol: analisis · **Líneas:** 107

Descripción funcional Análisis de concentración territorial (Issue #13): índice Herfindahl-Hirschman (HHI) por convocatoria, top territorios y tabla de cuotas departamento x año. Entrada: DataFrame transformado; salida: DataFrames exportados a evidencias/territorial_*.csv.

Análisis de la lógica `hhi()` normaliza conteos a cuotas y suma sus cuadrados (devuelve NaN si la suma es 0). `hhi_por_convocatoria()` filtra 'NO REPORTADO' y `dropna`, agrupa por año y reporta `hhi`, `n_total` y `n_territorios`. `top_territorios()` calcula conteos por (año, territorio) con `pct` sobre el total anual y filtra los `n` territorios más frecuentes del total acumulado. `tabla_cuotas()` pivotea departamento x año con las cuotas. `sprint2_territorial.py` los consume (HHI por departamento y por región).

Lo que está bien construido

- HHI implementado correctamente (cuotas normalizadas, rango [0,1]) con docstring de interpretación.
- Filtro explícito de 'NO REPORTADO' y `dropna` antes de calcular.
- Métricas auxiliares (`n_total`, `n_territorios`) dan contexto al índice.

Puntos de mejora (si aplican)

- `top_territorios` mezcla dos alcances: el 'top n' se calcula sobre el total acumulado del período, pero `pct` se calcula por año -> un territorio top en un año específico puede quedar excluido de la tabla (pérdida de información por año).
- `hhi()` con una sola unidad devuelve 1.0 (concentración total), correcto matemáticamente pero debe interpretarse junto con `n_territorios`.
- El filtro de 'NO REPORTADO' depende de que `Transformacion.py` haya etiquetado así los faltantes: acoplamiento implícito a un valor mágico.
- Sin normalización adicional de nombres de departamentos (p.ej. 'Bogotá D.C.' vs 'Bogota') más allá del upper del pipeline.

Sugerencias concretas

- Separar el alcance del top-n (por año o global) en parámetros explícitos.
- Mover el filtro de 'NO REPORTADO' a una constante compartida (p.ej. en `analisis/__init__`) para evitar valores mágicos.
- Añadir HHI normalizado (HHI^*) para comparar entre años con distinto número de territorios.
- Tests: serie de conteos, suma cero, una sola unidad.

Ejecutabilidad Sí en el clon: solo requiere DataFrame transformado (XLSX presente). Sin datos/raw ni .duckdb.

¿Produce lo que se buscaba? Sí: evidencias/territorial_hhi_por_convocatoria.csv, territorial_hhi_region_por_convocatoria.csv y territorial_cuotas_departamento.csv existen, junto con artifacts/sprint2_territorial/fig01-03.png — consistente con el README (Issue #13).

Riesgo / Limitación

Problemas detectados:

- src/analisis/territorial.py:81-87: el top-n global combinado con pct anual puede producir tablas donde ningún año muestre el territorio #1 de ese año; no es error de cálculo, pero distorsiona el concepto de 'top territorios por convocatoria'.

Evidencia en src/analisis/territorial.py (líneas 81-87):

```

1     top = (
2         sub[col_geo]
3         .value_counts()
4         .head(n)
5         .index.tolist()
6     )
7     [...]
```

- src/analisis/territorial.py:44 y 71: si col_geo contuviera valores no-cadena (el NaN ya se dropea, pero p.ej. códigos numéricos), .str.strip() lanzaría AttributeError — depende del contrato de ingesta.

Evidencia en src/analisis/territorial.py (línea 44):

```

1     sub = sub[sub[col_geo].str.strip().str.upper() != "NO
        REPORTADO"]
```

Valoración global mantener — análisis correcto y exportable; mejorar la semántica del top-n y centralizar el filtro de faltantes.

2.0.15 scripts/generar_manual.py

Rol: documentacion · **Líneas:** 610

Descripción funcional Genera docs/manual.ipynb, un notebook ejecutable de 49 celdas (23 markdown + 26 código) que reproduce los doce hallazgos del observatorio de investigadores MinCienas. Construye el notebook programáticamente con nbformat: va apilando celdas markdown y de código mediante los helpers md()/code() y las escribe en docs/ al final. Cada sección (1-14) combina texto explicativo con cifras del observatorio y celdas que reutilizan el pipeline moderno (src/ingesta, src/Transformacion.py, src/analisis/*) en lugar de duplicar lógica.

Análisis de la lógica El script no ejecuta análisis: solo ensambla el notebook. Declara ROOT como el padre del directorio del script y crea docs/ con mkdir(exist_ok=True). Luego encadena 14 secciones: portada, setup (celda que inserta src/ en sys.path), carga de datos (from ingesta import cargar_consolidado, cargar_produccion; transformar; normalizar_produccion), y secciones temáticas que importan funciones de src/analisis (calidad, longitudinal, territorial, genero, diversidad, produccion). Incrusta figuras mediante IFrame con rutas relativas a artifacts/sprint6_sankey/*.html y CSV ya generados en evidencias/ (tabla_maestra_ies.csv, geografia_*.csv, ocde_*.csv). Cuatro celdas lanzan subprocess.run(["python3", "scripts/sprint6_*.py"], cwd=ROOT, check=True) para regenerar Sankeys y CSVs. Al final, main() crea nbformat.v4.new_notebook() con metadata kernelspec python3 y escribe el JSON en docs/manual.ipynb con codificación utf-8.

Lo que está bien construido

- Estructura clara y reproducible: 49 celdas en 14 secciones numeradas, con markdown explicativo de cada hallazgo, incluyendo citas textuales del director y caveats metodológicos (Emérito vitalicio, diversidad solo capturada desde 2021, comparación DANE vs población con educación superior).
- Reutiliza el pipeline moderno (ingesta, Transformacion, analisis.*) en las celdas en lugar de duplicar lógica de análisis, lo que mantiene coherencia con scripts/sprint*.py.
- Documenta explícitamente la decisión metodológica de tratamiento de atípicos (eliminación sobre imputación) con justificación cuantitativa, y ofrece la alternativa imputar_mediana aunque no se aplique por defecto.
- El notebook generado es versionable y el script es idempotente (siempre escribe el mismo manual.ipynb desde cero).
- El notebook existente en el clon coincide con lo que el script genera: 49 celdas (23 markdown + 26 código), mismo título UTF-8 correcto y mismas celdas clave (carga, subprocess, matrices) — verificado byte a byte en la sección relevante.

Puntos de mejora (si aplican)

- Cifras hardcodedas en markdown (22 atípicos de edad, 77.237 filas, 51 % Bogotá+Antioquia, 242 investigadores Antioquia→Bogotá, retención del 92 %) que nunca se recalculan; ya hay discrepancia real: el docstring de src/analisis/calidad.py dice 'diez registros' y '0.013 %', mientras el manual dice 22 y 0,03 %.
- El script asume el dataset completo de 6 convocatorias (77.237 filas), pero el clon solo dispone del XLSX de respaldo datos/tarea_join/investigadores_consolidado.xlsx con 3 convocatorias (2017, 2019, 2021) y 50.891 filas, por lo que las cifras regeneradas no coincidirían con el texto (p. ej. 'Cinco diagramas' de Sankey solo podrían ser 2 pares).
- No declara dependencias del script ni del notebook: nbformat (necesario para ejecutar el script), seaborn e IPython (importados por las celdas) no están en requirements.txt.

- Depende de subprocess con 'python3' hardcodeado en 4 celdas, lo que lo hace no portable a Windows.
- Rutas relativas en IFrame ("../artifacts/...") dependen del cwd del kernel de Jupyter; frágiles si el notebook se abre desde otro directorio.
- No valida que los CSV leídos existan ni que los subprocess tengan éxito con mensajes claros (check=True lanza excepción genérica).

Sugerencias concretas

- Corregir la celda de matrices: matrices_todos_periodos() retorna dict de tuplas (conteos, probs); usar m[0] o cambiar la función para retornar dict con claves 'conteos'/'probabilidades'.
- Reemplazar "python3" por sys.executable (o "python") en los subprocess.run para portabilidad a Windows.
- Agregar nbformat, seaborn e IPython a requirements.txt.
- Recalcular cifras clave en celdas (n_atipicos, top departamentos, flujos) y compararlas con asserts contra los valores del texto, o generar el markdown de hallazgos dinámicamente.
- Documentar que el XLSX de respaldo cubre solo 3 convocatorias y que las secciones 11-13 requieren datos/raw/produccion_grupos.csv; añadir try/except en la carga de producción con mensaje de acción clara.
- Reemplazar subprocess por llamadas directas a funciones de src/analisis dentro del notebook (más robusto y auditable), o usar rutas basadas en ROOT en lugar de rutas relativas en IFrame.

Ejecutabilidad El script en sí (python scripts/generar_manual.py) es ejecutable tal cual en el clon siempre que nbformat esté instalado (no está en requirements.txt, aunque probablemente sí en .venv por Jupyter); escribe docs/manual.ipynb y no requiere datos. El notebook generado NO es ejecutable de punta a punta en el clon: (1) la celda de carga llama cargar_produccion(), que lanza FileNotFoundError porque datos/raw/produccion_grupos.csv está ausente (gitignored), rompiendo las secciones 11-13; (2) las 4 celdas con subprocess.run(["python3", ...]) fallan en Windows (python3 no es ejecutable estándar; sería python o py); (3) la celda de matrices accede m[conteos] sobre tuplas y lanza TypeError; (4) los IFrames usan rutas relativas a artifacts/ que dependen del cwd. Los CSV de evidencias/ y los HTML de artifacts/sprint6_sankey/ existen (generados previamente con el dataset completo), por lo que las celdas de solo-lectura (IFrame, read_csv) funcionarían si las anteriores no fallaran.

¿Produce lo que se buscaba? Sí genera docs/manual.ipynb con 49 celdas (23 markdown + 26 código) y metadata kernelspec python3 / language_info 3.11. El notebook existente en el clon coincide con lo que el script generaría: misma estructura (49 celdas), mismo título (em-dash UTF-8 correcto, sin BOM) y las mismas celdas clave (carga de datasets, subprocess con python3, IFrames y las celdas bugueadas con m[conteos] y con columnas= en HHI); el notebook

no tiene outputs ni execution_count, es decir, nunca fue ejecutado. El notebook resultante NO es totalmente ejecutable: además del TypeError de m[conteos] y del FileNotFoundError de cargar_produccion(), la celda HHI lanza TypeError por el kwarg 'columna=' (el parámetro real es col_geo), usa import seaborn e IPython.display sin declarar dependencias y rutas relativas frágiles; solo la sección 1 y parcialmente la 2 serían viables de forma inmediata con el XLSX de respaldo.

Riesgo / Limitación

Problemas detectados:

- Línea 197 (celda 'Trayectoria longitudinal'): matrices_todos_periodos(src/analisis/longitudinal.py:164-179) retorna {periodo: (conteos_df, probs_df)}, pero la celda hace m[conteos] → TypeError: tuple indices must be integers or slices, not str en runtime; el notebook existente contiene el mismo bug.

Evidencia en src/analisis/longitudinal.py (líneas 164-179):

```

1 def matrices_todos_periodos(
2     panel: pd.DataFrame, incluir_desaparece: bool = True
3 ) -> dict:
4     """
5     Calcula matrices de transición para todos los pares
6     consecutivos.
7     [...]

```

- Línea 251 (celda HHI de la sección 5): hhi_por_convocatoria(inv_clean, columna="NME-DEPARTAMENTO-RES-PR") pasa el kwarg 'columna', pero el parámetro real de la función es 'col_geo' (src/analisis/territorial.py:32) → TypeError en runtime; la sección 5 no es viable aunque no requiera producción.

Evidencia en src/analisis/territorial.py (línea 32):

```

1 def hhi_por_convocatoria(

```

- Líneas 213, 318, 357 y 509: subprocess.run(["python3", "scripts/sprint6-*.py", ...]) — 'python3' no existe en Windows (el clon vive en Windows); falla con FileNotFoundError. Debería ser sys.executable o 'python'.
- Celda de carga (líneas 107-117): prod = normalizar_produccion(cargar_produccion()) lanza FileNotFoundError en el clon porque datos/raw/produccion_grupos.csv no existe; rompe secciones 11, 12 y 13.
- Línea 136-140 vs src/analisis/calidad.py:1-26: el manual afirma '22 registros con edad superior a 100 años' y '0,03%', pero el módulo de calidad documenta 'diez registros' y '0.013%' — cifras contradictorias sin recalcular.

Evidencia en src/analisis/calidad.py (líneas 1-26):

```

1 """
2 analisis/calidad.py
3
4 Validación documentada de calidad de los datos del padrón de
   investigadores

```

```

5 reconocidos. La decisión metodológica adoptada se documenta
  de forma explícita
6 para que cualquier auditor pueda reproducirla y discutirla.
7 [...]

```

- Líneas 44-47 de la portada: dice 'los doce hallazgos' pero enumera 14 secciones numeradas (1-14) — incoherencia menor de numeración.
- requirements.txt no incluye nbformat (necesario para ejecutar el script), ni seaborn ni IPython (importados por celdas del notebook).
- Celda setup (líneas 89-94): ROOT = pathlib.Path.cwd() depende del directorio desde donde se lance el kernel; el fallback a ROOT.parent solo cubre un nivel, frágil en Jupyter Lab/VS Code con cwd distinto.

Valoración global mejorar — El script cumple bien su rol documental (estructura clara, reuso de src/, notebook reproducible y coincidente con el del clon), pero no es ejecutable de punta a punta en el estado actual del repositorio: hay un bug de API real (m[conteos"]), dependencia de un CSV de producción ausente, subprocess no portables a Windows y cifras hardcodeadas que ya divergen del código de calidad. Se recomienda una mejora enfocada: corregir la celda de matrices, usar sys.executable, declarar dependencias y recargar/validar las cifras — sin necesidad de refactorizar la arquitectura del script.

2.0.16 scripts/sprint2_duckdb.py

Rol: orquestacion · **Líneas:** 127

Descripción funcional Construye el modelo dimensional estrella (7 dimensiones + fact_clasificacion + bridge.hecho_institucion) en datos/processed/observatorio.duckdb a partir del dataset consolidado transformado, y lo valida con 6 consultas de negocio (investigadores por convocatoria, distribución de categorías, top-10 instituciones, gran área, concentración Bogotá/Medellín/Cali, edad promedio por categoría y año). Usa src/ingesta.cargar_consolidado, src/Transformacion.transformar y src/modelo/dimensional (cargar_en_duckdb, resumen_modelo, DB_PATH). Salidas: la base DuckDB y evidencias/duckdb_resumen_modelo.txt.

Análisis de la lógica main() encadena [1/3] transformar(cargar_consolidado()), [2/3] cargar_en_duckdb(df, path=DB_PATH, verbose=True) + resumen_modelo(con) y [3/3] ejecución de CONSULTAS (dict nombre→SQL) con con.execute(sql).fetchdf(), imprimiendo cada resultado y acumulando lineas_reporte que se escriben al final en evidencias/duckdb_resumen_modelo.txt (encoding utf-8). La construcción del esquema y el DROP/CREATE idempotente viven en src/modelo/dimensional.py, no en el script; este solo orquesta carga, validación y persistencia del reporte. Cierra la conexión al final.

Lo que está bien construido

- Modelo estrella explícito y bien documentado en el docstring (dims, fact, bridge N:N con dim_institucion), coherente con src/modelo/dimensional.py.
- Las consultas de validación son preguntas de negocio reales (concentración territorial, edad por categoría, distribución OCDE), no solo conteos triviales; funcionan como smoke test del modelo.
- Idempotente gracias al DROP TABLE IF EXISTS + CREATE en cargar_en_duckdb: permite re-ejecución sin estado corrupto acumulado.
- El reporte de texto en evidencias es verificable y de hecho existe en el clon con el formato exacto que genera este script.

Puntos de mejora (si aplican)

- Usa duckdb fetchdf(), API deprecada desde duckdb 0.9.0 en favor de df(); con el pin duckdb ^1.0 del pyproject, una instalación reciente puede emitir DeprecationWarning o directamente fallar (riesgo de ejecutabilidad futura).
- resumen_modelo(con) solo imprime en stdout: el resumen del modelo estrella (conteos por tabla) NO queda en el reporte de evidencias, que solo captura las 6 consultas.
- Sin try/finally: si una consulta o la construcción falla, la conexión duckdb queda abierta y no hay mensaje de error contextual.
- No valida el total de registros (esperado 77.237) contra fact_clasificacion ni reporta duplicados de id_persona_pr por convocatoria.
- DB_PATH está fijado en src/modelo/dimensional.py y no hay parámetro CLI; el script no permite apuntar a otra base ni modo -solo-consultas.

Sugerencias concretas

- Reemplazar fetchdf() por df() (línea 110) y también en src/modelo/dimensional.py:283.
- Incluir el output de resumen_modelo() en lineas_reporte para que la evidencia contenga el esquema completo.
- Envolver el flujo en try/finally con con.close() y capturar excepción con mensaje accionable.
- Añadir una consulta de sanidad: COUNT(*) de fact_clasificacion $\approx$ 77.237 y verificar que ninguna dim tenga claves NULL usadas por los JOINS.
- Parametrizar DB_PATH vía argparse/entorno para reutilizar el script en entornos CI.

Ejecutabilidad Ejecutable en el clon solo tras instalar dependencias: pyproject declara duckdb ^1.0, pero el .venv del clon está vacío (solo pip/setuptools) y requirements.txt no incluye duckdb. El input sí existe: cargar_consolidado() prioriza datos/raw/investigadores_consolidado.csv (NO existe, gitignored) y cae al fallback datos/tarea_join/investigadores_

consolidado.xlsx (Sí existe). La base datos/processed/observatorio.duckdb no existe en el clon (solo .gitkeep) y el script la crea. Riesgo adicional: fetchdf() deprecado según la versión de duckdb instalada.

¿Produce lo que se buscaba? Sí, con matiz: evidencias/duckdb_resumen_modelo.txt existe en el clon y su contenido (encabezado 'Modelo estrella — observatorio.duckdb' + las 6 consultas con sus to_string) coincide exactamente con lo que genera el script, lo que demuestra que se ejecutó al menos una vez con datos. El archivo observatorio.duckdb NO está en el clon (gitignored, datos/processed solo contiene .gitkeep), por lo que esa salida no es reproducible sin re-ejecución. Coincidencia declarado-vs-generado: OK para el reporte.

Riesgo / Limitación

Problemas detectados:

- scripts/sprint2_duckdb.py:110 — con.execute(sql).fetchdf(): fetchdf() está deprecado desde duckdb 0.9.0 (reemplazado por .df()); con duckdb ^1.0 en pyproject.toml una versión reciente puede eliminarlo, rompiendo el script en runtime.

Evidencia en scripts/sprint2_duckdb.py (línea 110):

```
1 resultado = con.execute(sql).fetchdf()
```

- scripts/sprint2_duckdb.py:104 — resumen_modelo(con) imprime a stdout pero su salida no se agrega a líneas_reporte: la evidencia guardada (duckdb_resumen_modelo.txt) omite el resumen del modelo estrella que el propio docstring describe como parte de la validación.

Evidencia en scripts/sprint2_duckdb.py (línea 104):

```
1 resumen_modelo(con)
```

- scripts/sprint2_duckdb.py:97-123 — sin try/finally: cualquier excepción entre [2/3] y [3/3] deja la conexión duckdb sin cerrar y sin reporte parcial.

Evidencia en scripts/sprint2_duckdb.py (líneas 97-123):

```
1 def main() -> None:
2     print("[1/3] Cargando y transformando datos...")
3     df = transformar(cargar_consolidado())
4     print(f"          Shape: {df.shape}")
5
6     print("\n[2/3] Construyendo modelo estrella en DuckDB..."
7         )
8     [...]
```

Valoración global mantener — es la base del modelo dimensional del observatorio y sus consultas de validación son valiosas; solo requiere la migración fetchdf()→df(), incluir el resumen en el reporte y robustecer el manejo de conexión.

2.0.17 `scripts/sprint2_genero_ocde.py`**Rol:** orquestacion · **Líneas:** 182

Descripción funcional Analiza la representación femenina y la brecha de género por área OCDE (gran área, área) en las 6 convocatorias 2013-2021. Usa `src/analisis/genero` (`pct_femenino_por_area`, `tabla_pivot_pct_femenino`, `brecha_genero`, `evolucion_pct_femenino`) sobre `transformar(cargar_consolidado())`. Salidas: 4 figuras PNG en `artifacts/sprint2_genero_ocde/` (barras, heatmap gran área×convocatoria, líneas de evolución, brecha primera vs última convocatoria) y 4 CSVs en `evidencias/`.

Análisis de la lógica `main()` imprime el % femenino global (filtrando solo FEMENINO/MASCULINO), genera las 4 figuras con funciones dedicadas (`fig_barras_pct_femenino`, `fig_heatmap_pct_femenino`, `fig_lineas_evolucion`, `fig_brecha_genero`), y exporta las tablas: pct por gran área, pivot con promedio, brecha (M%-F%) y pct por área. Cada función de figura llama a la función de `analisis/genero` correspondiente y guarda con `plt.savefig(dpi=150)` tras `plt.tight_layout()`; usa `matplotlib.use('Agg')` para entornos headless.

Lo que está bien construido

- Código limpio y directo: separación clara carga→figuras→exportación, funciones de figura autocontenidas y reutilizables.
- Uso correcto de la capa analítica (`src/analisis/genero`): el script no recalcula métricas, solo orquesta y visualiza.
- Backend Agg antes de importar pyplot, correcto para ejecución por consola/CI.
- Salidas verificadas: las 4 figuras y los 4 CSVs existen en el clon, en las rutas declaradas en el docstring.

Puntos de mejora (si aplican)

- Escalas fijas que saturan la información: heatmap con `vmin=0`, `vmax=60` (`fig02`) y líneas con `ylim(0,70)` (`fig03`); si las áreas STEM superan 60% de presencia masculina, el color y la curva quedan aplastados y no se distingue entre áreas.
- `fig_brecha_genero` alinea `b0` con `reindex` sobre las áreas de la última convocatoria: si una gran área existe en `a0` pero no en `a1` (o viceversa), se produce NaN silencioso y la comparación pierde áreas sin ninguna advertencia.
- El resumen global (pct femenino total) solo se imprime en consola; no se persiste como evidencia.
- Sin validación del mínimo de registros por área/convocatoria: celdas con `n` muy bajo (posibles identificadores) se grafican igual que áreas con miles de registros.

Sugerencias concretas

- Autoeescalar el heatmap (vmin/vmax del dato o por cuantiles) o documentar por qué 60 % es el tope; igual para ylim de fig03.
- En fig_brecha_genero usar union de áreas (b0 y b1) y rellenar NaN con 0 o marcar 'sin dato', e imprimir advertencia.
- Exportar el resumen global a un CSV de evidencias.
- Anotar n por celda (o tamaño de punto) en el heatmap para visibilizar áreas con pocos investigadores.

Ejecutabilidad Ejecutable en el clon solo tras instalar dependencias (matplotlib, seaborn, pandas; .venv del clon vacío, requirements.txt no las lista aunque pyproject sí). El input existe: cargar_consolidado() usa el fallback datos/tarea_join/investigadores_consolidado.xlsx (presente). No requiere datos/raw ni duckdb. Con poetry install el script corre de punta a punta en headless.

¿Produce lo que se buscaba? Sí, coincidencia total: existen artifacts/sprint2_genero_ocde/fig01_barras_pct_femenino.png, fig02_heatmap_pct_femenino.png, fig03_lineas_evolucion.png, fig04_brecha_genero.png y evidencias/genero_pct_femenino_por_gran_area.csv, genero_tabla_pivot_gran_area.csv, genero_brecha_por_gran_area.csv, genero_pct_femenino_por_area.csv. Las 8 salidas declaradas en el docstring y en el README están presentes y con los nombres exactos, lo que indica ejecución exitosa previa.

Riesgo / Limitación

Problemas detectados:

- scripts/sprint2_genero_ocde.py:132-135 — b0.reindex(areas) con areas derivada solo de b1: si el conjunto de gran áreas difiere entre convocatorias extremas, las barras de b0 quedan en NaN (no se dibujan) sin advertencia, y las áreas exclusivas de a0 desaparecen del gráfico.

Evidencia en scripts/sprint2_genero_ocde.py (líneas 132-135):

```

1      ax.bar([i - width / 2 for i in x], b0.reindex(areas),
            width,
2          label=str(a0), color="steelblue", alpha=0.8)
3      ax.bar([i + width / 2 for i in x], b1.reindex(areas),
            width,
4          label=str(a1), color="tomato", alpha=0.8)
```

- scripts/sprint2_genero_ocde.py:81 — sns.heatmap con vmin=0, vmax=60 fijos: cualquier gran área con % femenino >60 (esperable en ciencias de la salud o educación) satura el rojo/verde de RdYlGn y pierde discriminación visual.

Evidencia en scripts/sprint2_genero_ocde.py (línea 81):

```

1      vmax=60,
```

- scripts/sprint2_genero_ocde.py:108 — ax.set_ylim(0, 70) en fig03: series que superen 70 % (posible en áreas pequeñas) quedan cortadas en el borde superior sin indicación.

Evidencia en scripts/sprint2_genero_ocde.py (línea 108):

```
1 ax.set_ylim(0, 70)
```

Valoración global mantener — análisis correcto y reproducible con salidas verificadas; las debilidades son de visualización (escalas fijas) y de robustez del alineamiento de áreas, corregibles con cambios menores.

2.0.18 scripts/sprint2_matrices_transicion.py

Rol: orquestacion · **Líneas:** 125

Descripción funcional Calcula y visualiza las matrices de probabilidad de transición entre categorías (Junior/Asociado/Senior/Emérito) para los 5 periodos consecutivos de las 6 convocatorias (2013-2021), en dos variantes: observada (solo investigadores retenidos) y extendida (incluye el estado 'Desaparece'). Usa src/analisis/longitudinal (construir_panel, comparar_periodo, matriz_transicion, matrices_todos_periodos). Salidas: 2 figuras de heatmaps en artifacts/sprint2_transiciones/ y 20 CSVs (conteos y probabilidades $\times$ ext/obs $\times$ 5 periodos) en evidencias/.

Análisis de la lógica main() construye el panel (ID_PERSONA_PR, año, categoria, orden), imprime las matrices por periodo en consola con un bucle comparar_periodo+matriz_transicion, y luego recalcula todo con matrices_todos_periodos para las figuras y la exportación. fig_heatmaps() dibuja un heatmap por periodo en una sola fila de subplots (1 $\times$ n) con sns.heatmap(annot=True, fmt='.2f', vmin=0, vmax=1). Los nombres de archivo derivan del periodo reemplazando el guion largo '-' por '_' (p. ej. matriz_probabilidades_ext_2015_2017.csv).

Lo que está bien construido

- Presenta explícitamente las dos variantes metodológicas (observada vs extendida con 'Desaparece'), lo que permite ver el sesgo de selección por deserción en lugar de ocultarlo.
- La lógica estadística está delegada en src/analisis/longitudinal (crosstab, renormalización por fila), manteniendo el script como orquestador puro.
- El manejo del guion largo '-' en nombres de archivo evita problemas de codificación de rutas en Windows.
- Salidas verificadas: las 2 figuras y los 20 CSVs de la nomenclatura actual existen en el clon.

Puntos de mejora (si aplican)

- Cómputo duplicado: el bucle de impresión (líneas 83-91) recalcula las matrices que `matrices_todos_periodos` vuelve a calcular en la línea 94-95; con 77.237 registros y 5 periodos es ineficiente y propenso a divergencias si cambia la lógica.
- La matriz 'observada' renormaliza por fila excluyendo a los desaparecidos: sus probabilidades no suman a la población real y pueden sobreestimar la movilidad; el script no advierte de esta limitación interpretativa.
- Evidencias con nomenclatura heredada: en evidencias/ coexisten archivos de una versión anterior (`matriz_transicion_extendida_*.csv`, `matriz_probabilidades_extendida_*.csv` y variantes sin prefijo `ext/obs` como `matriz_transicion_2017_2019.csv`) que este script ya no produce, duplicando y confundiendo la evidencia.
- `fig_heatmaps` asume $n \geq 1$ periodo: con un solo par de años `plt.subplots(1, n)` devolvería un Axes escalar y `zip()` fallaría (robustez marginal).
- Sin validación de que el panel contenga exactamente 6 convocatorias / 5 periodos ni de tamaños de celda mínimos.

Sugerencias concretas

- Calcular matrices una sola vez (fuera del bucle de impresión) y reutilizarlas para print, figuras y exportación.
- Añadir advertencia explícita cuando la matriz observada descarte $>X\%$ de la fila por deserción, y priorizar la lectura de la matriz extendida en el informe.
- Eliminar o archivar los CSVs legacy (`extendida_/sin prefijo`) para dejar una sola convención de nombres.
- Proteger `fig_heatmaps` para $n == 1$ (`ax = axes` si escalar) y añadir un `assert` del número de periodos esperados.
- Escribir un test que verifique que las filas de la matriz de probabilidades sumen 1.0 (`obs`) y 1.0 incluyendo `Desaparece` (`ext`).

Ejecutabilidad Ejecutable en el clon solo tras instalar dependencias (`pandas`, `matplotlib`, `seaborn`; `.venv` del clon vacío, `requirements.txt` incompleto). El input existe: `datos/tarea_join/investigadores_consolidado.xlsx` vía `cargar_consolidado()` (fallback del CSV raw inexistente). No requiere `duckdb` ni `datos/raw`. Con `poetry install`, corre de punta a punta.

¿Produce lo que se buscaba? Sí: existen `artifacts/sprint2_transiciones/fig01_heatmaps_ext.png` y `fig02_heatmaps_obs.png`, y en evidencias/ los 20 CSVs con la nomenclatura actual (`matriz_transicion_ext_/matriz_probabilidades_ext_/matriz_transicion_obs_/matriz_probabilidades_obs_ × 2013.2014, 2014.2015, 2015.2017, 2017.2019, 2019.2021`), coincidiendo con lo declarado (`evidencias/matriz_transicion_*.csv`, `matriz_probabilidades_*.csv`). Adicionalmente hay 8 archivos de nomenclatura antigua (`extendida_/sin prefijo`) que evidencian ejecuciones previas con una versión distinta del script.

Riesgo / Limitación**Problemas detectados:**

- scripts/sprint2_matrices_transicion.py:83-95 — las matrices se calculan dos veces (bucle de impresión y matrices_todos_periodos): doble costo de cómputo sobre 77.237 registros y riesgo de que las cifras impresas difieran de las exportadas si la función cambiara entre ambas llamadas.

Evidencia en scripts/sprint2_matrices_transicion.py (líneas 83-95):

```

1     for a0, a1 in zip(anios, anios[1:]):
2         comp = comparar_periodo(panel, a0, a1)
3         _, probs_obs = matriz_transicion(comp,
4             incluir_desaparece=False)
5         _, probs_ext = matriz_transicion(comp,
6             incluir_desaparece=True)
7         print(f"\n{'='*55}")
8         print(f"Periodo {a0}-{a1} -- Observada (solo retenidos
9             ):")
10    [...]
```

- scripts/sprint2_matrices_transicion.py:49 — plt.subplots(1, n) con n=1 devuelve un solo Axes (no iterable); el zip(axes, periodos) fallaría con TypeError si el panel tuviera un único periodo.

Evidencia en scripts/sprint2_matrices_transicion.py (línea 49):

```

1     fig, axes = plt.subplots(1, n, figsize=(5 * n, 4))
```

- Evidencias/ legacy: los archivos matriz_transicion_extendida_2017_2019.csv, matriz_probabilidades_extendida_2019_2021.csv y matriz_transicion_2017_2019.csv (sin prefijo) ya no son generados por este script; quedan como evidencia huérfana que puede confundir la trazabilidad.

Valoración global mantener — es la entrega central del análisis longitudinal y produce evidencia verificable en el clon; requiere eliminar el cómputo duplicado, advertir el sesgo de la matriz observada y limpiar la nomenclatura legacy.

2.0.19 scripts/sprint2_panel_longitudinal.py

Rol: orquestacion · **Líneas:** 229

Descripción funcional Orquesta el análisis longitudinal del Sprint 2 (Issue #11): carga el consolidado (cargar_consolidado + transformar), construye el panel de investigadores por convocatoria con src/analisis/longitudinal.py, genera 7 figuras (evolución total, por género, por gran área, categorías, nuevos ingresos, tracking por periodo, tasas de retención) y exporta 4+ CSVs de evidencias.

Análisis de la lógica `main()` en 4 pasos: [1/4] carga y transforma; [2/4] genera figuras descriptivas (funciones `fig_*` con `matplotlib/seaborn`, backend `Agg`); [3/4] construye el panel (`construir_panel`), calcula tracking (`tracking_todos_periodos`) y retención (`tasa_retencion_por_periodo`); [4/4] exporta resumen, ingresos, tasas y tracking por categoría a evidencias/. Patrón claro: una función por figura, salidas documentadas en el docstring.

Lo que está bien construido

- Docstring con uso y lista explícita de salidas (`artifacts/` y evidencias/).
- Delega la lógica en `src/analisis/longitudinal.py` (separación de responsabilidades correcta).
- Usa `matplotlib.use('Agg')` (headless, reproducible).
- Maneja los 5 periodos consecutivos de las 6 convocatorias de forma genérica (zip de años).

Puntos de mejora (si aplican)

- Estilo global de `seaborn/matplotlib` aplicado al módulo (efecto secundario al importar).
- Sin tests ni manejo de errores (si `cargar_consolidado` fallara, crash sin mensaje claro).
- Figuras con tamaños fijos (`figsize`) poco adaptables a distintos formatos de reporte.
- No guarda los DataFrames intermedios (`panel`) para reutilización.

Sugerencias concretas

- Extraer la configuración de estilo a una función o archivo de tema.
- Añadir un pequeño test de humo (`panel` con datos sintéticos).
- Permitir configurar `figsize` por parámetro y guardar el panel en evidencias/.

Ejecutabilidad Sí, verificada empíricamente en la auditoría 2026: ejecutado en un clon fresco (`venv Python`, `pandas 3.0.5`) con `EXIT=0` en 108.4 s. Requiere el XLSX de datos/tarea_join (presente). NOTA: al correr sobre el XLSX versionado (3 convocatorias) genera solo 2 periodos de tracking, frente a los 5 periodos de las evidencias versionadas — consecuencia del consolidado incompleto, no del script.

¿Produce lo que se buscaba? Sí: genera las 7 figuras en `artifacts/sprint2_longitudinal/` (versionadas en el clon) y los CSVs en evidencias/ (`panel_longitudinal_resumen_todos_periodos.csv`, `ingresos_por_convocatoria`, `tasas_retencion`, `panel_longitudinal_cat_*.csv`). Los artefactos existentes coinciden con lo declarado, pero con 6 convocatorias (los regenerados con el clon tienen 3).

Riesgo / Limitación**Problemas detectados:**

- Ninguno crítico; dependencia implícita de que el consolidado tenga las 6 convocatorias para reproducir los 5 periodos del README (problema de datos, no de código).

Valoración global **mantener** — **bien estructurado y verificable; solo requiere el consolidado completo de 6 convocatorias para reproducir los resultados del README.**

2.0.20 scripts/sprint2_territorial.py

Rol: orquestacion · **Líneas:** 142

Descripción funcional Analiza la concentración territorial de los investigadores con el índice de Herfindahl-Hirschman (HHI) por departamento y por región para las 6 convocatorias (2013-2021), más las cuotas (%) por departamento y el top-10 de departamentos. Usa `src/analisis/territorial` (`hhi_por_convocatoria`, `top_territorios`, `tabla_cuotas`) sobre `transform(cargar_consolidado())`. Salidas: 3 figuras PNG en `artifacts/sprint2_territorial/` y 3 CSVs en `evidencias/`.

Análisis de la lógica `main()` genera `fig_top_departamentos` (barras apiladas por convocatoria del top 10 con pct por año), calcula e imprime el HHI departamental y regional, genera `fig_hhi_evolucion` (2 subplots lado a lado con anotaciones de valor) y `fig_heatmap_cuotas` (heatmap % por departamento×convocatoria, top 15), que además retorna `cuotas_pivot` para exportar. Exporta `territorial_hhi_por_convocatoria.csv`, `territorial_hhi_region_por_convocatoria.csv` y `territorial_cuotas_departamento.csv`.

Lo que está bien construido

- Usa correctamente el HHI normalizado a [0,1] sobre cuotas reales (cálculo delegado en `src/analisis/territorial.hhi`), con exclusión de valores NO REPORTADO.
- Triple perspectiva complementaria: top-10, evolución HHI (depto y región) y cuotas completas → buena triangulación del hallazgo de concentración.
- Las figuras anotan los valores exactos de HHI, facilitando la lectura sin tabla auxiliar.
- Salidas verificadas: las 3 figuras y los 3 CSVs existen en el clon en las rutas declaradas.

Puntos de mejora (si aplican)

- `fig_hhi_evolucion` mezcla tipos en las coordenadas de anotación: el eje X se plotea con strings (`astype(str)`) pero `annotate` recibe `str(int(row['ANO_CONVO_INT']))`,

dependiendo de la conversión categórica implícita de matplotlib; frágil ante cambios de versión y de difícil depuración.

- `int(row['ANO_CONVO_INT'])` lanzaría `ValueError` si hubiera `NaN` en el año (los filtros de `hhi_por_convocatoria` no garantizan limpieza total del año).
- El HHI por región depende de la columna `NME_REGION_RES_PR` derivada en `Transformacion`; el script no verifica que exista, fallando con `KeyError` poco descriptivo si la transformación cambia.
- Sin validación del tamaño muestral por año (el HHI de un año con pocos registros no es comparable) ni intervalo/estabilidad del índice.

Sugerencias concretas

- Anotar usando las mismas posiciones categóricas del plot (p. ej. `range(len(data))` o `plt.xticks` con los strings) en lugar de pasar el string como coordenada.
- Proteger con `.dropna()` sobre `ANO_CONVO_INT` antes del gráfico y validar la presencia de `NME_REGION_RES_PR` con un mensaje claro.
- Exportar también `n_total` y `n_territorios` junto al HHI (ya los retorna `hhi_por_convocatoria`) para contextualizar la evolución.
- Añadir test de humo: HHI Bogotá+Antioquia $\approx 51\%$ de concentración reportado en el README.

Ejecutabilidad Ejecutable en el clon solo tras instalar dependencias (pandas, matplotlib, seaborn; `.venv` del clon vacío, `requirements.txt` incompleto). El input existe: `datos/tarea_join/investigadores consolidado.xlsx` vía `cargar_consolidado()`. No requiere `duckdb`. Con `poetry install`, corre de punta a punta; el único supuesto frágil es la columna `NME_REGION_RES_PR` producida por `Transformacion`.

¿Produce lo que se buscaba? Sí, coincidencia total: existen `artifacts/sprint2_territorial/fig01_top_departamentos.png`, `fig02_hhi_evolucion.png`, `fig03_heatmap_cuotas.png` y `evidencias/territorial_hhi_por_convocatoria.csv`, `territorial_hhi_region_por_convocatoria.csv`, `territorial_cuotas_departamento.csv`, todos con los nombres exactos del docstring y del README, lo que confirma ejecución previa exitosa.

Riesgo / Limitación

Problemas detectados:

- `scripts/sprint2_territorial.py:82` — `ax.annotate` con `x = str(int(row['ANO_CONVO_INT']))`: se mezcla una cadena como coordenada de datos en un eje categórico (ploteado con `astype(str)`); funciona solo por la conversión unitaria implícita de matplotlib y es un punto de fallo frágil.

Evidencia en `scripts/sprint2_territorial.py` (línea 82):

```
1 (str(int(row["ANO_CONVO_INT"])), row["hhi"]),
```

- scripts/sprint2_territorial.py:82 — `int(row['ANO_CONVO_INT'])` lanza `ValueError` si el año es `NaN`; `hhi_por_convocatoria` filtra geo y año (dropna en sub), pero el `DataFrame` impreso/graficado aquí no garantiza ausencia de `NaN` en otras filas.

Evidencia en scripts/sprint2_territorial.py (línea 82):

```
1 (str(int(row["ANO_CONVO_INT"])), row["hhi"]),
```

- scripts/sprint2_territorial.py:123 — accede a `NME_REGION_RES_PR` sin verificar su existencia: si `Transformacion` renombra/elimina esa columna, el script cae con `KeyError` sin mensaje orientativo.

Evidencia en scripts/sprint2_territorial.py (línea 123):

```
1 hhi_region = hhi_por_convocatoria(df, col_geo="
    NME_REGION_RES_PR")
```

Valoración global **mantener** — el análisis **HHI** es metodológicamente sólido y las salidas están verificadas; corregir las anotaciones del gráfico de evolución y blindar la dependencia de la columna de región.

2.0.21 scripts/sprint3_grafo_interactivo.py

Rol: orquestación · **Líneas:** 72

Descripción funcional Genera dos visualizaciones interactivas HTML (Pyvis) del grafo de co-filiación institucional: el grafo completo y un subgrafo filtrado con grado ≥ 2 . Usa `src/analisis/redes` (`construir_pares`, `construir_grafo`, `subgrafo_grado_minimo`, `generar_html_pyvis`, `metricas_grafo`) sobre `transformar(cargar_consolidado())`. Salidas: `hallazgos/sprint3_grafo_completo.html` (docstring: 498 nodos) y `hallazgos/sprint3_grafo_filtrado.html` (146 nodos).

Análisis de la lógica `main()` construye los pares de doble afiliación (`construir_pares` sin filtro de año), el grafo global (`construir_grafo` con `anio=None`), el subgrafo de grado mínimo, imprime las métricas de ambos y genera los dos HTML con `generar_html_pyvis`, que en `src/analisis/redes.py` calcula `betweenness` (`weight='weight'`), escala tamaño por grado ponderado y color por `betweenness`. No exporta CSVs ni figuras estáticas; sus únicos artefactos son los dos HTML en `hallazgos/`.

Lo que está bien construido

- Entrega un artefacto de alto valor de comunicación (grafo interactivo navegable) y lo versiona en `hallazgos/` como entregable, no como figura estática.
- Delega toda la lógica de red en `src/analisis/redes` (normalización de instituciones, pesos, `betweenness`), manteniendo el script mínimo y legible.

- El filtrado por grado ≥ 2 elimina nodos hoja y produce un subgrafo interpretable; las métricas de ambos grafos se imprimen para contraste.
- Salidas verificadas: ambos HTML existen en el clon y están versionados en git.

Puntos de mejora (si aplican)

- Los títulos de los HTML hardcodean '(2019)' (líneas 53 y 62) pero el grafo se construye con TODAS las convocatorias (`construir_pares(df)` sin filtro de año y `construir_grafo` con `anio=None`): la etiqueta contradice el dato real (co-filiación global 2013-2021) y es engañosa para quien abre el HTML.
- Los conteos '498 nodos' y '146 nodos' del docstring son literales de una corrida puntual; al re-ejecutar con datos actualizados el docstring queda desactualizado (drift documental).
- No genera evidencias tabulares (pares, métricas) propias: quien quiera verificar los 498/146 nodos debe re-ejecutar o abrir el HTML.
- Sin parámetro de año: no permite generar el grafo por convocatoria (p. ej. solo 2019) aunque la capa analítica lo soporta (`construir_grafo` acepta `anio`).

Sugerencias concretas

- Eliminar el año hardcodeado de los títulos o calcularlo dinámicamente (p. ej. '2013-2021' derivado del rango real del panel) para no contradecir los datos.
- Derivar los conteos de nodos del docstring dinámicamente o quitar los números literales.
- Exportar además evidencias/redes_grafo_metricas.csv (m completo y m filtrado) para trazabilidad de los 498/146 nodos.
- Añadir un flag `-anio` para reproducir la etiqueta 2019 de forma honesta (filtrando el grafo por convocatoria).

Ejecutabilidad Ejecutable en el clon solo tras instalar dependencias: `pyproject` declara `networkx ^3.2` y `pyvis ^0.3`, pero el `.venv` del clon está vacío (solo `pip/setuptools`) y `requirements.txt` no los lista. El input existe: `datos/tarea_join/investigadores_consolidado.xlsx` vía `cargar_consolidado()`. No requiere `duckdb`. Con `poetry install`, corre de punta a punta y sobrescribe los HTML existentes.

¿Produce lo que se buscaba? Sí: `hallazgos/sprint3_grafo_completo.html` y `hallazgos/sprint3_grafo_filtrado.html` existen en el clon (rutas exactas del docstring y del README, que documenta 'hallazgos/ — HTMLs interactivos (grafos Pyvis)'), lo que confirma ejecución previa. Los conteos 498/146 nodos no son verificables por análisis estático sin abrir los HTML, pero la existencia y el versionado de ambos archivos es coherente con una corrida exitosa. El script no declara ni produce figuras PNG ni CSVs, y así es.

Riesgo / Limitación**Problemas detectados:**

- scripts/sprint3_grafo_interactivo.py:53 y 62 — títulos hardcodeados 'Co-filiación institucional — Grafo completo (2019)' y '(2019)': el grafo se construye con todas las convocatorias (construir_pares(df) línea 39 y construir_grafo(pares) línea 40 con anio=None en src/analisis/redes.py:83-90), por lo que la etiqueta 2019 es incorrecta y contradice el alcance real del grafo.

Evidencia en scripts/sprint3_grafo_interactivo.py (línea 53):

```
1      titulo="Co-filiación institucional -- Grafo completo
      (2019)",
```

Evidencia en src/analisis/redes.py (líneas 83-90):

```
1  def construir_grafo(pares: pd.DataFrame, anio: int = None) ->
    nx.Graph:
2      """
3      Construye grafo de co-filiación a partir de los pares.
4
5      Si anio != None filtra por convocatoria. Peso de arista =
        n investigadores compartidos.
6      """
7  [...]
```

- scripts/sprint3_grafo_interactivo.py:10-11 — el docstring fija '498 nodos' y '146 nodos' como constantes; si el dataset cambia, el docstring y los HTML pueden divergir sin ningún aviso.

Evidencia en scripts/sprint3_grafo_interactivo.py (líneas 10-11):

```
1      hallazgos/sprint3_grafo_completo.html      (498 nodos)
2      hallazgos/sprint3_grafo_filtrado.html      (146 nodos,
        grado >= 2)
```

Valoración global mantener — el entregable interactivo es valioso y está versionado; corregir el año hardcodeado en los títulos (el bug más visible) y desacoplar los conteos del docstring para evitar drift documental.

2.0.22 scripts/sprint3_redes.py

Rol: orquestación · **Líneas:** 134

Descripción funcional Análisis de redes de co-filiación institucional: grafo donde nodo = institución y arista = investigadores con doble afiliación, a nivel global (2013-2021) y por convocatoria. Usa src/analisis/redes (construir_pares, construir_grafo, metricas_grafo, tabla_nodos, tabla_aristas, metricas_por_convocatoria) sobre transformar(cargar_consolidado()).

Salidas: 3 figuras PNG en `artifacts/sprint3/redes/` (distribución de grado, evolución de aristas/nodos, top-20 instituciones por grado ponderado) y 4 CSVs en `evidencias/` (pares, nodos, aristas, métricas por convocatoria).

Análisis de la lógica `main()` extrae los pares de doble afiliación (solo INST_FILIA con '|'), construye el grafo global con peso = investigadores compartidos, imprime métricas (nodos, aristas, densidad, componentes, hub), genera 3 figuras con funciones dedicadas y exporta 4 CSVs. `metricas_por_convocatoria` reutiliza `construir_grafo(pares, anio)` para la evolución anual. La normalización de instituciones (colapsar sedes/seccionales) ocurre en `src/analisis/redes.normalizar_institucion`.

Lo que está bien construido

- Cobertura completa del análisis de red: grafo global, evolución por convocatoria, top instituciones y exportación tabular de nodos/aristas para reproducibilidad.
- El uso de grado ponderado (`weight`) en nodos y aristas refleja correctamente el conteo de investigadores compartidos.
- La normalización de instituciones (sedes, paréntesis, mayúsculas) está delegada y documentada en la capa analítica, evitando duplicados espurios.
- Salidas verificadas: las 3 figuras y los 4 CSVs existen en el clon en las rutas declaradas.

Puntos de mejora (si aplican)

- Línea 102: 'instituciones mencionadas' = `nunique(inst_a) + nunique(inst_b)` cuenta dos veces toda institución que aparece en ambas columnas; la cifra impresa está sobreestimada (debería ser la unión).
- `fig_distribucion_grado` dibuja una barra por nodo sin etiquetas de eje X legibles: con cientos de nodos la figura es poco informativa; un histograma log-log del grado sería más analítico.
- No reporta métricas de comunidad (modularidad, componentes gigantes) pese a imprimir `n_componentes`; el análisis de 'poder institucional' queda en grado/betweenness solamente.
- Depende de la columna INST_FILIA con separador '|': si Transformacion cambia el formato (p. ej. ';'), `construir_pares` devuelve un DataFrame vacío sin advertencia.

Sugerencias concretas

- Corregir el conteo de instituciones con `pd.unique(concat(inst_a, inst_b))` o una unión de conjuntos.
- Sustituir las barras por nodo por un histograma de distribución de grado (escala log-log) y anotar el grado medio.

- Añadir detección de comunidad (`nx.community.louvain_communities`) y tamaño del componente gigante al resumen.
- Validar en `construir_pares` que `INST_FILIA` contenga `'|'` y advertir si el número de pares es 0 o anómalo.

Ejecutabilidad Ejecutable en el clon solo tras instalar dependencias (`pandas`, `matplotlib`, `seaborn`, `networkx`; `.venv` del clon vacío, `requirements.txt` no los lista pero `pyproject` sí). El input existe: `datos/tarea_join/investigadores_consolidado.xlsx` vía `cargar_consolidado()`. No requiere `duckdb`. Con `poetry install`, corre de punta a punta.

¿Produce lo que se buscaba? Sí, coincidencia total: existen `artifacts/sprint3_redes/fig01_distribucion_grado.png`, `fig02_evolucion_aristas.png`, `fig03_top_instituciones.png` y `evidencias/redes_pares_cofiliacion.csv`, `redes_nodos_global.csv`, `redes_aristas_global.csv`, `redes_metricas_por_convocatoria.csv`, con los nombres exactos del docstring y del README; confirma ejecución previa exitosa.

Riesgo / Limitación

Problemas detectados:

- `scripts/sprint3_redes.py:102` — `pares['inst_a'].nunique() + pares['inst_b'].nunique()` sobreestima 'instituciones mencionadas' al sumar cardinalidades de columnas que comparten elementos; la cifra impresa en consola no es el número real de instituciones.

Evidencia en `scripts/sprint3_redes.py` (línea 102):

```
1 print(f"          {pares['inst_a'].nunique() + pares['inst_b']
      '}.nunique()} instituciones mencionadas")
```

- `scripts/sprint3_redes.py:48-57` — `fig_distribucion_grado` genera una barra por nodo (`len(grados)` barras) sin eje X útil: con cientos de instituciones la figura se vuelve ilegible y aporta poco frente a un histograma de la distribución de grado.

Evidencia en `scripts/sprint3_redes.py` (líneas 48-57):

```
1 def fig_distribucion_grado(G) -> None:
2     grados = sorted([d for _, d in G.degree()], reverse=True)
3     fig, ax = plt.subplots(figsize=(10, 4))
4     ax.bar(range(len(grados)), grados, color="steelblue",
5           width=1.0)
6     ax.set_xlabel("Nodo (ordenado por grado)")
7     ax.set_ylabel("Grado")
8     [...]
```

- `scripts/sprint3_redes.py:100` — `construir_pares(df)` asume implícitamente el separador `'|'` en `INST_FILIA` (`src/analisis/redes.py:61-64`); si la transformación cambia el formato, el grafo queda vacío sin ningún warning.

Evidencia en `scripts/sprint3_redes.py` (línea 100):

```
1 pares = construir_pares(df)
```

Evidencia en src/analisis/redes.py (líneas 61-64):

```

1 multi = df[df["INST_FILIA"].str.contains("|", na=False,
2         regex=False)].copy()
3 multi[["inst_a", "inst_b"]] = (
4     multi["INST_FILIA"]
5     .str.split("|", n=1, expand=True)

```

Valoración global mantener — es el análisis de redes estándar del observatorio con salidas verificadas y evidencia completa; corregir el doble conteo impreso y mejorar la figura de distribución de grado.

2.0.23 scripts/sprint4_diversidad.py

Rol: orquestacion · **Líneas:** 172

Descripción funcional Analiza las variables de diversidad (víctima de conflicto, grupo étnico, discapacidad) comparando a los investigadores reconocidos en la convocatoria 2021 con proporciones poblacionales de referencia DANE 2018 / RUV 2021. Usa src/analisis/diversidad (cobertura_por_convocatoria, distribucion_categoria, comparar_dane, interseccional_genero_etnia, categoria_por_minoria) sobre transformar(cargar_consolidado()). Salidas: 3 figuras PNG en artifacts/sprint4_diversidad/ y 7 CSVs en evidencias/ (uno más que los 6 del docstring).

Análisis de la lógica main() calcula la cobertura temporal de las 3 variables (solo se capturan desde 2021, hallazgo crítico documentado en el docstring), filtra df_2021 = df[ANO_CONVO_INT == 2021], imprime las distribuciones excluyendo NO DISPONIBLE/NO REGISTRA, ejecuta comparar_dane (subrepresentación de minorías vs DANE/RUV con razón pct_dane/pct_minc) y genera 3 figuras + exporta 7 CSVs. fig_categorias_por_minoria apila la distribución de categorías académicas por grupo étnico minoritario.

Lo que está bien construido

- El hallazgo metodológico central está bien documentado: las variables de diversidad solo existen desde la convocatoria 894 (2021), por lo que todo el análisis se acota honestamente a 2021.
- La comparación con DANE 2018 / RUV 2021 con razón de subrepresentación es metodológicamente apropiada y da un resultado de política pública accionable.
- Exporta evidencia completa: cobertura, distribuciones por variable, comparación DANE, cruce interseccional género×etnia y categorías por etnia.
- Salidas verificadas: las 3 figuras y los 7 CSVs existen en el clon.

Puntos de mejora (si aplican)

- Drift docstring-código: el docstring declara 6 CSVs pero el código escribe 7 (agrega diversidad_distribucion_victimas_2021.csv en la línea 158, no listado).
- Bug metodológico heredado de src/analisis/diversidad.comparar_dane (líneas 78-80): el comentario afirma 'excluir NINGUN GRUPO ETNICO del denominador para ver minorías', pero el código solo excluye NO DISPONIBLE; quienes respondieron 'NINGUN GRUPO ETNICO' permanecen en el denominador, diluyendo los porcentajes de minorías y la razón de subrepresentación.
- Categorías académicas hardcodeadas con tildes ('INVESTIGADOR SÉNIOR', 'INVESTIGADOR EMÉRITO', línea 105): si la normalización de NME_CLASIFICACION_PR usa otro casing/acentuación, la figura puede salir vacía (el filtro cats_disponibles lo mitiga pero no avisa).
- El filtro 'NING' (línea 103) elimina cualquier valor que contenga esas tres letras (p. ej. un grupo 'NINGUNA' mal codificado) sin validación del vocabulario real.
- No hay intervalos de confianza ni tests de significancia para las diferencias vs DANE; la 'subrepresentación' se reporta sin incertidumbre.

Sugerencias concretas

- Actualizar el docstring para incluir diversidad_distribucion_victimas_2021.csv (o eliminar esa exportación si no es entregable).
- Corregir comparar_dane para excluir 'NINGUN GRUPO ETNICO' del denominador (como dice su comentario) y añadir un test que verifique denominador y suma de pct.
- Derivar las categorías de las columnas reales del DataFrame (p. ej. de un catálogo) en lugar de hardcodear strings con acentos.
- Añadir IC (binomial) a los porcentajes vs DANE y una nota sobre el tamaño muestral de 2021.
- Reportar n de 'NINGUN GRUPO ETNICO' y NO DISPONIBLE para que el lector vea la cobertura real del denominador.

Ejecutabilidad Ejecutable en el clon solo tras instalar dependencias (pandas, matplotlib, seaborn; .venv del clon vacío, requirements.txt incompleto). El input existe: datos/tarea_join/investigadores_consolidado.xlsx vía cargar_consolidado(). Requiere que Transformacion genere ANO_CONVO_INT == 2021 para la convocatoria 894 (no verificado en el script). No requiere duckdb ni datos/raw. Con poetry install, corre de punta a punta.

¿Produce lo que se buscaba? Sí: existen artifacts/sprint4_diversidad/fig01_cobertura_temporal.png, fig02_comparacion_dane.png, fig03_categorias_por_minoria.png y los 7 CSVs de evidencias (diversidad_cobertura_por_convocatoria.csv, diversidad_distribucion_etnia_2021.csv, diversidad_distribucion_discapacidad_2021.csv, diversidad_distribucion_victimas_2021.csv, diversidad_comparacion_dane.csv, diversidad_interseccional_genero_etnia.csv, diversidad_categorias_

por.etnia.csv). Coincide con el código; difiere del docstring en 1 CSV (victimas.2021 no declarado), lo que confirma que se ejecutó una versión del script y hubo drift documental.

Riesgo / Limitación

Problemas detectados:

- src/analisis/diversidad.py:78-80 — comentario vs código: 'excluir NINGUN GRUPO ETNICO del denominador' pero el código filtra solo 'NO DISPONIBLE' (etnia = df_2021[TXT_GRUPO_ETNICO != 'NO DISPONIBLE']; n.etnia_resp = len(etnia) incluye NINGUN GRUPO ETNICO): los pct.minciencias y las razones de subrepresentación de minorías quedan diluidos por el denominador inflado. Bug metodológico que afecta la figura fig02 y evidencias/diversidad_comparacion_dane.csv.

Evidencia en src/analisis/diversidad.py (líneas 78-80):

```
1      # Etnia: excluir "NINGUN GRUPO ETNICO" del denominador
      para ver minorias
2      etnia = df_2021[df_2021["TXT_GRUPO_ETNICO"] != "NO
      DISPONIBLE"]
3      n_etnia_resp = len(etnia)
```

- scripts/sprint4.diversidad.py:105 — lista hardcodeada con tildes ['INVESTIGADOR SÉNIOR', 'INVESTIGADOR EMÉRITO']: depende del casing/acentuación exacta que produzca Transformacion en NME_CLASIFICACION_PR; si difiere, la figura 03 muestra categorías faltantes en silencio.

Evidencia en scripts/sprint4.diversidad.py (línea 105):

```
1      cats = ["INVESTIGADOR JUNIOR", "INVESTIGADOR ASOCIADO", "
      INVESTIGADOR SÉNIOR", "INVESTIGADOR EMÉRITO"]
```

- scripts/sprint4.diversidad.py:103 — cat_etnia[...].str.contains('NING', case=False, na=False) es un filtro de subcadena: eliminaría también valores como 'NINGUNA...' si el vocabulario de la variable cambiara.

Evidencia en scripts/sprint4.diversidad.py (línea 103):

```
1      cat_etnia = cat_etnia[~cat_etnia["TXT_GRUPO_ETNICO"].str.
      contains("NING", case=False, na=False)]
```

- scripts/sprint4.diversidad.py:158 — exporta diversidad_distribucion_victimas.2021.csv sin declararlo en el docstring (líneas 12-22), generando evidencia no documentada.

Evidencia en scripts/sprint4.diversidad.py (línea 158):

```
1      dist_vict.to_csv(EVIDENCIAS / "
      diversidad_distribucion_victimas_2021.csv", index=
      False)
```

Valoración global mejorar — el análisis es el más sensible metodológicamente (comparación con DANE/RUV) y tiene salidas verificadas, pero el denominador incorrecto en comparar_dane compromete la cifra central de subrepresentación; corregir ese bug antes de publicar y alinear docstring-código.

2.0.24 scripts/sprint5_duckdb.py**Rol:** orquestacion · **Líneas:** 193

Descripción funcional Extiende el modelo dimensional en observatorio.duckdb (creado por scripts/sprint2_duckdb.py) con fact_produccion, dim_producto, dim_grupo y la vista vw_investigador_x_produccion. Usa src/ingesta.cargar_produccion y src/analisis/produccion.normalizar_produccion. Entradas: datos/raw/produccion_grupos.csv y la DB previa. Salida: DB ampliada + reporte evidencias/duckdb_resumen_sprint5.txt con 5 consultas de validación.

Análisis de la lógica Carga y normaliza producción (IDs a Int64, extrae ANO_CONVO_INT), registra el DataFrame como tabla temporal 'prod_raw' y crea fact_produccion (una fila por id_persona_pd × id_convocatoria × id_producto_pd), dim_producto y dim_grupo con SELECT DISTINCT. Crea la vista vw_investigador_x_produccion con LEFT JOIN fact_clasificacion fc ON f.id_persona_pd = fc.id_persona_pr AND f.id_convocatoria = fc.id_convocatoria, definiendo es_reconocido = (fc.id_persona_pr IS NOT NULL). Luego ejecuta 5 consultas: productos por convocatoria; cobertura por año con pct_reconocido; productividad por categoría = COUNT(*) / COUNT(DISTINCT id_persona_pd) unida a dim_categoria (cat.categoria_normalizada / orden_normalizado); top 10 grupos por n_productos; y tipologías dominantes con ventana SUM(COUNT(*)) OVER () para el porcentaje. El reporte se escribe concatenando los to_string() de cada DataFrame.

Lo que está bien construido

- Modelado dimensional limpio: separa hechos (fact_produccion) de dimensiones (dim_producto, dim_grupo) y expone una vista lista para análisis cruzados, reutilizable por otros scripts o la app.
- Idempotente: DROP TABLE/VIEW IF EXISTS antes de cada CREATE, permitiendo re-ejecución sin corrupción acumulada.
- Validación integrada al pipeline: imprime conteos de filas por tabla y ejecuta consultas de negocio que verifican el resultado en el mismo run.
- Los artefactos declarados existen en el clon y el reporte reproduce exactamente las métricas esperadas (ver produce_esperado).

Puntos de mejora (si aplican)

- La vista asume que el id_convocatoria del dataset de producción es el mismo dominio que el de fact_clasificacion; si no coinciden (p. ej. tipos str vs int o convocatorias de grupos vs de clasificación), el LEFT JOIN degrada silenciosamente a es_reconocido = False sin ningún aviso.

- No reporta la cobertura del join (cuántos productos quedan sin clasificación asociada), clave para interpretar el `pct_reconocido`.
- La definición de 'reconocido' aquí es 'reconocido en esa misma convocatoria', distinta de la usada en `sprint5_produccion.py` (cualquier convocatoria), lo que produce cifras de cobertura no comparables entre scripts del mismo sprint.
- No hay validación de tipos entre `id_persona_pd` (Int64) y `id_persona_pr` de `fact_clasificacion` (depende de `sprint2`); un desajuste de tipo en DuckDB rompe el join en runtime, no en validación.
- La modificación de la DB no es transaccional: si falla a mitad del script, la DB queda con un subconjunto de objetos nuevos.

Sugerencias concretas

- Agregar validación post-join: COUNT productos sin match en `fact_clasificacion` y advertir si supera un umbral.
- Añadir CAST/verificación explícita de tipos de `id_persona` e `id_convocatoria` antes de construir la vista.
- Escribir un test de humo que verifique $n_{\text{investigadores}} \approx 77.237$ y que `prom_productos` por categoría esté en el rango esperado (Junior ~30, Asociado ~65, Senior ~121).
- Parametrizar `DB_PATH` y permitir `-solo-validacion` para no reconstruir tablas.
- Documentar el supuesto de que `fact_produccion.id_convocatoria` es el id de la convocatoria de clasificación del grupo.

Ejecutabilidad NO ejecutable tal cual con el clon: falta datos/processed/observatorio.duckdb (solo hay .gitkeep; requiere ejecutar scripts/sprint2.duckdb.py antes, que a su vez necesita el raw de investigadores) y datos/raw/produccion_grupos.csv (gitignored, ~1.2 GB). Requiere la dependencia duckdb instalada. El resto (pathlib, sys) es estándar. Con el dataset y la DB presentes, el script corre de punta a punta.

¿Produce lo que se buscaba? Sí: evidencias/duckdb_resumen_sprint5.txt existe y contiene las 5 consultas. La consulta `productividad_por_categoria` reproduce Junior 29.99, Asociado 65.36, Senior 121.49, Emérito 33.75 (redondeando: 30/65/121), que es exactamente la métrica que los investigadores buscaban. La cobertura por año va de 51.88 % (2013) a 63.34 % (2021). No hay discrepancias entre lo declarado y lo generado.

Riesgo / Limitación

Problemas detectados:

- `scripts/sprint5_duckdb.py:155-157` — el LEFT JOIN a `fact_clasificacion` no valida que `f.id_convocatoria = fc.id_convocatoria` tenga coincidencias; si los dominios de convocatoria difieren (producción vs clasificación), toda la columna `es_reconocido` queda en False y la cobertura reportada (51-63 %) sería espuria sin ningún error visible.

Evidencia en scripts/sprint5.duckdb.py (líneas 155-157):

```
1      LEFT JOIN fact_clasificacion fc
2          ON f.id_persona_pd = fc.id_persona_pr
3          AND f.id_convocatoria = fc.id_convocatoria
```

- scripts/sprint5_duckdb.py:59 — COUNT(*)::DOUBLE es sintaxis válida en DuckDB, pero el SUM(CASE WHEN es_reconocido THEN 1 ELSE 0 END) de la consulta cobertura_reconocimiento devuelve tipo DOUBLE (visible como '123683.0' en el reporte) frente a COUNT(*) entero; inconsistencia de tipos menor en el output.

Evidencia en scripts/sprint5.duckdb.py (línea 59):

```
1      ROUND(COUNT(*)::DOUBLE / COUNT(DISTINCT v.
          id_persona_pd), 2) AS prom_productos
```

- scripts/sprint5_duckdb.py:162 — valida COUNT(*) de fact_produccion/dim_producto/dim_grupo pero nunca valida la vista vw_investigador_x_produccion (el objeto más usado después).

Evidencia en scripts/sprint5.duckdb.py (línea 162):

```
1      n = con.execute(f"SELECT COUNT(*) FROM {tabla}").
          fetchone()[0]
```

- scripts/sprint5.duckdb.py:96-136 — DROP TABLE IF EXISTS sin transacción; una excepción entre DROPs deja la DB parcialmente actualizada (riesgo de estado inconsistente).

Evidencia en scripts/sprint5.duckdb.py (líneas 96-136):

```
1      con.execute("DROP TABLE IF EXISTS fact_produccion")
2      con.execute("""
3          CREATE TABLE fact_produccion AS
4          SELECT
5              id_persona_pd,
6              id_convocatoria,
7      [...]
```

Valoración global **mantener** — es el núcleo del modelo DuckDB del sprint 5, bien estructurado y con resultados verificados; requiere solo las mejoras de validación de join y tipos para blindar la cobertura reportada.

2.0.25 scripts/sprint5_produccion.py

Rol: orquestacion · **Líneas:** 251

Descripción funcional Análisis integral de producción de grupos: cobertura de reconocimiento (autores únicos y por convocatoria), productividad por categoría, brecha de género × gran área OCDE, concentración territorial, mix de tipologías y reconocidos sin producción. Usa src/ingesta (cargar_consolidado, cargar_produccion), src/Transformacion.transformar y

10 funciones de `src/analisis/produccion`. Entradas: investigadores consolidados + producción Socrata; salidas: 6 figuras PNG y 8 evidencias (7 CSVs + 1 JSON).

Análisis de la lógica Flujo 1→8: (1) carga y transforma; (2) `cobertura_autores_unicos` devuelve dict de solape de IDs; (3) `cobertura_reconocidos` agrupa por `ANO_CONVOC_INT` marcando `es_reconocido = ID_PERSONA_PD ∈ set(ID_PERSONA_PR del padrón EN CUALQUIER convocatoria)`; (4) `productividad_por_categoria` agrupa producción por (persona, convocatoria), hace left merge con el padrón y agrega por (año, `NME_CLASIFICACION_PR`) media/mediana/pct con ≥ 1 producto; (5) `brecha_productividad_genero` pivotea promedios F/M por área OCDE; (6) `productividad_territorial` calcula % productos vs % investigadores y ratio; (7) `mix_tipologias` y `tipologias_por_categoria` (porcentajes por año/categoría) y `reconocidos_sin_produccion` (intersección de sets por convocatoria); (8) genera 6 figuras matplotlib/seaborn y exporta CSVs y JSON.

Lo que está bien construido

- Orquestación clara y progresiva [1/8]...[8/8] con impresión de resultados intermedios (auditable en consola).
- Separación limpia entre lógica de análisis (`src/analisis/produccion.py`) y presentación (figuras), facilitando testing y reuso.
- Salidas exhaustivas y documentadas en el docstring; las 6 figuras y las 8 evidencias existen en el clon.
- Incluye indicador de calidad de captura (reconocidos sin producción) y métricas robustas (mediana además de media).

Puntos de mejora (si aplican)

- Definición de 'reconocido' inconsistente con `sprint5_duckdb.py`: `cobertura_reconocidos` marca como reconocido a quien lo fue en cualquier convocatoria (`src/analisis/produccion.py:63-65`), mientras la vista DuckDB exige la misma convocatoria; las coberturas resultantes (63.34% DuckDB vs. CSV) no son comparables.
- Carga ~3.16M filas de producción en pandas y hace merges/grupos completos en memoria: alto consumo de RAM y sin streaming ni particionado.
- `normalizar_produccion` convierte IDs a numérico `Int64` (`src/analisis/produccion.py:38-40`): cualquier id con ceros a la izquierda se pierde, riesgo silencioso de cruces falsos si el padrón conserva el formato string.
- El pct de la figura 1 se dibuja a total+5000 unidades fijas, escala que no se adapta a convocatorias pequeñas.
- No hay validación de que el join producción↔padrón haya encontrado coincidencias (una caída de cobertura se reporta como hallazgo, no como error).

Sugerencias concretas

- Unificar la definición de 'reconocido' entre scripts (recomendado: misma convocatoria) y documentarla en un único lugar.
- Validar tipos y solape de IDs antes de los merges (advertir si el match < umbral).
- Reducir memoria: leer solo columnas necesarias de producción o procesar por convocatoria.
- Añadir tests sobre `produccion_productividad_por_categoria.csv` (p. ej. productos_promedio Senior > Junior en todas las convocatorias).
- Hacer la posición de la etiqueta de % proporcional al total (ax.text con offset relativo).

Ejecutabilidad NO ejecutable tal cual con el clon: `cargar_produccion()` lanza `FileNotFoundError` porque `datos/raw/produccion_grupos.csv` está gitignored y ausente. `cargar_consolidado()` sí funciona (cae a `datos/tarea_join/investigadores_consolidado.xlsx`). Dependencias: pandas, numpy, matplotlib, seaborn (todas estándar del proyecto). Con el CSV de producción en `datos/raw`, el script corre completo.

¿Produce lo que se buscaba? SÍ: las 6 figuras de `artifacts/sprint5_produccion/` y las 8 evidencias (`produccion_cobertura_por_convocatoria.csv`, `produccion_productividad_por_categoria.csv`, `produccion_brecha_genero.csv`, `produccion_concentracion_territorial.csv`, `produccion_mix_tipologias.csv`, `produccion_tipologias_por_categoria.csv`, `produccion_reconocidos_sin_produccion.csv`, `produccion_resumen_cobertura.json`) existen. El CSV de productividad muestra Junior 9.9-17.9 y Senior 39.0-50.1 productos promedio según convocatoria, coherente con el hallazgo 'Senior produce más'. Nota: la cifra 30/65/121 mencionada en el encargo corresponde a la métrica acumulada de `sprint5_duckdb.py`, no a este CSV (que es por convocatoria).

Riesgo / Limitación

Problemas detectados:

- `src/analisis/produccion.py:63-65` (invocado desde `sprint5_produccion.py:203`) — `cobertura_reconocidos` marca `es_reconocido` sin condicionar la convocatoria: un producto de 2013 firmado por alguien reconocido solo en 2021 cuenta como 'firmado por reconocido', sobreestimando la cobertura frente a la definición por convocatoria del DuckDB (documentado en el docstring de la función como decisión, pero no advertido en el script orquestador).

Evidencia en `src/analisis/produccion.py` (líneas 63-65):

```
1  ids_reconocidos = set(inv["ID_PERSONA_PR"].dropna().
    astype("Int64"))
2  p = prod.copy()
3  p["es_reconocido"] = p["ID_PERSONA_PD"].isin(
    ids_reconocidos)
```

Evidencia en `sprint5_produccion.py` (línea 203):

```
1  cov = cobertura_reconocidos(prod, inv)
```

- scripts/sprint5_produccion.py:75-78 — la etiqueta de porcentaje se posiciona en total+5000 y el bucle itera zip(n_reconocidos, n_productos, pct_reconocidos): si n_productos < 5000 la etiqueta queda fuera del rango visible del eje (cosmético).

Evidencia en scripts/sprint5_produccion.py (líneas 75-78):

```

1     for i, (rec, total, pct) in enumerate(
2         zip(cob_df["n_reconocidos"], cob_df["n_productos"],
3           cob_df["pct_reconocidos"]))
4     ):
        ax.text(i, total + 5000, f"{pct:.1f}%", ha="center",
              fontsize=9)

```

- scripts/sprint5_produccion.py:243-244 — import json dentro de main() (estilo menor; debería ir al tope con los demás imports).

Evidencia en scripts/sprint5_produccion.py (líneas 243-244):

```

1     with open(EVIDENCIAS / "produccion_resumen_cobertura.json",
2               "w", encoding="utf-8") as f:
3         json.dump(cov_unicos, f, ensure_ascii=False, indent=2)

```

- src/analisis/produccion.py:38-40 — pd.to_numeric(errors='coerce') convierte IDs no numéricos a NA silenciosamente; si ID_PERSONA_PD tiene formatos mixtos, filas completas se pierden del análisis sin reporte.

Evidencia en src/analisis/produccion.py (líneas 38-40):

```

1     for c in ("ID_PERSONA_PD", "ID_CONVOCATORIA"):
2         if c in df.columns:
3             df[c] = pd.to_numeric(df[c], errors="coerce").
4                 astype("Int64")

```

Valoración global mantener — es el script central del sprint 5, con salidas verificadas y documentadas; las mejoras prioritarias son unificar la definición de 'reconocido' entre scripts y validar la calidad del join antes de interpretar coberturas.

2.0.26 scripts/sprint6_geografia_institucional.py

Rol: orquestacion · **Líneas:** 302

Descripción funcional Análisis geográfico institucional: cruza el departamento de residencia del investigador con el departamento de la sede de su institución principal (vía el mapping de la tabla maestra IES), respondiendo la pregunta del director sobre dónde se forman/afilian los investigadores de departamentos como Vichada (¿en Meta o Bogotá?). Usa src/ingesta.cargar_consolidado, src/Transformacion.transformar y evidencias/mapping_inst_filia_to_ies.csv. Salidas: 3 figuras y 3 CSVs.

Análisis de la lógica `asignar_institucion_principal()`: toma la primera institución de `INST_FILIA` (split por '|'), la normaliza (`upper/strip`) y hace lookup en el mapping por `inst_filia_str` → (`nombre_canonico`, `sigla`, `departamento_sede`, `naturaleza`); normaliza departamentos con `NORM_DPTO` + eliminación de acentos (`unicodedata`); genera `DPTO_INSTITUCION` y `DPTO_RESIDENCIA` canónicos. `tabla_flujos()`: `nunique` de `ID_PERSONA_PR` por par (`residencia`, `institución`), excluyendo `NO_MAPEADA/SIN_FILIA`. `resumen_por_residencia()`: por departamento, `total/local/fuera`, % local y destino externo principal. Figuras: heatmap `top 15×12` de flujos, % local por departamento (`top 25`) y destino de departamentos pequeños (`VICHADA`, `GUAINIA`, etc.).

Lo que está bien construido

- Responde exactamente la pregunta del director con un indicador accionable (% local vs. destino externo principal) y foco explícito en departamentos pequeños.
- Normalización consistente de departamentos (`NORM_DPTO` + quita acentos) que unifica el formato del padrón ('BOGOTÁ, D. C.') con el de la tabla maestra ('Bogotá D.C.').
- Exporta el flujo completo (pares `residencia`→`institución` con conteos) y no solo resúmenes, permitiendo reproducir el análisis.
- Artefactos verificados: 3 figuras y 3 CSVs existen; el resumen muestra resultados coherentes (`BOGOTA 91.8 % local`, `ANTIOQUIA 89.2 %`, destinos externos hacia `BOGOTA`).

Puntos de mejora (si aplican)

- El lookup depende de coincidencia exacta entre `INST_FILIA` (tras `upper/strip`, SIN quitar acentos) y las claves de mapping (mayúsculas sin acentos): un padrón con tildes ('UNIVERSIDAD TECNOLÓGICA') caería a `NO_MAPEADA` silenciosamente; el script solo reporta el % global de mapeados, no por string.
- Tomar la primera institución cuando `INST_FILIA` trae varias (separadas por |) es una simplificación no documentada como limitación.
- `EXTRANJERO/EXTERIOR/NO IDENTIFICADO` no se excluyen de los flujos y pueden ocupar lugares en los top (heatmap/top destinos).
- Los tops (`15×12`, `top 25`, umbral) están hardcodeados sin parámetros CLI.

Sugerencias concretas

- Quitar acentos (`unicodedata`) a `INST_FILIA` antes del lookup y reportar el % de match por grupo de error (`NO_MAPEADA` con `top-10 strings`).
- Excluir `EXTRANJERO/EXTERIOR/NO IDENTIFICADO` de los flujos o tratarlos como categoría aparte en las figuras.
- Documentar la regla de 'primera institución' y evaluar asignar por frecuencia/antigüedad en vez de orden textual.

- Parametrizar top_n y umbrales por CLI.

Ejecutabilidad Sí, ejecutable tal cual con el clon: cargar_consolidado() resuelve vía datos/tarea_join/investigadores_consolidado.xlsx, el mapping existe (evidencias/mapping_inst_filia_to_ies.csv) y solo requiere pandas, numpy, matplotlib y seaborn.

¿Produce lo que se buscaba? Sí: artifacts/sprint6-geografia/{fig-heatmap-residencia-institucion,fig_pct_local_por_dpto,fig_top_destinos-pequenos}.png y evidencias/{geografia-flujo-completo,geografia-resumen_por_dpto-residencia,geografia_top-flujos}.csv existen. El resumen confirma el hallazgo buscado (Bogotá 91.8% local, departamentos pequeños con destinos externos), y geografia-flujo-completo.csv alimenta el Sankey territorial.

Riesgo / Limitación

Problemas detectados:

- scripts/sprint6-geografia_institucional.py:81-88 — INST_FILIA solo se pasa a upper/strip, sin quitar acentos, mientras las claves del mapping (generadas desde MAPEO y el dump) son mayúsculas sin acentos; cualquier tilde en el dato fuente rompe el match y degrada a NO_MAPEADA sin aviso específico.

Evidencia en scripts/sprint6-geografia_institucional.py (líneas 81-88):

```
1 primera = (df["INST_FILIA"]
2           .fillna("")
3           .astype(str)
4           .str.split("|")
5           .str[0]
6           .str.strip()
7 [...])
```

- scripts/sprint6-geografia_institucional.py:88 — el reemplazo cubre ", 'NAN' y 'NO REPORTADO', pero si el padrón usa otro marcador de ausencia (p. ej. 'NO DISPONIBLE' o 'SIN INFORMACION') quedaría como institución ficticia y se contaría en los flujos.

Evidencia en scripts/sprint6-geografia_institucional.py (línea 88):

```
1 df["INST_PRIMERA"] = primera.replace({"": np.nan, "NAN":
    np.nan, "NO REPORTADO": np.nan})
```

- scripts/sprint6-geografia_institucional.py:118-119 — se excluyen NO_MAPEADA/SIN_FILIA pero no 'EXTRANJERO'/'EXTERIOR'/'NO IDENTIFICADO': los flujos internacionales/desconocidos pueden dominar los top y desdibujar el mensaje territorial.

Evidencia en scripts/sprint6-geografia_institucional.py (líneas 118-119):

```
1 sub = df[~df["DPTO_INSTITUCION"].isin(["NO_MAPEADA", "
    SIN_FILIA"])]
2 sub = sub[sub["DPTO_RESIDENCIA"].notna()]
```

- scripts/sprint6-geografia_institucional.py:141-145 — fuera.iloc[0] depende de que el grupo mantenga el orden descendente global por n_investigadores; correcto por construcción (flujo

ya ordenado), pero frágil si tabla.flujos se reordenara.

Evidencia en scripts/sprint6_geografia_institucional.py (líneas 141-145):

```

1      fuera = grupo[grupo["DPTO_INSTITUCION"] != dpto]
2      if len(fuera):
3          top = fuera.iloc[0]
4          top_destino = top["DPTO_INSTITUCION"]
5          top_n = int(top["n_investigadores"])

```

Valoración global mantener — es el análisis estrella del sprint 6 (pregunta directa del director, evidencia verificada y reutilizable); requiere endurecer la normalización de strings y el manejo de categorías no territoriales para que la cobertura de mapeo no degrade silenciosamente.

2.0.27 scripts/sprint6_ocde_composicion.py

Rol: orquestacion · **Líneas:** 183

Descripción funcional Responde dos preguntas del director sobre la producción nacional: qué rama del conocimiento produce más y cómo se compone por tipo de medición (nuevo conocimiento, formación, apropiación), segmentando por gran área OCDE. Cruza producción (Socrata 33dq-ab5a) con el padrón por (ID_PERSONA_PD = ID_PERSONA_PR, ID_CONVOCATORIA) mediante inner join. Usa src/ingesta, src/Transformacion.transformar y src/analisis/produccion.normalizar_produccion. Salidas: 3 figuras y 3 CSVs.

Análisis de la lógica cruzar(): merge inner de producción con (ID_PERSONA_PR→ID_PERSONA_PD, ID_CONVOCATORIA, NME.GRAN_AREA_PR) y filtra 'NO REGISTRA'/'NO REPORTADO'. Volumen = groupby área con size(); productividad = n_productos (filas producto-por-convocatoria) / n_investigadores (nunique de ID_PERSONA_PR del padrón en esa área, todas las convocatorias); composición = groupby (área, tipo medición) con pct dentro del área. Tres figuras: barras horizontales de volumen, barras apiladas de composición con PALETA.TIPO, y barras de productividad.

Lo que está bien construido

- Responde directamente la pregunta del director con una lectura clara (volumen por área, composición por tipo de medición).
- Paleta de tipos de medición explícita y consistente con la presentación.
- Filtra áreas no informadas y excluye del denominador de investigadores a áreas sin producción (evita ratios 0).

- Artefactos verificados: 3 figuras y 3 CSVs (ocde_volumen, ocde_productividad, ocde_composicion) existen en el clon.

Puntos de mejora (si aplican)

- El denominador de productividad mezcla unidades: n_productos cuenta filas producto $\times$ convocatoria (una persona puede sumar productos de 5 convocatorias), mientras n_investigadores es nunique de personas en TODO el padrón $\rightarrow$ infla la productividad por área hasta $\sim 5\times$ frente a un promedio por persona-convocatoria.
- El inner join reduce la 'producción nacional' a productos firmados por reconocidos: el título '¿Qué rama produce más en Colombia?' es engañoso si se interpreta como toda la producción del país.
- No reporta cuántos productos quedan fuera del cruce (autores no reconocidos o convocatorias sin match), pese a imprimir solo len(df) del resultado.
- La productividad se calcula sin controlar categoría ni año: un área con muchos Senior en 2021 (convocatoria más productiva) se beneficia del efecto composición.

Sugerencias concretas

- Calcular productividad como productos por (persona, convocatoria, área) y luego promediar por persona-área, o dividir por persona-convocatoria únicos.
- Reportar la pérdida del inner join (n productos sin match / por qué).
- Renombrar la métrica a 'producción de investigadores reconocidos por área' y añadir caveat en el título.
- Controlar por año y categoría (p. ej. productividad estandarizada) antes de afirmar que un área produce más.

Ejecutabilidad NO ejecutable tal cual con el clon: cargar_produccion() falla por ausencia de datos/raw/produccion_grupos.csv (gitignored). Requiere pandas, matplotlib y seaborn. Con el dataset de producción presente, el resto del pipeline (padrón vía datos/tarea_join) está disponible.

¿Produce lo que se buscaba? SÍ: artifacts/sprint6_ocde/{fig_volumen_por_area,fig_composicion_tipo_por_area,fig_productividad_por_area}.png y evidencias/{ocde_volumen_por_area,ocde_productividad_por_area,ocde_composicion_por_area}.csv existen y son coherentes con la estructura del script.

Riesgo / Limitación

Problemas detectados:

- scripts/sprint6_ocde_composicion.py:151-154 — n_productos/n_investigadores mezcla granularidades: numerador filas (persona $\times$ convocatoria $\times$ producto) y denominador personas

únicas en todo el periodo; si una persona aparece en varias convocatorias, su producción completa se divide por 1, inflando sistemáticamente la productividad de todas las áreas.

Evidencia en scripts/sprint6_ocde_composicion.py (líneas 151-154):

```
1 prod_area = volumen.merge(inv_por_area, on="gran_area")
2 prod_area["productos_por_investigador"] = (
3     prod_area["n_productos"] / prod_area["
4         n_investigadores"]
5     ).round(2)
```

- scripts/sprint6_ocde_composicion.py:62 — how='inner' descarta en silencio los productos sin autor reconocido o con ID_CONVOCATORIA no coincidente; el print de [2/5] solo muestra len(df) del cruce y no la pérdida frente a len(prod).

Evidencia en scripts/sprint6_ocde_composicion.py (línea 62):

```
1 df = prod.merge(inv_slim, on=["ID_PERSONA_PD", "
    ID_CONVOCATORIA"], how="inner")
```

- scripts/sprint6_ocde_composicion.py:64 — el filtro solo cubre 'NO REGISTRA'/'NO REPORTADO'; variantes como 'SIN INFORMACIÓN' o NaN en NME_GRAN_AREA_PR pasarían y crearían una categoría basura en volumen/composición.

Evidencia en scripts/sprint6_ocde_composicion.py (línea 64):

```
1 df = df[~df["NME_GRAN_AREA_PR"].isin(["NO REGISTRA", "NO
    REPORTADO"])]
```

Valoración global mejorar — el análisis responde la pregunta del director y los entregables existen, pero la métrica de productividad por área es metodológicamente inconsistente (unidades mezcladas) y debe corregirse antes de usarse como hallazgo.

2.0.28 scripts/sprint6_sankey_categoria.py

Rol: orquestacion · **Líneas:** 184

Descripción funcional Genera diagramas de Sankey de transición de categoría (Junior/Asociado/Senior/Emérito + 'Desaparece') para cada par de convocatorias consecutivas 2013→2021 (5 pares). Usa src/ingesta.cargar_consolidado, src/Transformacion.transformar y src/analisis/longitudinal (construir_panel, comparar_periodo, matriz_transicion, ORDEN_CATEGORIAS). Salidas: 5 HTML interactivos + 5 PNG estáticos y evidencias/sankey_transiciones.csv con todos los flujos.

Análisis de la lógica Construye el panel longitudinal por convocatoria, obtiene los años ordenados (2013, 2014, 2015, 2017, 2019, 2021) y sus pares consecutivos. Por par: comparar_

`periodo(panel, a, b) → matriz_transicion(comp, incluir_desaparece=True)` devuelve (conteos, probs); reordena filas por `ORDEN_CATEGORIAS`; `nodos` = categoría+año (izquierda `t`, derecha `t+1`) con colores de `COLOR_CAT`; `flujos` = celdas `>0` con color `rgba` del origen; figura `go.Sankey` con `arrangement='snap'` y `hovertemplate`. Exporta HTML (plotly CDN) y PNG vía `write_image`, y consolida todos los flujos en `sankey_transiciones.csv` con clasificación Sube/Baja/Mantiene/Desaparece (`_etiqueta_transicion`) y resumen por periodo.

Lo que está bien construido

- Respetar un orden canónico de categorías (`ORDEN_CATEGORIAS`) para filas y columnas, evitando ordenamientos arbitrarios en el diagrama.
- Los flujos se exportan en formato largo (periodo, origen, destino, `n`), lo que permite reproducir el análisis sin el Sankey.
- Estado 'Desaparece' explícito: mide retención/abandono real entre convocatorias.
- Artefactos verificados: los 5 pares de HTML/PNG y `sankey_transiciones.csv` (76 líneas) existen en el clon.

Puntos de mejora (si aplican)

- `fig.write_image` exige `kaleido`, dependencia no declarada en el docstring ni en requirements (si falta, el script muere en la primera iteración).
- Los HTML usan `include_plotlyjs='cdn'`: sin conexión a internet no renderizan.
- Cualquier categoría del panel que no esté en `ORDEN_CATEGORIAS` se excluye silenciosamente de `fila_orden` (línea 54) y desaparece del Sankey sin warning, con la consiguiente pérdida de investigadores del total.
- El título suma `total_t` incluyendo los que 'Desaparecen' (correcto como total en `t`, pero puede confundir si se lee como flujo completo).
- Parámetros (`top`, `umbral`, `altura`) hardcodedados sin interfaz CLI.

Sugerencias concretas

- Declarar `kaleido` en requirements/README o manejar su ausencia con un `try/except` que solo escriba HTML.
- Validar que todas las categorías del panel estén en `ORDEN_CATEGORIAS` y advertir/fallar si no.
- Escribir los PNG con un backend alternativo (p. ej. captura de HTML con playwright) o documentar el requisito `kaleido`.
- Exportar también las probabilidades de transición (matriz) como evidencia complementaria.

Ejecutabilidad Sí en cuanto a datos desde el clon: `cargar_consolidado()` resuelve vía `datos/tarea_join/investigadores_consolidado.xlsx`. Requiere `plotly` Y `kaleido` instalados; sin `kaleido`, `write_image` (línea 146) lanza error y aborta. Con ambas librerías el script corre completo.

¿Produce lo que se buscaba? Sí: `artifacts/sprint6_sankey/sankey_2013-2014.{html,png}` ... `sankey_2019-2021.{html,png}` y `evidencias/sankey_transiciones.csv` existen. El CSV muestra flujos coherentes (2013-2014: Junior→Junior 1967, Junior→Desaparece 2823; Senior→Senior 591, etc.) y 5 periodos (2013-2014, 2014-2015, 2015-2017, 2017-2019, 2019-2021).

Riesgo / Limitación

Problemas detectados:

- `scripts/sprint6_sankey_categoria.py:52-55` — `fila_orden` filtra las categorías presentes en `ORDEN_CATEGORIAS`, pero si `construir_panel` produce una categoría no mapeada (p. ej. variante con acento 'INVESTIGADOR SÉNIOR' vs 'Senior'), esa fila se elimina del Sankey y del conteo sin ningún mensaje; el total del título quedaría subestimado.

Evidencia en `scripts/sprint6_sankey_categoria.py` (líneas 52-55):

```
1     conteos, _ = matriz_transicion(comp, incluir_desaparece=
      True)
2     # Reordenar filas siguiendo el orden canónico
3     fila_orden = [c for c in ORDEN_CATEGORIAS if c in conteos
      .index]
4     conteos = conteos.loc[fila_orden]
```

- `scripts/sprint6_sankey_categoria.py:105` — `total_t = conteos.sum().sum()` incluye la columna 'Desaparece'; el texto del título 'investigadores reconocidos en t' es correcto, pero si `fila_orden` ya excluyó categorías, `total_t` no coincide con el padrón de `t`.

Evidencia en `scripts/sprint6_sankey_categoria.py` (línea 105):

```
1     total_t = conteos.sum().sum()
```

- `scripts/sprint6_sankey_categoria.py:146-147` — `write_image` sin comprobación de `kaleido`: dependencia implícita que rompe la ejecución en entornos mínimos.

Evidencia en `scripts/sprint6_sankey_categoria.py` (líneas 146-147):

```
1     fig.write_image(ARTIFACTS / f"{nombre}.png",
2                     width=1100, height=520, scale=2)
```

Valoración global `mantener` — produce el entregable de transiciones solicitado con evidencia verificada; la prioridad es declarar `kaleido` y validar el mapeo de categorías para no perder investigadores silenciosamente.

2.0.29 scripts/sprint6_sankey_territorial.py

Rol: orquestacion · **Líneas:** 135

Descripción funcional Materializa el Sankey territorial residencia del investigador → departamento de su institución, usando como entrada directa evidencias/geografia_flujo_completo.csv (generado por sprint6_geografia_institucional.py), sin recargar el padrón. Respuesta visual a la pregunta del director sobre los flujos de departamentos pequeños hacia Meta/Bogotá. Salidas: sankey_territorial.html (interactivo) y sankey_territorial.png (estático).

Análisis de la lógica construir_sankey(): selecciona top 12 departamentos de residencia por volumen total, top 10 de institución y flujos con n_investigadores ≥ 15 ; construye nodos 'X (res.)' e 'Y (inst.)' con colores de PALETA por residencia (nodos de institución en gris); flujos con alpha 0.40 si el flujo es local (residencia == institución) y 0.65 si es transversal; figura go.Sankey con arrangement='snap' y hovertemplate con formato de miles. Exporta HTML (plotly CDN) y PNG vía write_image a 2x.

Lo que está bien construido

- Diseño de pipeline limpio: consume el CSV ya generado por el script de geografía (una sola fuente de verdad, sin duplicar la lógica de cruce).
- Parámetros de saturación explícitos (top_res, top_inst, umbral_min) que evitan un diagrama ilegible con miles de flujos.
- Distinción visual local vs. transversal (alpha) y paleta estable.
- Artefactos verificados: sankey_territorial.html y sankey_territorial.png existen en el clon.

Puntos de mejora (si aplican)

- write_image requiere kaleido (dependencia no declarada): el PNG falla en entornos sin esa librería aunque el HTML se escriba bien.
- HTML con include_plotlyjs='cdn': requiere internet para renderizar.
- Los tops y el umbral están fijos en main() sin interfaz CLI para explorar variantes.
- No exporta el subconjunto de flujos usados en el diagrama (CSV), limitando la reproducibilidad del gráfico exacto.
- Depende de que geografía_flujo_completo.csv use los strings normalizados; si ese script cambia de normalización, este se rompe sin mensaje claro.

Sugerencias concretas

- Declarar kaleido o envolver write_image en try/except para degradar a solo HTML con aviso.

- Exportar el CSV de los flujos filtrados usados en la figura.
- Parametrizar `top_res/top_inst/umbral_min` por `argparse` para iterar rápido.
- Añadir validación de columnas esperadas (`DPTO_RESIDENCIA`, `DPTO_INSTITUCION`, `n_investigadores`) al leer el CSV.

Ejecutabilidad Sí, ejecutable tal cual con el clon: la entrada `evidencias/geografia_flujo_completo.csv` existe; requiere `plotly` y `kaleido` instalados (sin `kaleido` falla solo la escritura del PNG). No depende de `datos/raw` ni del padrón.

¿Produce lo que se buscaba? Sí: `artifacts/sprint6_sankey/sankey_territorial.html` y `sankey_territorial.png` existen, completando el set de 6 sankeys del sprint 6 (5 de categoría + 1 territorial). La lógica produce exactamente el flujo solicitado (residencia → institución con umbral de ruido).

Riesgo / Limitación

Problemas detectados:

- `scripts/sprint6_sankey_territorial.py:125-129` — `write_image` sin comprobación de `kaleido`: dependencia implícita; en un entorno sin `kaleido` el script aborta tras haber generado el HTML (comportamiento no degradable).

Evidencia en scripts/sprint6_sankey_territorial.py (líneas 125-129):

```
1 fig.write_html(ARTIFACTS / "sankey_territorial.html",
2                 include_plotlyjs="cdn",
3                 full_html=True)
4 fig.write_image(ARTIFACTS / "sankey_territorial.png",
5                 width=1200, height=620, scale=2)
```

- `scripts/sprint6_sankey_territorial.py:69-70` — `top_residencias.index(row['DPTO_RESIDENCIA'])` asume que toda fila de `sub` está en la lista `top`; correcto por el filtro `isin` previo (línea 56-58), pero si `top_residencias` tuviera duplicados (no los tiene, viene de `nlargest`), el `index()` devolvería la primera aparición.

Evidencia en scripts/sprint6_sankey_territorial.py (líneas 69-70):

```
1 s_idx = top_residencias.index(row["DPTO_RESIDENCIA"])
2 t_idx = len(top_residencias) + top_instituciones.
        index(row["DPTO_INSTITUCION"])
```

- `scripts/sprint6_sankey_territorial.py:56-58` — el filtro por `umbral_min ≥ 15` es absoluto, no proporcional al total del departamento: un flujo pequeño de VICHADA (dpto de pocos investigadores) puede quedar excluido justo donde el director quiere verlo; convendría umbral relativo por origen.

Evidencia en scripts/sprint6_sankey_territorial.py (líneas 56-58):

```
1 sub = flujo[flujo["DPTO_RESIDENCIA"].isin(top_residencias)
2         &
        flujo["DPTO_INSTITUCION"].isin(
            top_instituciones) &
```

```
3 (flujo["n_investigadores"] >= umbral_min)].
   copy()
```

Valoración global mantener — script simple, enfocado y con artefactos verificados que completan el entregable del director; mejoras menores (kaleido declarado, CSV de flujos filtrados, umbral relativo para departamentos pequeños) aumentan su robustez y utilidad.

2.0.30 scripts/sprint6_tabla_maestra_ies.py

Rol: orquestacion · **Líneas:** 635

Descripción funcional Construye el borrador de la tabla maestra canónica de IES a partir de evidencias/instituciones_unicas_raw.csv (dump de strings únicos de INST_FILIA con frecuencias), aplicando un diccionario MAPEO hardcodeado de ~250 claves string → (nombre_canonico, sigla, departamento_sede, naturaleza). Solo usa pandas, sin tocar el padrón ni producción. Salidas: tabla_maestra_ies.csv, mapping_inst_filia_to_ies.csv e ies_pendientes_revision.csv.

Análisis de la lógica aplicar_mapeo(row): si inst_filia_str ∈ MAPEO devuelve la tupla canónica con estado 'asignado'; si no, NA con estado 'pendiente_revision'. main(): lee el dump, aplica el mapeo fila a fila (apply axis=1), calcula cobertura sobre apariciones (asignadas/total), agrupa las asignadas por (nombre_canonico, sigla, naturaleza) con agregados (departamentos únicos, apariciones, variantes) ordenadas por apariciones desc, y exporta los 3 archivos, imprimiendo cobertura y top 20.

Lo que está bien construido

- Borrador transparente y auditable: el mapeo completo es legible, con comentarios por pasadas y objetivo de cobertura (>90 %).
- Reporta cobertura real sobre apariciones del padrón y separa explícitamente las entradas pendientes de revisión.
- Cero dependencias externas más allá de pandas y ejecutable con el clon (el input instituciones_unicas_raw.csv existe, 2763 filas).
- Decisiones de negocio explícitas: una IES por sede/razón social con departamento_sede por ubicación física.

Puntos de mejora (si aplican)

- Mapeo 100 % manual y hardcodeado (~250 de 2763 strings): frágil ante variantes nuevas (acentos, guiones, 'SAS', 'LTDA') y costoso de mantener; es una solución de 'borrador', no un componente de datos.
- La columna 'sedes_registradas' en realidad cuenta departamentos distintos (nunique de departamento_sede), no sedes físicas: nombre engañoso.
- Coincidencia exacta por mayúsculas-sin-acentos: si el dump cambia de normalización (p. ej. conservara tildes), todo el mapeo se rompe en silencio y la cobertura cae sin error.
- Variantes duplicadas sin mapear (p. ej. 'UNIVERSIDAD NACIONAL DE COLOMBIA' sin sede) asumen Bogotá por defecto sin validación.
- apply(axis=1) con función Python es lento, aunque tolerable para ~2.8k filas.

Sugerencias concretas

- Externalizar el MAPEO a un CSV/JSON versionado (fuente de datos, no código) con validación de unicidad de claves.
- Añadir normalización difusa o reglas (contiene 'UNIVERSIDAD NACIONAL DE COLOMBIA' → UNAL) para cubrir variantes no listadas, reportando la confianza.
- Renombrar 'sedes_registradas' a 'departamentos_sede' o contar sedes reales.
- Añadir un test que falle si la cobertura baja de un umbral (p. ej. 85 %) ante cambios del dump.

Ejecutabilidad Sí, ejecutable tal cual con el clon: solo necesita pandas y evidencias/instituciones_unicas.raw.csv (existente, columnas inst_filia_str, n_apariciones, pct, pct_acum — coinciden con lo que lee el script). No depende de datos/raw ni de observatorio.duckdb.

¿Produce lo que se buscaba? Sí: evidencias/tabla_maestra_ies.csv, evidencias/mapping_inst_filia_to_ies.csv e evidencias/ies_pendientes_revision.csv existen. El mapping tiene las columnas esperadas (inst_filia_str, nombre_canonico, sigla, departamento_sede, naturaleza, estado) y es el insumo que consumen sprint6_geografia_institucional.py y la app.

Riesgo / Limitación**Problemas detectados:**

- scripts/sprint6_tabla_maestra_ies.py:607 — 'sedes_registradas' = nunique de departamento_sede: dos sedes en el mismo departamento (p. ej. dos campus de la UNAL en Bogotá) cuentan como 1, mientras dos sedes en departamentos distintos cuentan como 2; la métrica no es número de sedes.

Evidencia en scripts/sprint6_tabla_maestra_ies.py (línea 607):

```
1      sedes_registradas=("departamento_sede", "nunique"
                        ),
```

- scripts/sprint6_tabla_maestra_ies.py:38-562 — claves hardcodeadas sin validación de duplicados ni de normalización: si instituciones_unicas_raw.csv se regenera con otro case/acentos, la cobertura cae sin ninguna señal de error.

Evidencia en scripts/sprint6_tabla_maestra_ies.py (líneas 38-562):

```

1 MAPEO = {
2     # --- Top 20 por frecuencia
3     "UNIVERSIDAD DE ANTIOQUIA":
4         ("Universidad de Antioquia", "UdeA", "Antioquia", "
5         publica"),
6     "UNIVERSIDAD NACIONAL DE COLOMBIA SEDE BOGOTA (
7     UNIVERSIDAD NACIONAL DE COLOMBIA)":
8         ("Universidad Nacional de Colombia", "UNAL", "Bogotá
9         D.C.", "publica"),
10    [...]

```

- scripts/sprint6_tabla_maestra_ies.py:592 — pd.concat([raw, asignaciones], axis=1) asume que aplicar_mapeo devuelve una Series por fila alineada por índice; correcto aquí, pero frágil si se reordenara raw antes.

Evidencia en scripts/sprint6_tabla_maestra_ies.py (línea 592):

```

1 asignaciones = raw.apply(aplicar_mapeo, axis=1)

```

- scripts/sprint6_tabla_maestra_ies.py:620-623 — ies_pendientes_revision.csv incluye todas las entradas sin mapeo, pero el script no imprime las 10 pendientes más frecuentes para facilitar la revisión manual (solo cobertura agregada).

Evidencia en scripts/sprint6_tabla_maestra_ies.py (líneas 620-623):

```

1 pendientes = mapping[mapping["estado"] == "
2     pendiente_revision"].copy()
3 pendientes.to_csv(EVIDENCIAS / "ies_pendientes_revision.
4     csv",
5                   index=False, encoding="utf-8")
6 cobertura_pendientes = pendientes["n_apariciones"].sum()
7     / total_apariciones * 100

```

Valoración global refactorizar — como borrador cumple y sus productos existen, pero un catálogo canónico de IES no debería vivir en código: externalizar el mapeo a datos versionados, renombrar métricas y añadir cobertura mínima garantizada antes de que otros scripts (geografía, app) dependan de él en producción.

2.0.31 scripts/sprint6_validacion_calidad.py

Rol: orquestacion · **Líneas:** 121

Descripción funcional Valida y documenta el tratamiento de valores atípicos de la variable EDAD_ANOS_PR del padrón: detecta los casos >100 (máximo observado 956, error de digitación), compara tres estrategias (original, eliminación, imputación por mediana de grupo) y genera evidencia auditable. Usa `src/ingesta.cargar_consolidado`, `src/Transformacion.transformar` y `src/analisis/calidad` (`detectar_atipicos_edad`, `resumen_tratamiento`, `filtrar`, `imputar_mediana`, `UMBRAL_EDAD_MAX`). Salidas: 2 CSVs, 1 JSON y 1 figura.

Análisis de la lógica Flujo 1→4: (1) `transformar(cargar_consolidado())`; (2) `detectar_atipicos_edad` devuelve los registros con EDAD_ANOS_PR >100 ordenados desc; (3) `resumen_tratamiento` calcula `n_total/n_atipicos/n_validos` y justificación, y se construye un DataFrame comparativo `edad_stats` con `media/mediana/desv/max` para original, `filtrar()` (eliminación) e `imputar_mediana()`; (4) histogramas antes/después con umbral trazado como `axvline`, y exportación de atipicos a CSV, comparación a CSV y resumen a JSON.

Lo que está bien construido

- Metodología explícita y auditable: umbral documentado (100), justificación cuantitativa ($10/77.237 = 0.013\%$) y decisión adoptada (eliminación) con alternativa de imputación comparada.
- La evidencia confirma los números esperados: `n=77.237` registros, máx original 956.25 → 100 tras filtro (`evidencias/calidad_comparacion.csv`).
- Script pequeño, legible y sin dependencias exóticas (`pandas`, `matplotlib`, `seaborn`).
- Reutiliza funciones de negocio de `src/analisis/calidad.py` con docstrings de decisión metodológica.

Puntos de mejora (si aplican)

- `n_registros` usa `len(df)` (cuenta filas, incluidas las de EDAD NaN) mientras `media/mediana/std` ignoran NaN: unidades de conteo mixtas en la misma tabla comparativa.
- La mediana se deja sin redondear mientras `media` y `desv_estandar` se redondean a 2 decimales (formato inconsistente).
- El script solo valida edad; otros problemas documentados en `calidad.py` (departamento NO DISPONIBLE 5.2 %, género NO REPORTADO 2.7 %, convocatoria 833 con año 2019) se citan pero no se verifican ni exportan.
- La figura compara 'original' vs 'filtrado' pero no muestra el efecto de la imputación, pese a que se calcula en el DataFrame comparativo.

Sugerencias concretas

- Calcular `n_registros` como `count no-NA` de EDAD_ANOS_PR para consistencia con las demás métricas.

- Redondear mediana igual que media y documentar el criterio.
- Ampliar la validación a duplicados, nulos por columna y distribución de tipos, exportando un perfil de calidad completo.
- Fijar seed/versión del dataset en el JSON de resumen para trazabilidad.

Ejecutabilidad Sí, ejecutable tal cual con el clon: `cargar_consolidado()` resuelve desde `datos/tarea_join/investigadores_consolidado.xlsx` (no requiere raw Socrata); depende solo de `pandas`, `matplotlib` y `seaborn`. El único requisito no verificado es que `transformar()` produzca `EDAD_ANOS_PR`, lo cual confirman los artefactos existentes.

¿Produce lo que se buscaba? Sí: `evidencias/calidad_atipicos_edad.csv`, `evidencias/calidad_comparacion.csv`, `evidencias/calidad_resumen.json` y `artifacts/sprint6_calidad/fig_edad_antes_despues.png` existen. El CSV de comparación confirma 77.237 registros, media 44.84 → 44.73, desv 13.23 → 10.04 y máx 956.25 → 100.0, coherente con la decisión de eliminación.

Riesgo / Limitación

Problemas detectados:

- `scripts/sprint6_validacion_calidad.py:62-84` — `edad_stats` mezcla unidades: `len(df)` cuenta filas (incluye NaN en `EDAD`) mientras `mean/median/std` de `pandas` ignoran NaN; si hubiera filas con edad faltante, `n_registros` y las demás columnas no serían comparables.

Evidencia en scripts/sprint6_validacion_calidad.py (líneas 62-84):

```

1  edad_stats = pd.DataFrame({
2      "metodo": ["original", "eliminacion (>100)", "
      imputacion mediana por grupo"],
3      "n_registros": [len(df), len(df_filtrado), len(
      df_imputado)],
4      "media": [
5          round(df["EDAD_ANOS_PR"].mean(), 2),
6          round(df_filtrado["EDAD_ANOS_PR"].mean(), 2),
7      [...]
```

- `scripts/sprint6_validacion_calidad.py:70-73` — `df["EDAD_ANOS_PR"].median()` sin `round` frente a `round(..., 2)` en `media` (formato inconsistente; en el CSV aparece 43.29 por coincidencia del valor).

Evidencia en scripts/sprint6_validacion_calidad.py (líneas 70-73):

```

1      "mediana": [
2          df["EDAD_ANOS_PR"].median(),
3          df_filtrado["EDAD_ANOS_PR"].median(),
4          df_imputado["EDAD_ANOS_PR"].median(),
```

- `scripts/sprint6_validacion_calidad.py:50-51` — `detectar_atipicos_edad` solo imprime, no informa el conteo de NaN de `EDAD_ANOS_PR`, que podría alterar la interpretación del umbral.

Evidencia en scripts/sprint6_validacion_calidad.py (líneas 50-51):

```

1 atipicos = detectar_atipicos_edad(df)
2 print(atipicos.to_string(index=False))

```

Valoración global mantener — cumple su propósito (decisión metodológica auditable) con evidencia verificada; requiere solo ajustes menores de consistencia de conteos y redondeo.

2.0.32 app/streamlit_app.py

Rol: visualizacion · **Líneas:** 17

Descripción funcional Placeholder de 17 líneas que solo configura la página (título 'Observatorio Ustadistica', icono de gráfico, layout wide) y muestra un título con el aviso 'Dashboard en construcción. Consulta el README para el plan de desarrollo.' No carga datos, no importa src/ y no contiene ningún análisis. Es la entrada que ejecuta el Dockerfile (CMD streamlit run app/streamlit_app.py).

Análisis de la lógica Estructura mínima: import streamlit; st.set_page_config (page_title 'Observatorio Ustadistica', page_icon '\{\}U0001f4ca, layout wide); st.title; st.info. Sin st.cache_data, sin filtros, sin tabs, sin conexión con src/. El docstring documenta el uso con 'poetry run streamlit run app/streamlit_app.py'.

Lo que está bien construido

- Sintaxis Streamlit válida: no crashea al arrancar
- Sirve como esqueleto para desarrollo futuro (README del proyecto menciona 'plan de desarrollo')

Puntos de mejora (si aplican)

- Es un placeholder que contradice al dashboard real (streamlit_app.py, 983 líneas, 7 tabs)
- El README documenta el deploy del dashboard real (main file streamlit_app.py en Streamlit Cloud, línea 192) pero el Dockerfile despliega este placeholder: dos 'dashboards' distintos según la vía de deploy
- La imagen Docker no copia el dashboard real (COPY solo src/, app/, datos/) ni hallazgos/, así que en el contenedor el usuario final ve solo el cartel de 'en construcción'
- Duplica la identidad del proyecto (título distinto al del dashboard real: 'Observatorio Ustadistica' vs 'Observatorio MinCiencias - Informe crítico')

Sugerencias concretas

- Eliminar el placeholder y apuntar el Dockerfile al dashboard real (copiar streamlit_app.py de la raíz, hallazgos/ y datos/), o
- Mover el dashboard real a app/streamlit_app.py (haciendo app/ la entrada canonica) y actualizar README y Streamlit Cloud en consecuencia, o
- Si se mantiene como esqueleto, excluirlo del Dockerfile y documentar que no es el dashboard de produccion

Ejecutabilidad Sí, se ejecuta tal cual (solo requiere streamlit instalado), pero solo muestra el aviso de 'Dashboard en construccion'. No lee ningun dato ni modulo del proyecto.

¿Produce lo que se buscaba? NO produce el dashboard de 7 capitulos documentado: renderiza un cartel estatico. En el despliegue Docker, el contenedor (python:3.12-slim + poetry + datos/) muestra este placeholder en vez del dashboard real, contradiciendo el README y el proposito del observatorio.

Riesgo / Limitación**Problemas detectados:**

- Bug de deploy (no de sintaxis): el Dockerfile ejecuta 'streamlit run app/streamlit_app.py' (línea 22) ->el contenedor despliega el placeholder y no el dashboard real; ademas streamlit_app.py de la raíz ni siquiera se copia a la imagen (COPY src/, app/, datos/ solamente)
- Contradiccion de documentacion: README línea 175 ('poetry run streamlit run streamlit_app.py') y línea 192 (Streamlit Cloud main file streamlit_app.py) apuntan al dashboard real, mientras que el docstring de este archivo y el Dockerfile apuntan al placeholder
- La imagen Docker tampoco copia hallazgos/sprint3_grafo_filtrado.html: aunque se cambiara el CMD al dashboard real, la tab Redes fallaria en el contenedor

Valoración global eliminar: el placeholder no aporta valor de produccion y rompe la consistencia del deploy (Docker muestra algo distinto a lo documentado y a Streamlit Cloud). La accion correcta es hacer que el Dockerfile ejecute el dashboard real (streamlit_app.py de la raíz o movido a app/) y copiar hallazgos/; o, si se prefiere app/ como canonica, mover el dashboard real ahi y borrar este archivo.

2.0.33 streamlit_app.py

Rol: visualizacion · **Líneas:** 983

Descripción funcional Dashboard Streamlit principal del Observatorio MinCiencias con 7 pestañas tipo capítulo (Calidad, Trayectoria, Territorio, Diversidad, Redes, Productividad, Datos crudos). Lee el consolidado vía src/ingesta.cargar_consolidado (XLSX de respaldo

en datos/tarea.join), lo transforma con src/Transformacion.transformar y lo cruza con ~30 funciones de src/analisis/*. Cada sección sigue el patrón pregunta ->evidencia ->hallazgo ->caveat, con filtros globales en la barra lateral.

Análisis de la lógica Estructura: constantes (COORDENADAS_DEPTO, POBLACION_DANE_2018, CAT_CORTA) ->cargar_datos() con @st.cache_data (aplica transformar y crea edad_valida) ->sidebar_filtros() (año, género, gran área, depto top-15, categoría, checkbox excluir calidad) ->cabecera() con 6 métricas ->7 tabs que llaman a una funcion seccion_* cada una. Caches: cargar_datos, panel_completo(len(df), df) y cargar_produccion_normalizada. Conexion con src/: sys.path.insert(ROOT/src) e imports de ingesta, Transformacion y analisis.{longitudinal, territorial, genero, diversidad, redes, produccion}. La seccion 5 embebe hallazgos/sprint3_grafo_filtrado.html (existe) y la 6 depende de datos/raw/produccion_grupos.csv (NO existe en el repo).

Lo que está bien construido

- Arquitectura limpia por secciones con una funcion por capitulo; facil de auditar
- Narrativa critica consistente (pregunta/hallazgo/caveat) con caveats metodologicos muy buenos (p. ej. subrepresentacion vs poblacion con educacion superior, Eméritos vitalicios)
- Manejo defensivo: try/except que degrada con st.info en vez de crashear (HHI, genero, DANE, longitudinal)
- Reutiliza las funciones reales de src/analisis; no duplica logica de analisis en la UI
- Filtros globales coherentes entre secciones y metricas de cabecera bien elegidas
- Imports y rutas verificados: todos los modulos src/analisis importados existen con las firmas usadas

Puntos de mejora (si aplican)

- La seccion 6 (Productividad, capitulo 6 del informe) queda vacia en esta maquina: falta datos/raw/produccion_grupos.csv (~1,2 GB); solo muestra un warning
- El grafico HHI de la seccion 3 esta roto por un bug de shape y degrada silenciosamente a 'HHI no disponible'
- Numeros hardcoded en texto y logica: '30.086 investigadores unicos', fallback 16796 para la anomalia de convocatoria, '6 convocatorias (2013-2021)' en la caption
- Caché de panel_completo con clave debil (len(df)): filtros con el mismo tamaño colisionan y pueden devolver paneles obsoletos
- Docstring dice '6 secciones tipo capitulos' pero hay 7 tabs (desincronizacion interna)
- Seccion 6 no cubierta por try/except: si el CSV de produccion existiera vacio, cov.iloc[0]/iloc[-1] lanzarian IndexError sin captura
- Metricas de cabecera usan df_full mientras las secciones usan df filtrado: el texto 'global' mezcla poblaciones

Sugerencias concretas

- Corregir el bug del HHI: eliminar el `.reset_index()` o renombrar usando las columnas reales (`ANO_CONVO_INT`, `hhi`)
- Documentar el prerequisite del dataset de produccion en la UI (ya hay warning) o agregar fallback a evidencias/produccion_*.csv
- Calcular en runtime los numeros hardcodeados (unicos, convocatorias) en vez de fijarlos
- Cachear `panel_completo` con un hash real del filtro (tupla de selecciones) en vez de `len(df)`
- Envolver la seccion 6 completa en try/except para evitar `IndexError` con produccion vacia
- Sincronizar docstring (6 ->7 secciones) y caption

Ejecutabilidad Parcial. ‘streamlit run streamlit_app.py’ desde la raiz funciona con streamlit, pandas, numpy, plotly y openpyxl (requirements.txt los declara). cargar consolidado cae al XLSX datos/tarea_join/investigadores_consolidado.xlsx (existe; datos/raw/investigadores_consolidado.csv no existe). Con ese insumo, las secciones 1, 2, 3 (sin HHI), 4, 5 y 7 deberian renderizar. La seccion 6 (productividad) no se ejecuta: requiere descargar datos/raw/produccion_grupos.csv con ‘python -m src.ingesta.produccion’; la app lo maneja con `st.warning` y `return`. En entorno poetry (Docker) openpyxl figura en poetry.lock (grupo main) aunque pyproject.toml no lo declara: riesgo de lock desincronizado.

¿Produce lo que se buscaba? Sí, el dashboard implementa los 7 capitulos del informe y coincide con el README (‘streamlit_app.py # Dashboard - 7 tabs’). Sin embargo, el capitulo 6 (productividad) no produce nada sin el dataset de produccion, y el HHI (parte del capitulo 3) nunca dibuja por el bug de shape. El placeholder app/streamlit_app.py no contradice a este archivo en local ni en Streamlit Cloud (el README apunta a este), pero sí en el despliegue Docker.

Riesgo / Limitación

Problemas detectados:

- Seccion 3: `hhi_por_convocatoria(df, col_geo=col).reset_index()` devuelve 5 columnas (`index` + `ANO_CONVO_INT` + `hhi` + `n_total` + `n_territorios`) y luego `hhi_df.columns = ['Año', 'HHI']` lanza `ValueError 'Length mismatch'`; el except lo captura y el grafico HHI nunca se muestra (lineas 434-435)
- Seccion 6: depende de `datos/raw/produccion_grupos.csv` que no esta en el repo; toda la seccion (4 metricas + 6 graficos) se degrada a un warning
- Seccion 1: fallback hardcodeado ‘else 16796’ y umbral magico `ID_CONVOCATORIA == 20` sin validacion ->numero potencialmente incorrecto en silencio
- `panel_completo(len(df), df)` usa `len(df)` como hash de cache ->colisiones entre filtros del mismo tamaño (linea 347)
- Si el CSV de produccion existiera pero vacio, `cov.iloc[0]/cov.iloc[-1]` (lineas 761-762) lanzarian `IndexError` no capturado y crashearia la app

- Caption y docstring desincronizados: '6 secciones' en el header (línea 7) vs 7 tabs reales

Valoración global mejorar: es la pieza central del proyecto, bien estructurada y con 5 de 7 secciones funcionales con el XLSX actual. Corregir el bug del HHI, robustecer la cache, documentar/proveer el dataset de producción y eliminar los hardcodes antes de considerar un refactor.

2.0.34 docs/manual.ipynb

Rol: documentacion · **Líneas:** 805

Descripción funcional Manual reproducible de 49 celdas (23 markdown + 26 código) generado por scripts/generar_manual.py (no ejecutado: todas las celdas tienen execution_count null). 14 secciones que reproducen los hallazgos del observatorio: calidad (atípicos de edad), trayectoria y matrices de transición, Sankeys de categoría, concentración territorial (HHI), tabla maestra de IES, geografía institucional (residencia vs institución), brecha de género, diversidad vs DANE/RUV, productividad por categoría, brecha de género en productividad, composición OCDE y recomendaciones de política pública.

Análisis de la lógica Celda 1: ROOT = pathlib.Path.cwd() con fallback a ROOT.parent si (ROOT/src) no existe (funciona desde la raíz del repo o desde docs/); inserta src/ en sys.path. Celda 2: inv = transformar(cargar_consolidado()) y prod = normalizar_produccion(cargar_produccion()). Secciones 2-3: funciones de src/analisis/calidad.py y longitudinal.py (construir_panel, comparar_periodo, matriz_transicion, matrices_todos_periodos). Secciones 4, 7, 8 y 13: subprocess.run(['python3', 'scripts/sprint6_*.py'], cwd=ROOT). Secciones 6-8 y 13: pd.read_csv de evidencias/*.csv. IFrame apunta a artifacts/sprint6_sankey/*.html. Todo el código del manual reutiliza los módulos reales de src/ (no duplica lógica).

Lo que está bien construido

- Intención didáctica y reproducible clara: cada hallazgo con su código mínimo y su caveat
- Reutiliza el código real (src/ingesta, src/Transformacion, src/analisis/*): el manual no inventa lógica paralela
- Los insumos que referencia existen en el repo: evidencias/tabla_maestra_ies.csv, mapping_inst_filia_to_ies.csv, geografía_*.csv, ocde_*.csv, scripts/sprint6_*.py y artifacts/sprint6_sankey/*.html
- Caveats de alta calidad (Eméritos vitalicios, comparación contra población con educación superior, captura de doble afiliación solo en 2021)
- Resolución de ROOT con fallback (mejor que la del EDA)

Puntos de mejora (si aplican)

- Dos celdas con bugs de API que crashean: `m['conteos']` (`matrices_todos_periodos` devuelve tuplas) y `columna=` en `hhi_por_convocatoria` (la firma es `col_geo=`)
- `subprocess` hardcodea `'python3'`, no portable a Windows (el repo vive en Windows)
- Depende de `datos/raw/produccion_grupos.csv` (~1,2 GB) que NO esta en el repo: la celda 2 de carga crashea y las secciones 11-13 no tienen prod
- Inconsistencias numericas entre el texto del manual y el codigo real: '22 registros con edad >100 (0,03 %)' vs `src/analisis/calidad.py` '10 registros (0,013 %)'; '211 IES canonicas / 91 %' vs celda '258 strings mapeados'
- Nunca ejecutado de punta a punta (`execution_count` null en las 26 celdas de codigo): el manual no tiene evidencia de reproducibilidad
- El generador `scripts/generar_manual.py` contiene los mismos bugs (los hereda el notebook)

Sugerencias concretas

- Corregir la celda de matrices: `m` es una tupla (`conteos, probs`) -> iterar con `for` periodo, (`conteos, probs`) in `matrices.items()`
- Corregir la celda HHI: `hhi_por_convocatoria(inv_clean, col_geo='NME_DEPARTAMENTO_RES_PR')`
- Usar `'python'` o `sys.executable` en `subprocess` para portabilidad a Windows
- Anadir celda condicional que avise si falta el dataset de produccion y permita continuar con las secciones que no lo necesitan
- Armonizar cifras con `src/analisis/calidad.py` y `scripts/sprint6_tabla_maestra_ies.py`, y regenerar con `generar_manual.py`
- Ejecutar el notebook completo y commitear los outputs como evidencia de reproducibilidad

Ejecutabilidad Parcial y no reproducible de punta a punta tal cual. Con `'poetry run jupyter notebook docs/manual.ipynb'` (README) la carga de `inv` funciona (XLSX existe; `openpyxl` en `lock/requirements`), pero: (1) `cargar_produccion()` lanza `FileNotFoundError` (`datos/raw/produccion_grupos.csv` ausente) -> la celda 2 crashea y las secciones 11-13 carecen de prod; (2) la celda de matrices lanza `TypeError` (`m['conteos']` sobre una tupla); (3) la celda HHI lanza `TypeError` (`kwargs columna=` inexistente); (4) los 4 `subprocess` con `'python3'` fallan en Windows. Las secciones 6-10 (tabla maestra, geografia, sankeys, genero, diversidad) leerian archivos que sí existen en el repo y funcionarían si se superan las anteriores.

¿Produce lo que se buscaba? El manual documenta los 12 hallazgos y sus rutas de evidencias/artifacts existen, pero NO es ejecutable tal cual: 3 celdas fallan por API incorrecta y 1 por dependencia de datos ausente. No contradice al dashboard (usa las mismas funciones de `src/`), pero la cifra '22 atipicos de edad' contradice a `src/analisis/calidad.py` ('10'), indicando

que el texto del manual no se actualizo con el codigo final. El placeholder `app/streamlit_app.py` es irrelevante para este notebook.

Riesgo / Limitación

Problemas detectados:

- Celda seccion 3: `'print(m['conteos'])'` - `matrices_todos_periodos` devuelve `{periodo: (conteos, probs)}` (tupla, segun docstring y codigo de `src/analisis/longitudinal.py` linea 164-179); `m` es una tupla -> `TypeError: tuple indices must be integers`
- Celda seccion 5: `'hhi_por_convocatoria(inv_clean, column='NME.DEPARTAMENTO_RES.PR)'` - la firma es `(df, col_geo, col_year)` -> `TypeError: unexpected keyword argument 'column'`
- Celda 2: `'cargar_produccion()'` -> `FileNotFoundError` porque `datos/raw/produccion_grupos.csv` no existe en el repo (hay que descargarlo, ~1,2 GB)
- 4 celdas usan `subprocess.run(['python3', ...])`: en Windows no existe `python3.exe` -> `FileNotFoundError` (portabilidad rota)
- Texto vs codigo: '22 registros con edad superior a 100 (0,03 %)' vs `src/analisis/calidad.py` 'diez registros (0,013 %)'; '211 IES canonicas que mapean el 91 %' vs celda que reporta '258 strings mapeados' (`scripts/sprint6_tabla_maestra_ies.py`)

Valoración global refactorizar: el manual es valioso como documentacion de los hallazgos pero no es reproducible tal cual. Corregir los 2 bugs de API (`m['conteos']`, `col_geo`), hacer portable el `subprocess`, documentar el prerequisite del dataset de produccion (o leer muestras de evidencias/) y armonizar las cifras con `src/`. Los mismos cambios deben aplicarse en `scripts/generar_manual.py` para que el notebook se pueda regenerar.

2.0.35 notebooks/01_edu.ipynb

Rol: edu · **Líneas:** 240

Descripción funcional EDA exploratorio de 16 celdas (6 markdown + 10 codigo) sobre el dataset consolidado de investigadores reconocidos (77.237 registros, 30 variables, 6 convocatorias 2013-2021). Carga con `src/ingesta.cargar_consolidado`, transforma con `src/Transformacion.transformar` y produce: tipos y faltantes, estadísticas de 3 variables numericas, frecuencias de 5 categoricas y 4 visualizaciones (genero, gran area OCDE, edad por convocatoria, top 15 departamentos). Es el punto de entrada analitico del proyecto, anterior a los sprint 2-6.

Análisis de la lógica Celda 1: `ROOT = pathlib.Path().resolve().parent` (comentario: 'raiz del repo (`notebooks/./`)', asume `cwd = notebooks/`); `sys.path.insert(0, ROOT/src)`; imports de `pandas/numpy/matplotlib/seaborn` y `'from ingest import cargar_consolidado'`, `'from Transformacion import transformar'`. Celda 2: `df_raw = cargar_consolidado()`; `df =`

`transformar(df_raw)`; `print shape` y `head`. Celdas siguientes analizan `df` directamente: `dtypes`, `faltantes (isnull().mean())`, `heatmap` de `NaN`, `describe` de `EDAD_ANOS_PR/NRO_ORDEN_FORM_PR/ORDEN_CLAS_PR`, `histogramas`, `value_counts` de 5 categoricas, `pie` de genero, `barh` de areas, `boxplot` de edad por convocatoria, `barplot` de top departamentos. Sin `cache` ni `subprocesos`; `input unico`: el consolidado.

Lo que está bien construido

- Sencillo y autocontenido: funciona sin el dataset de produccion (solo necesita el consolidado)
- Usa el pipeline real de ingesta+transformacion, misma fuente que el dashboard (consistencia de datos)
- Estructura EDA estandar y legible: carga, calidad, numericas, categoricas, visualizaciones
- Todas las columnas que referencia (`EDAD_ANOS_PR`, `NRO_ORDEN_FORM_PR`, `ORDEN_CLAS_PR`, `NME_REGION_RES_PR`, etc.) existen en el header del CSV consolidado (30 columnas, verificado)

Puntos de mejora (si aplican)

- Resolucion de ROOT fragil: depende de que el `cwd` del kernel sea `notebooks/`; si JupyterLab arranca el kernel desde la raiz del repo, ROOT apunta al padre del repo y `'from ingesta import ...'` lanza `ModuleNotFoundError` (a diferencia de `manual.ipynb`, no tiene fallback a `ROOT.parent`)
- El analisis de faltantes se hace sobre `df` YA transformado: `transformar` imputa mediana en numericas y `'NO REPORTADO'` en categoricas (`tratar_faltantes`), así que el `heatmap` de `NaN` y los % de faltantes quedan vacios o minimos (solo sobreviven `COD_DANE_*` y `ANO_CONVO_FECHA`); la seccion 'Calidad de datos' del EDA es poco informativa
- Nomenclatura inconsistente: el titulo dice 'Grupo 7' (la carpeta del repo es Grupo 8; `Transformacion.py` tambien dice Grupo 7)
- Usa `palette=` en `sns.boxplot/sns.barplot` (deprecacion en `seaborn` ≥ 0.14 ; cosmético, no rompe)
- No documenta como se lanza (`cwd` requerido) ni fija versiones de `seaborn`

Sugerencias concretas

- Adoptar el patron de ROOT del `manual.ipynb`: `Path.cwd()` con fallback a `ROOT.parent` (o derivar de la ubicacion del notebook) para funcionar desde cualquier `cwd`
- Correr el analisis de calidad sobre `df_raw` (antes de `transformar`) o documentar que los faltantes ya fueron imputados/etiquetados
- Unificar la nomenclatura de grupo (Grupo 7 vs Grupo 8) en titulo y docstrings
- Quitar `palette=` o fijar `seaborn<0.14` en `pyproject`

- Anadir celda que confirme `shape == (77237, 30)` como asercion de insumo

Ejecutabilidad Sí, con una condicion de entorno: se ejecuta tal cual si el kernel arranca con `cwd = notebooks/` (p. ej. ‘jupyter notebook notebooks/01_eda.ipynb’ desde la raiz, que en el cliente clasico pone el cwd en notebooks/). Entrada: `datos/tarea_join/investigadores_consolidado.xlsx` via `cargar_consolidado` (existe; el CSV de `datos/raw` no existe, se cae al XLSX). Requiere `pandas`, `numpy`, `matplotlib`, `seaborn` y `openpyxl`. No requiere el dataset de produccion. Si el cwd del kernel es la raiz del repo, el import de `ingesta` falla (`ModuleNotFoundError`).

¿Produce lo que se buscaba? Cumple su rol de EDA: carga y describe el consolidado y genera las 4 visualizaciones sin errores bajo la condicion de cwd correcta. Nota de verificacion: el CSV paralelo `datos/tarea_join/investigadores_consolidado.csv` tiene ~50.891 filas (no 77.237): conviene confirmar que el XLSX (fuente real de `cargar_consolidado`) tenga las 77.237 filas y que no haya divergencia entre formatos. La seccion de faltantes es casi vacia por la imputacion previa de transformar. No contradice al dashboard (misma fuente de datos).

Riesgo / Limitación

Problemas detectados:

- `ROOT = pathlib.Path().resolve().parent` (celda 1): si el cwd del kernel no es `notebooks/`, `ROOT` no apunta a la raiz del repo y ‘from `ingesta` import `cargar_consolidado`’ lanza `ModuleNotFoundError`; no hay fallback (a diferencia de `docs/manual.ipynb` que sí lo tiene)
- Analisis de calidad sobre `df` transformado: `transformar` (`tratar_faltantes`) imputa mediana en numericas y ‘NO REPORTADO’ en categoricas, por lo que `df.isnull()` queda casi sin NaN; el mapa de calor y los % de faltantes de la seccion 2 no muestran la calidad real del dato crudo
- Titulo ‘Grupo 7’ vs carpeta ‘Grupo 8’ (y `Transformacion.py` tambien dice Grupo 7): inconsistencia de nomenclatura del proyecto
- `sns.boxplot/palette` y `sns.barplot/palette`: deprecacion de `palette` en `seaborn` ≥ 0.14 (aviso, no falla)

Valoración global mejorar (mantener): el EDA es valido y util como punto de entrada; endurecer la resolucion de rutas con el patron del `manual.ipynb`, mover el analisis de calidad a `df_raw` (o aclarar que los faltantes ya fueron tratados), unificar la nomenclatura de grupo y fijar/eliminar `palette=`.

2.0.36 tests/__init__.py

Rol: testing · **Líneas:** 0

Descripción funcional Marcador de paquete Python para tests/ (vacío, 0 líneas).

Análisis de la lógica Sin lógica.

Lo que está bien construido

- Correcto por convención (permite descubrir tests con pytest).

Puntos de mejora No se identifican debilidades significativas: el archivo cumple su función de forma clara y consistente.

Ejecutabilidad N/A.

¿Produce lo que se buscaba? N/A.

Valoración global mantener — estándar de Python.

2.0.37 tests/test_ingesta.py

Rol: testing · **Líneas:** 8

Descripción funcional Suite de tests del módulo de ingesta. Contiene un único test (test_catalogo_exists) que verifica que datos/catalogo.yaml existe.

Análisis de la lógica test_catalogo_exists usa Path('datos/catalogo.yaml').exists() y assert. Configuración de testpaths en pyproject.toml apunta a tests/.

Lo que está bien construido

- Estructura de tests creada y ejecutable con pytest.
- Configuración en pyproject.toml (testpaths) correcta.

Puntos de mejora (si aplican)

- Cobertura insuficiente: 1 test trivial; no prueba descargar/guardar/transformar ni ninguna función de análisis.
- pandera declarado en dev-dependencies pero sin uso (no hay contratos de esquema).
- El test depende del cwd (Path relativo): falla si pytest se ejecuta desde otra carpeta.

Sugerencias concretas

- Añadir tests de humo para cargar consolidado (shape, columnas, unicidad de ID_PERSONA_PR).
- Añadir contratos de esquema con pandera (completitud, tipos, codificación).
- Usar rutas relativas al proyecto (pathlib con `_file_`).

Ejecutabilidad Sí (pytest); verificado que el único test pasa si el catálogo existe. No se ejecutó pytest en la auditoría por prioridad, pero no hay dependencias externas más allá de pathlib.

¿Produce lo que se buscaba? Parcial: valida solo la existencia del catálogo, no el comportamiento de la ingesta — no cumple el objetivo de 'probar' el pipeline.

Riesgo / Limitación**Problemas detectados:**

- Path relativo frágil (depende del directorio de ejecución).

Valoración global mejorar — ampliar la cobertura a transformación y análisis; es la acción de mayor impacto en la deuda técnica de pruebas.

2.0.38 notebooks/tarea_analisis_bases_de_datos/2019_codigo.py

Rol: legacy · **Líneas:** 174

Descripción funcional EDA descriptivo de la convocatoria 2019 (Investigadores_Reconocidos_por_convocatoria_20250909.csv): calcula el % de faltantes por variable, conteos y gráficos de barras para género, región/departamento de nacimiento y residencia, histogramas de variables numéricas y tablas cruzadas (género x región, gran área x género). Es un script exportado de Colab y corresponde a la tarea de análisis de bases de datos.

Análisis de la lógica pandas + matplotlib: lectura con `pd.read_csv`, limpieza de nombres de columnas, `df.isna().sum()` con % de faltantes ordenado, `value_counts` y `pd.crosstab(normalize='index')` para proporciones por fila, detección de columnas clave por subcadena ('GENER', 'REGION', 'DEPARTAMENTO'), histogramas y heatmap con `imshow`. No usa librerías de inferencia estadística; es puramente descriptivo.

Lo que está bien construido

- Reporta faltantes por variable con porcentaje ordenado, práctica correcta de EDA
- Usa crosstab con proporciones por fila (composición de género dentro de cada gran área), lectura adecuada para comparar grupos
- Detecta columnas de forma defensiva por subcadena en el nombre, tolerante a cambios de esquema
- Excluye columnas tipo ID/código de los histogramas (evita describir claves)

Puntos de mejora (si aplican)

- Ruta hardcodeada de Colab `'/content/Investigadores_Reconocidos_por_convocatoria_20250909.csv'`, inexistente en el clon y en cualquier equipo local
- Usa `display()` de IPython/Colab; como script Python estándar lanza `NameError`
- Bloque de cálculo de faltantes duplicado (`falt` y `faltantes`): código muerto
- El heatmap de proporciones no está acompañado de ninguna prueba de independencia (chi-cuadrado)
- Etiquetas de porcentaje con desplazamiento fijo `'v + 50'` sobre conteos absolutos, frágil si cambia la escala
- Las proporciones se calculan sobre `n` total incluyendo `NaN` en el denominador; sin documentar
- No guarda figuras ni resultados; sin estructura de función o parámetros

Sugerencias concretas

- Parametrizar la ruta (`Pathlib`/argumentos) en vez de hardcodear `/content`
- Reemplazar `display()` por `print()` para que corra como script
- Eliminar el bloque duplicado de faltantes
- Añadir `chi2.contingency` a las tablas cruzadas y reportar residuos
- Guardar figuras y tablas en una carpeta de salida

Ejecutabilidad NO. Falta el input `'/content/Investigadores_Reconocidos_por_convocatoria_20250909.csv'` (ruta de Colab, no existe en el clon) y además usa `display()` de IPython, que falla fuera de un entorno de notebook.

¿Produce lo que se buscaba? Metodología descriptiva correcta para el objetivo (EDA por convocatoria): no hay errores estadísticos graves porque no realiza inferencia. Único matiz: las proporciones usan el total con `NaN` y no se prueba la asociación en los cruces, por lo que los heatmaps son solo exploratorios.

Riesgo / Limitación**Problemas detectados:**

- `display()` no existe en scripts Python estándar (`NameError`)
- Cálculo de faltantes duplicado (`falt` y `faltantes`) sin uso del primero
- `plt.text(pct + 0.5, ...)` asume coordenadas de barras horizontales con holgura fija

Valoración global Mantener como referencia histórica del EDA inicial de la convocatoria 2019. No migrar: el pipeline moderno (`src/ingesta/minciencias.py`, `src/analisis/calidad.py`) ya sistematiza faltantes y perfiles descriptivos; este script no aporta lógica reutilizable.

2.0.39 notebooks/tarea_analisis_bases_de_datos/datos_2021.ipynb

Rol: legacy · **Líneas:** 2338

Descripción funcional Notebook Colab con el EDA de la convocatoria 2021 (Convocatoria 894): filtra Investigadores_Consolidado por año (21.094 registros x 30 variables), describe, analiza faltantes y duplicados, y gráfica género (pie), edad (histograma + detección de outliers >100 años), región de residencia, nivel de formación, gran área, área, clasificación (violinplot edad x clasificación) y proporciones de víctimas del conflicto, discapacidad y grupo étnico. Cierra con interpretaciones sustantivas en markdown.

Análisis de la lógica pandas + matplotlib + seaborn: filtra con `df['ANO_CONVO'].astype(str).str.contains('2021')`, `describe()`, `isnull().sum()`, `duplicated()`, `value_counts()`, `sns.histplot/countplot/violinplot`, y crea `df_2021_filtered` (edad <= 100) para los gráficos posteriores. Todo descriptivo; sin pruebas de hipótesis.

Lo que está bien construido

- Identifica y documenta el outlier de edad (máximo 956.25 años) y filtra edad <= 100 antes de las gráficas (decisión de limpieza explícita)
- Revisa duplicados y faltantes por columna con interpretación
- El violinplot edad x clasificación apoya una conclusión sustantiva correcta (Eméritos mayores, Junior más jóvenes)
- Buena práctica de narrativa: cada gráfico va acompañado de interpretación en markdown

Puntos de mejora (si aplican)

- Ruta hardcodeada de Colab `'/content/Investigadores_Consolidado.xlsx'`, inexistente en el clon

- La celda del countplot de nivel de formación x género usa `df_2021_gender_filtered`, variable que NO está definida en ninguna celda del notebook (NameError al re-ejecutar; probablemente se borró la celda que la creaba)
- `describe()` incluye IDs y códigos (media de `ID_PERSONA_PR` y `COD_DANE` sin sentido estadístico)
- Pie chart para género con categorías de `n=4-5` ('Intersexual', 'No disponible'): visualización poco informativa
- Salidas embebidas en base64 inflan el archivo a 2338 líneas, dificultando el diff
- Filtrado por `str.contains('2021')` sobre la fecha depende del formato de la columna (frágil si el Excel cambia el tipo)
- Sin seed, sin guardado de resultados y sin checklist de calidad al final

Sugerencias concretas

- Definir `df_2021_gender_filtered` o eliminar la celda rota
- Excluir IDs/códigos del `describe()`
- Reemplazar el pie por barras horizontales
- Limpiar salidas (Output -> Clear) antes de commitear
- Parametrizar la lectura del Excel y extraer el año con `dt.year` en vez de `str.contains`

Ejecutabilidad NO. Falta `'/content/Investigadores.Consolidado.xlsx'` (no existe en el clon) y la celda de nivel de formación x género lanza NameError porque `df_2021_gender_filtered` no está definida en el notebook.

¿Produce lo que se buscaba? Metodología descriptiva correcta para el objetivo (EDA por convocatoria). No hay errores de inferencia porque no se realizan pruebas; el filtro edad ≤ 100 es razonable pero el umbral no se justifica y no se hace análisis de sensibilidad (cuántos registros y cómo cambian los perfiles).

Riesgo / Limitación

Problemas detectados:

- `df_2021_gender_filtered` usada sin definición (NameError en re-ejecución)
- Filtro por `str.contains('2021')` sobre una columna de fecha: frágil ante cambios de formato
- `describe()` sobre columnas ID/código produce estadísticos sin interpretación

Valoración global Mantener como referencia del EDA de la convocatoria 2021 (documenta hallazgos de calidad como el outlier de edad). No migrar tal cual: los análisis equivalentes modernos están en `src/analisis/` (`produccion.py`, `genero.py`, `longitudinal.py`) con datos ya depurados.

2.0.40 notebooks/tarea_analisis_bases_de_datos/inv_17_codigo_graficas.R

Rol: legacy · **Líneas:** 614

Descripción funcional EDA descriptivo de la convocatoria 2017 (Investigadores_Reconocidos_por_convocatoria_20250831.xlsx): tablas de frecuencia y gráficos (barras, pie, histograma con densidad, boxplot) para área, gran área, género, país/región/departamento/municipio de nacimiento y de residencia, nivel de formación, clasificación, edad, víctimas del conflicto, grupo étnico, discapacidad e INST_FILIA. Detecta el valor imposible de edad (952 años) y crea una versión filtrada (edad <100). Corresponde a la tarea de análisis de bases de datos.

Análisis de la lógica readxl + ggplot2 + dplyr + forcats: count() con filtros por umbral de frecuencia para dividir categorías altas/bajas, geom_bar/geom_col con coord_flip y reorder/fct_infreq, pie charts con coord_polar, histograma con aes(y = ..density..) + geom_density + vline de la media. Sin pruebas formales; análisis univariado exploratorio.

Lo que está bien construido

- Reconoce explícitamente las variables de identificación (IDs, códigos DANE) y evita graficarlas, decisión de análisis correcta
- Detecta el valor imposible de edad (952) y construye edad_sin_error (edad <100), buen control de calidad de datos
- Ordena categorías con reorder/fct_infreq y separa frecuencias altas y bajas para legibilidad
- Autoevalúa gráficos poco informativos ('grafica sin sentido', 'no sirve la grafica'): honestidad analítica y criterio

Puntos de mejora (si aplican)

- Ruta hardcodeda 'Downloads/Investigadores_Reconocidos_por_convocatoria_20250831.xlsx', inexistente en el clon
- Usa plot_missing() de DataExplorer sin cargar library(DataExplorer): error en ejecución
- Referencia el objeto Investigadores_Consolidado (línea 21) sin definirlo en el archivo: error de ejecución
- Pie charts con decenas de categorías (país de nacimiento, región): mala práctica de visualización
- Umbrales de filtrado arbitrarios y comentarios desalineados con el código (p. ej., comenta 'menor a 140' pero filtra n<40)
- Número mágico hardcodedo: sum(NME_MUNICIPIO_RES_PR1\$n)/13001
- Sintaxis deprecada geom_text(aes(label = ..count..)) y aes(y = ..density..) en ggplot2 >= 3.4 (usa after_stat)

- No guarda gráficos ni tablas; sin documentación de parámetros ni versión de los datos

Sugerencias concretas

- Cargar DataExplorer o reemplazar plot_missing por un cálculo propio de faltantes
- Definir Investigadores_Consolidado o eliminar la línea que lo referencia
- Sustituir pie charts por barras horizontales cuando hay muchas categorías
- Documentar y justificar los umbrales ($n > 250$, $n > 500$, $n < 40$, etc.)
- Usar after_stat(count)/after_stat(density) y parametrizar la ruta y el denominador (13001)

Ejecutabilidad NO. Falta 'Downloads/Investigadores_Reconocidos_por_convocatoria_20250831.xlsx' (no existe en el clon) y además falla en plot_missing() por falta de library(DataExplorer) y en la referencia a Investigadores_Consolidado (objeto no definido).

¿Produce lo que se buscaba? Descriptivo correcto para el objetivo (EDA por convocatoria): no hay errores de inferencia porque no realiza pruebas. El filtro edad < 100 es apropiado. Los pie charts con muchas categorías son un problema de visualización, no estadístico.

Riesgo / Limitación

Problemas detectados:

- plot_missing() sin library(DataExplorer): función no encontrada
- Investigadores_Consolidado referenciado sin ser definido (línea 21)
- Sintaxis ..count../..density.. deprecada en ggplot2 moderno (deprecation warning/error según versión)
- Denominador hardcoded 13001 en el cálculo de proporción de municipios de residencia

Valoración global Mantener como referencia histórica del EDA de la convocatoria 2017 (documenta hallazgos de calidad de datos). No migrar: src/analisis/calidad.py y src/ingesta/minciencias.py sistematizan la limpieza; este script no aporta lógica reutilizable sin reescribirlo.

2.0.41 notebooks/tarea_dimensiones_y_hechos/codigo_dimensiones_C.R

Rol: legacy · **Líneas:** 40

Descripción funcional Crea tablas de dimensiones del modelo estrella a partir de Investigadores_Consolidado: dimensión investigador (solo ID_PERSONA_PR), dimensión municipio de nacimiento (COD_DANE_NAC_PR + país/región/departamento/municipio) y dimensión convocatoria (ID_CONVOCATORIA + nombre + fecha de convocatoria), exportadas a tres xlsx. Corresponde a la tarea de dimensiones y hechos.

Análisis de la lógica readxl + openxlsx + dplyr: unique() sobre la columna de ID, distinct(COD_DANE_NAC_PR, .keep_all = TRUE) para el municipio, distinct(ID_CONVOCATORIA, .keep_all = TRUE) para la convocatoria, y write.xlsx() por dimensión. Sin joins; solo extracción de claves y descriptores.

Lo que está bien construido

- Separa claves naturales (IDs) de atributos descriptivos, enfoque correcto de modelado dimensional
- Usa distinct() sobre la clave para garantizar unicidad en las dimensiones
- Genera artefactos accionables (xlsx) consumibles por la tabla de hechos

Puntos de mejora (si aplican)

- Ruta hardcodeada 'Downloads/Investigadores_Consolidado.xlsx', inexistente en el clon
- Dimensión investigador sin atributos (solo ID_PERSONA_PR): tabla sin valor analítico y sin vínculo a institución/universidad
- distinct(COD_DANE_NAC_PR, .keep_all = TRUE) toma la PRIMERA fila por código: si el mismo código aparece con nombres distintos o con NA (investigadores en el 'Exterior'), la dimensión es no determinista
- No maneja NA: los códigos DANE faltantes (Exterior) entran como fila NA en la dimensión
- Nombres de salida con errores de tipo (Dimension_investigadores.xlsx, Dimension_municipios_de_nacimientos.xlsx) que NO coinciden con los nombres que consume la tabla de hechos (dimensiones_investigadores.xlsx del notebook): dos implementaciones paralelas de dimensiones incompatibles entre sí
- Sin claves sustitutas para municipio y convocatoria; sin validación de cardinalidad
- No hay dimensión de formación, clasificación, área ni residencia aquí (la residencia se hace en el notebook, no en este script)

Sugerencias concretas

- Unificar con las dimensiones del notebook dimensiones.ipynb (una sola implementación)
- Añadir claves sustitutas y atributos a todas las dimensiones (incluida la de investigador)
- Decidir y documentar el tratamiento de 'Exterior'/NA antes de construir la dimensión
- Corregir nombres de archivo y validar la cardinalidad de la dimensión contra la tabla de hechos

Ejecutabilidad NO. Falta 'Downloads/Investigadores_Consolidado.xlsx' en el clon.

¿Produce lo que se buscaba? El diseño (claves naturales + descriptores) es conceptualmente correcto para un modelo estrella, pero incompleto: la dimensión investigador no tiene atributos, la de municipio depende del orden de filas (`.keep_all = TRUE`) y no se resuelve el caso 'Exterior'. Riesgo de claves inconsistentes entre dimensiones de archivos distintos.

Riesgo / Limitación

Problemas detectados:

- Typos en los nombres de archivo de salida (investigadores, municipios de naciminetos) que rompen la trazabilidad con la tabla de hechos
- `distinct()` con `.keep_all = TRUE`: resultado dependiente del orden de filas en la base
- Filas con `COD_DANE_NAC_PR = NA` ('Exterior') entran como clave NA en la dimensión

Valoración global Mantener como referencia histórica del modelado dimensional académico. No migrar: el modelo moderno está en `src/modelo/dimensional.py`; además conviene consolidar con el notebook de dimensiones para eliminar la duplicación incompatible.

2.0.42 notebooks/tarea_dimensiones_y_hechos/dimensiones.ipynb

Rol: legacy · **Líneas:** 289

Descripción funcional Notebook Colab que construye tres dimensiones del modelo estrella a partir de `Investigadores_Consolidado`: `Dim_Universidad` (`INST_FILIA` con clave sustituta 'Universidad'), `Dim_Formacion` (`ID_NIV_FORMACION_PR`, `NME_NIV_FORM_PR`, `NRO_ORDEN_FORM_PR`) y `Dim_Residencia` (`COD_DANE_RES_PR` + país/región/departamento/municipio), y las exporta a `dimensiones_investigadores.xlsx` (3 hojas) con descarga vía `files.download`. Es la implementación de dimensiones que luego consume `tabla_hechos_codigo.R`.

Análisis de la lógica `pandas` + `openpyxl` + `google.colab.files`: `dropna().drop_duplicates().reset_index(drop=True)` por dimensión, `insert(0, 'Universidad', range(1, len+1))` para la clave sustituta, `pd.ExcelWriter` con una hoja por dimensión y `files.download()`.

Lo que está bien construido

- Agrega clave sustituta a `Dim_Universidad` (buena práctica de modelado dimensional)
- Separa dimensiones por tema y las exporta en un único workbook con hojas nombradas
- Conserva la jerarquía de formación en `NRO_ORDEN_FORM_PR` (ordinal), lo que permite ordenar después
- Salida estructurada y lista para el join de hechos

Puntos de mejora (si aplican)

- Ruta 'Investigadores_Consolidado (1).xlsx' (archivo subido a Colab), inexistente en el clon
- INST_FILIA contiene LISTAS separadas por PIPE: drop_duplicates() trata 'UNIV A|UNIV B' como una 'universidad' falsa (dimension inflada) y dropna() elimina a los investigadores sin filiación
- Dim_Residencia con dropna() elimina TODAS las filas con COD_DANE_RES_PR NA, es decir, todos los investigadores en el 'Exterior' quedan fuera de la dimensión: la tabla de hechos luego hace left_join y pierde esas claves
- Sin clave sustituta para residencia y formación (solo claves naturales); la clave 'Universidad' no se usa en el join de hechos (une por INST_FILIA)
- Modelo incompleto: no hay dimensión de nacimiento, de área de conocimiento ni de clasificación
- El código está comentado en su versión de upload (files.upload) y no es reproducible fuera de Colab

Sugerencias concretas

- Splitear INST_FILIA por '|' (formato largo o primera institución documentado) antes de deduplicar
- Incluir 'Exterior' como categoría explícita (p. ej., código 0/99999) en lugar de dropna()
- Añadir claves sustitutas consistentes y dimensiones de nacimiento/área/clasificación
- Documentar la granularidad de la fila de hechos (persona x convocatoria) y validar cardinalidad de la dimensión
- Externalizar la lectura del Excel y la ruta de salida

Ejecutabilidad NO. Requiere subir el Excel en Colab (google.colab.files) y el archivo 'Investigadores_Consolidado (1).xlsx' no está en el clon; files.download() tampoco existe fuera de Colab.

¿Produce lo que se buscaba? Diseño de dimensiones razonable para un modelo estrella académico, pero con pérdida real de datos: el dropna() de Dim_Residencia excluye al 'Exterior' y la granularidad de INST_FILIA con pipes queda mal modelada. Estos defectos se propagan a la tabla de hechos (joins con NA).

Riesgo / Limitación**Problemas detectados:**

- dropna() en Dim_Residencia elimina las filas 'Exterior' (COD_DANE NA), dejando hechos sin clave de residencia
- INST_FILIA con pipes ('UNIV A|UNIV B') tratada como una sola institución en la

dimensión

- La clave sustituta 'Universidad' no se corresponde con el join de la tabla de hechos (que une por INST_FILIA)

Valoración global Mantener como referencia del diseño dimensional académico. No migrar: `src/modelo/dimensional.py` es el reemplazo moderno; si se conserva la idea, corregir el manejo de pipes y de 'Exterior'.

2.0.43 notebooks/tarea_dimensiones_y_hechos/tabla_hechos_codigo.R

Rol: legacy · **Líneas:** 20

Descripción funcional Construye la tabla de hechos del modelo estrella: hace `left_join` de `Investigadores_Consolidado` con `Dim_Universidad` (por `INST_FILIA`) y con `Dim_genero` (por `NME_GENERO_PR`), selecciona columnas por posición numérica y exporta `Tabla_hechos.xlsx`. Corresponde a la tarea de dimensiones y hechos.

Análisis de la lógica `readxl + openxlsx + dplyr`: dos `left_join` con las dimensiones, subconjunto por índices `c(1,4,5,14,15,18,21,26,27:29,31,32)` y `write.xlsx()`. La granularidad resultante es persona x convocatoria.

Lo que está bien construido

- Integra las claves de dimensiones a la fila de hechos mediante joins, enfoque correcto de modelo estrella
- El join por `INST_FILIA` y `NME_GENERO_PR` es conceptualmente válido si las dimensiones son únicas por esas columnas

Puntos de mejora (si aplican)

- `Investigadores_Consolidado` NO se define en este archivo: depende de un objeto global de una sesión R anterior (probablemente creado en `codigo_dimensiones.C.R`). No reproducible en una sesión limpia
- Selección de columnas por posición numérica (índices mágicos) sin documentar qué columnas son: frágil ante cualquier cambio de esquema
- El join por `INST_FILIA` arrastra el problema de las dimensiones: investigadores sin filiación o con `INST_FILIA` en PIPE quedan con clave de universidad NA o mal asignada
- No verifica cardinalidad: si la dimensión no es única por `INST_FILIA`, el `left_join` duplicaría filas silenciosamente

- Rutas hardcodeadas 'Downloads/dimensiones_investigadores.xlsx' y 'Downloads/Dim_genero (1).csv', inexistentes en el clon
- Nombre de salida sin carpeta explícita ('Tabla.hechos.xlsx' en el working directory)

Sugerencias concretas

- Leer el Excel al inicio del script (autocontenido)
- Seleccionar columnas por nombre (dplyr::select) y documentar el diccionario de la tabla de hechos
- Validar que nrow() no cambie tras cada join y reportar claves NA
- Escribir con ruta explícita y verificable

Ejecutabilidad NO. Falta el objeto Investigadores_Consolidado (no definido en el archivo) y los inputs 'Downloads/dimensiones_investigadores.xlsx' y 'Downloads/Dim_genero (1).csv' no existen en el clon.

¿Produce lo que se buscaba? La lógica de hechos es correcta para un modelo estrella simple (fila persona x convocatoria con claves de dimensión), pero la selección por posición y la dependencia de objetos globales la hacen propensa a errores. Nota: la tabla de hechos no contiene medidas/indicadores, solo claves; aceptable como entregable académico pero de uso analítico limitado.

Riesgo / Limitación

Problemas detectados:

- Investigadores_Consolidado no definido en el script (error de objeto no encontrado)
- Índices mágicos c(1,4,5,14,15,18,21,26,27:29,31,32) sin correspondencia verificable con el esquema
- Sin control de cardinalidad tras los joins (riesgo de duplicación de filas)

Valoración global Mantener como referencia del entregable de hechos. No migrar: src/modelo/dimensional.py genera el modelo estrella moderno; si se quisiera conservar, reescribir con joins por nombre, lectura autocontenida y validaciones.

2.0.44 notebooks/tarea_gran_tabla/analisis2.ipynb

Rol: legacy · **Líneas:** 294

Descripción funcional Análisis bivariado de la GRAN_TABLA(SIN ID): matriz de correlación entre numéricas, countplots con porcentajes (género x convocatoria, clasificación x gran área, formación x clasificación), tablas de contingencia con pruebas chi-cuadrado y

coeficiente de contingencia de Pearson (clasificación x región, formación x región, convocatoria x gran área), y Kruskal-Wallis + post-hoc de Dunn con Bonferroni para edad según gran área. NOTA: a pesar de la extensión .ipynb, el contenido es un script Python exportado de Colab (no es un notebook JSON), lo que sugiere una descarga mal nombrada.

Análisis de la lógica pandas + seaborn + scipy: `pd.crosstab`, `chi2.contingency`, `np.sqrt(chi2/(chi2+n))` para el coeficiente de contingencia, `scipy.stats.kruskal` y `scikit_posthocs.posthoc_dunn(p.adjust='bonferroni')`. Incluye anotación manual de porcentajes sobre barras apiladas y `!pip install` dentro del script.

Lo que está bien construido

- Usa chi-cuadrado con coeficiente de contingencia de Pearson: medida de asociación adecuada para variables nominales
- Elige Kruskal-Wallis (no paramétrico) para edad por gran área: robusto ante la no normalidad de la edad
- Aplica post-hoc de Dunn con corrección Bonferroni tras Kruskal-Wallis: secuencia inferencial correcta
- Ordena la tabla de contingencia por jerarquía de clasificación (ORDEN_CLAS_PR) para lectura sustantiva

Puntos de mejora (si aplican)

- La extensión .ipynb es engañosa: el archivo es un script .py exportado de Colab; no abre como notebook
- Ruta hardcodeada `'/content/GRAN_TABLA(SIN ID).xlsx'`, inexistente en el clon
- `!pip install scikit-posthocs` dentro del código: magia de notebook que falla como script y rompe la reproducibilidad de dependencias
- Chi-cuadrado sin verificación de frecuencias esperadas: las tablas región x clasificación y región x formación tienen celdas pequeñas ('Exterior', 'No disponible') que invalidan la aproximación asintótica
- Correlación de Pearson sobre variables ordinales (NRO_ORDEN_FORM_PR, ORDEN_CLAS_PR) tratadas como continuas; lo adecuado es Spearman
- El bloque de anotación de porcentajes usa aritmética frágil sobre ticks de matplotlib (código muerto 'categoria' y riesgo de división por cero)
- No guarda resultados ni figuras; sin semilla (no aplica) ni control de múltiples comparaciones más allá de Bonferroni

Sugerencias concretas

- Renombrar el archivo a .py y eliminar la magia !pip install (declarar dependencia en requirements)
- Parametrizar la ruta de lectura
- Reportar la mínima frecuencia esperada o agrupar categorías raras antes del chi-cuadrado (o usar test exacto)
- Usar Spearman para ordinales y simplificar las anotaciones de barras
- Guardar tablas de resultados en CSV

Ejecutabilidad NO. Falta '/content/GRAN_TABLA(SIN ID).xlsx' y las magias de Colab (!pip install) fallan al ejecutarlo como script Python estándar.

¿Produce lo que se buscaba? Metodología inferencial mayormente correcta: chi-cuadrado para asociación de nominales y Kruskal-Wallis + Dunn para edad por grupo. Riesgo: celdas con frecuencias esperadas bajas debilitan el chi-cuadrado, y con $n \sim 50.000$ casi cualquier asociación resulta significativa, por lo que el coeficiente de contingencia (bien calculado) es la métrica realmente informativa.

Riesgo / Limitación

Problemas detectados:

- Extensión .ipynb con contenido de script .py (no es JSON de notebook)
- !pip install dentro del archivo (falla fuera de notebook)
- Lógica de anotación de barras dependiente del espaciado de xticks (puede dividir por cero o etiquetar mal con hue)
- Pearson aplicado a variables ordinales

Valoración global Mantener como referencia del análisis bivariado de la gran tabla. No migrar tal cual: los análisis equivalentes modernos están en src/analisis/ (genero.py, territorial.py, produccion.py); si se quiere conservar el chi-cuadrado, reimplementarlo en el pipeline con validación de supuestos y guardado de resultados.

2.0.45 notebooks/tarea_gran_tabla/analisis_de_clusters.R

Rol: legacy · **Líneas:** 94

Descripción funcional Clustering k-prototypes sobre la GRAN_TABLA(SIN ID) para datos mixtos: convierte las categóricas en factor/ordinal, elimina casos incompletos con cc(), aplica kproto($k = 3$) con set.seed(123) y describe los clusters con tablas de proporción por variable (gran área, género, región de nacimiento/residencia, formación, clasificación, discapacidad) y medias de edad. Corresponde a la tarea de análisis multivariado de la gran tabla.

Análisis de la lógica `klaR` + `clustMixType` + `factoextra` + `dplyr`: `mutate(across(c(1:10,16:23), factor))` y `ordered()` para formación/clasificación), `cc()` para casos completos (retiene el 91.6 % de la base), `kproto(GT_NA, k = 3)`, `filter` por cluster y `table()/mean()` por grupo para perfilado.

Lo que está bien construido

- Elección correcta del método: k-prototypes (`clustMixType`) es el algoritmo apropiado para datos mixtos categórico-numérico
- Fija semilla (`set.seed(123)`): el resultado del `kproto` es reproducible
- Reporta explícitamente la proporción de datos retenidos tras eliminar casos incompletos (91.6 %)
- Perfilado de clusters con proporciones por variable y media de edad: enfoque descriptivo adecuado para interpretar grupos

Puntos de mejora (si aplican)

- `k = 3` fijado sin validación: no hay curva de `withinss/silhouette`, ni comparación de varios `k`, ni análisis del `lambda` (importancia de variables) de `kproto`
- `cc()` (función de casos completos del paquete `mice`) elimina el 8.4 % de las filas sin evaluar sesgo: los registros faltantes pueden ser sistemáticos (p. ej., 'Exterior', sin filiación, sin código DANE)
- `mice` NO está cargado en este script (solo `readxl`, `klaR`, `dplyr`, `factoextra`, `clustMixType`): `cc()` solo existe si se ejecutó antes `analisis_multivariado_base_gran_table.R` en la misma sesión: no reproducible
- Incluye el outlier de edad (956 años) en el clusterizado sin filtrar (los análisis previos sí lo descartaban)
- Categorías 'No disponible'/'Exterior' dominantes distorsionan las distancias mixtas del `kproto` (el algoritmo estandariza categorías de forma uniforme)
- El perfilado es solo inspección visual de proporciones: sin chi-cuadrado/ANOVA por variable vs cluster ni tamaño de clusters reportado
- Ruta hardcodeada 'Downloads/GRAN_TABLA(SIN ID).xlsx', inexistente en el clon

Sugerencias concretas

- Validar `k` con varios valores (p. ej., `k = 2..8`) usando métricas de `clustMixType` (`lambda`, `withinss`) o `silhouette` para datos mixtos
- Cargar `mice` (o definir casos completos con `complete.cases()`) y documentar el sesgo de la eliminación; evaluar imputación como alternativa
- Filtrar edad >100 antes de clusterizar (coherencia con la limpieza previa)

- Reportar el lambda de cada variable (contribución al clustering) y validar perfiles con pruebas formales
- Documentar la granularidad y el número de filas por cluster

Ejecutabilidad NO. Falta 'Downloads/GRAN_TABLA(SIN ID).xlsx' en el clon y, en una sesión limpia, `cc()` no existe porque `mice` no está cargado (depende de ejecutar antes el script multivariado en la misma sesión R).

¿Produce lo que se buscaba? El método (k-prototypes) es el adecuado para datos mixtos, pero la selección de k sin validación y la eliminación de NA por casos completos sin evaluar sesgo son errores metodológicos: los clusters pueden ser artefactos del número de grupos impuesto y de la pérdida de 8.4 % de la muestra. Los perfiles por proporción son descriptivos y no validan la estructura encontrada.

Riesgo / Limitación

Problemas detectados:

- `cc()` (`mice`) usado sin cargar `mice` en el script: dependencia oculta de sesión previa
- `k = 3` fijo sin justificación ni validación (no hay análisis de sensibilidad a k)
- Outlier de edad (956 años) incluido en el clusterizado
- Eliminación de casos incompletos (8.4 %) sin análisis de sesgo

Valoración global Mantener como referencia académica de la tarea (muestra el uso correcto de k-prototypes para datos mixtos). No migrar: el pipeline moderno no incluye clustering; si se reintroduce, hacerlo con validación de k, manejo de NA documentado y datos depurados.

2.0.46 notebooks/tarea_gran_tabla/analisis_multivariado_base_gran_table.R

Rol: legacy · **Líneas:** 405

Descripción funcional Análisis multivariado de la gran tabla: análisis de correspondencia (CA) para pares de categóricas (gran área x género, INST_FILIA x género, nivel de formación x clasificación), análisis de correspondencia múltiple (MCA) para conjuntos de categóricas (5 variables; 3 variables de población) y FAMD (factor analysis of mixed data) con ANO_CONVO, gran área, género, país de nacimiento y EDAD_ANOS_PR. Reporta eigenvalores, contribuciones y `cos2`, y construye biplots con `ggrepel`. Incluye análisis de faltantes (`plot_missing`, `md.pattern`) y retira una fila totalmente vacía.

Análisis de la lógica FactoMineR (CA, MCA, FAMD) + factoextra + ggplot2 + ggrepel + mice + DataExplorer: `table() -> CA(bc/ bd/ bf, graph = F)`, `MCA(GT[, c(2,5,6,7,16)], graph = F)` y `MCA(GT[, c(20:22)], ncp = 16)`, `FAMD(GT[-50891, c(2,5,6,7,15)], ncp = 20, graph = T)`; extrae `b$eig`, `$row/$col/$var/$quanti.var/$quali.var coord/cos2/contrib` y grafica varianza y biplots.

Lo que está bien construido

- Elección correcta de método según tipo de datos: CA para tablas de contingencia 2 x k, MCA para múltiples categóricas, FAMD para datos mixtos
- Reporta contribuciones y cos2 (calidad de representación de filas/columnas), buenas prácticas de lectura de CA/MCA/FAMD
- Detecta y elimina la fila totalmente vacía (`md.pattern / lis.finite`) antes del FAMD
- Gráficos de varianza explicada y acumulada con umbral del 70 % para decidir número de dimensiones

Puntos de mejora (si aplican)

- Ruta hardcodeada `'Downloads/GRAN_TABLA(SIN ID).xlsx'`, inexistente en el clon
- CA sobre `INST_FILIA` x género con miles de filas (instituciones y combinaciones PIPE sin normalizar): tabla dispersa donde las categorías con frecuencia 1 dominan el chi-cuadrado; resultado poco interpretable
- Remoción de la fila 50891 por índice hardcodeado (`GT[-50891, ...]`): frágil si cambia la base; debería ser un filtro por contenido
- Comentario del MCA (dice `NME_PAIS_RES_PR`) no coincide con el código (columna 16): documentación desalineada
- Los CA no van acompañados de `chisq.test`: no se reporta significancia ni frecuencias esperadas
- MCA/FAMD con categorías `'No disponible'`/`'Exterior'` y NAs sin tratamiento explícito documentado (depende del default `na.method` de FactoMineR)
- FAMD con `ncp = 20` para solo 5 variables: sobredimensionado
- No guarda resultados (coordenadas/contribuciones) en archivos

Sugerencias concretas

- Excluir `INST_FILIA` del CA o agregar a nivel de institución (eliminar pipes) antes de analizar
- Acompañar cada CA con `chisq.test` y verificación de frecuencias esperadas
- Eliminar la fila vacía con un filtro (p. ej., `rowSums(is.na()) < ncol`) en vez del índice 50891
- Alinear comentarios con columnas y documentar el tratamiento de NA

- Guardar eig/coord/cos2/contrib en xlsx para trazabilidad

Ejecutabilidad NO. Falta 'Downloads/GRAN_TABLA(SIN ID).xlsx' en el clon.

¿Produce lo que se buscaba? Metodología en general correcta: CA/MCA/FAMD están bien aplicados según el tipo de variables. Riesgos: el CA sobre INST_FILIA con tabla dispersa infla la inercia de categorías raras (poca interpretabilidad); FAMD con ncp=20 es inofensivo pero sobredimensionado; la lectura de contribuciones sin pruebas de significancia puede llevar a sobreinterpretación.

Riesgo / Limitación

Problemas detectados:

- Índice hardcodedo 50891 para eliminar la fila vacía (rompe con cualquier cambio de la base)
- Comentario del MCA (NME_PAIS_RES_PR) desalineado con la columna 16 del código
- CA sobre INST_FILIA sin normalizar los valores PIPE (instituciones múltiples tratadas como categorías únicas)
- Sintaxis ggplot con aes(shape = '...') dentro de geom_point que no varía la forma real (leyenda engañosa)

Valoración global Mantener como referencia académica (buen ejemplo de aplicación de CA/MCA/FAMD). No migrar tal cual: el pipeline moderno no realiza análisis factorial; si se requiere en el futuro, reimplementarlo con datos limpios, validación de supuestos y guardado de resultados.

2.0.47 notebooks/tarea_gran_tabla/codigo_analisis_univariado.py

Rol: legacy · **Líneas:** 123

Descripción funcional Análisis univariado de la gran tabla: revisa tipos de datos, describe() de las numéricas (NRO_ORDEN_FORM_PR, ORDEN_CLAS_PR, EDAD_ANOS_PR) con % de faltantes, histogramas y boxplots, value_counts de las categóricas, gráficos de barras de las principales variables y un bloque robusto de parsing de ANO_CONVO (texto dd/mm/aaaa o serial de Excel) para contar investigadores por año. Corresponde a la tarea de análisis univariado de la gran tabla.

Análisis de la lógica pandas + seaborn + matplotlib: describe().T con missing_% agregada, sns.histplot/boxplot, value_counts(dropna=False), y parsing de fecha con doble estrategia: pd.to_datetime(dayfirst=True) y, como respaldo, pd.to_datetime(serial, unit='D', origin='1899-12-30'); barplot por año de convocatoria.

Lo que está bien construido

- Parsing de fechas robusto con doble estrategia (texto y serial de Excel): buena práctica defensiva para datos heterogéneos
- Reporta % de faltantes junto con los estadísticos descriptivos
- Separa explícitamente variables numéricas, temporales y categóricas antes de analizar
- Análisis claro y estructurado, fácil de seguir

Puntos de mejora (si aplican)

- Ruta hardcodeada `'/content/GRAN_TABLA(SIN ID).xlsx'` (Colab), inexistente en el clon
- Lee el Excel dos veces (todo el archivo y luego solo la columna `ANO_CONVO`): ineficiente y evidencia de parche sobre el dato original
- Inconsistencia en el tratamiento de NA: las tablas usan `value_counts(dropna=False)` pero los gráficos usan el default `dropna=True`
- Las columnas numéricas llegan como texto/object (heredado de la creación de la gran tabla con `dtype='object'` en `codigo_creacion_gran_tabla_a_partir_del_modelo_estrella.py`), obligando a conversiones ad hoc: síntoma de un problema de tipos aguas arriba
- Sin pruebas estadísticas ni guardado de resultados

Sugerencias concretas

- Leer `ANO_CONVO` una sola vez y reutilizar el DataFrame
- Uniformizar el tratamiento de NA entre tablas y gráficos
- Corregir el dtype en el origen (creación de la gran tabla) en vez de parchear aquí
- Parametrizar la ruta y guardar el resumen descriptivo en CSV

Ejecutabilidad NO. Falta `'/content/GRAN_TABLA(SIN ID).xlsx'` (no existe en el clon; es una ruta de Colab).

¿Produce lo que se buscaba? Correcto como EDA univariado: no hay errores metodológicos porque no realiza inferencia. El parsing defensivo de fechas es una buena práctica. El único riesgo es interpretar estadísticos de variables ordinales (`NRO_ORDEN_FORM.PR`) como continuas, pero el script no extrae conclusiones inferenciales.

Riesgo / Limitación**Problemas detectados:**

- Doble lectura del Excel (desperdicio y riesgo de discrepancias entre lecturas)
- `value_counts` con `dropna=False` en tablas vs `dropna=True` en gráficos (conteos inconsisten-

tes)

- Dependencia de parche: supone que ANO_CONVO viene mal tipada y la re-parsea

Valoración global Mantener como referencia del EDA univariado de la gran tabla. **No migrar:** `src/analisis/calidad.py` y los scripts de sprint (`sprint2_duckdb.py`, `sprint2_genero_ocde.py`) ya cubren descriptivos y perfiles con datos tipados correctamente.

2.0.48 notebooks/tarea_gran_tabla/codigo_creacion_gran_tabla_a_partir_del_modelo_estrella.py

Rol: legacy · **Líneas:** 205

Descripción funcional Ensambla la gran tabla a partir del modelo estrella: lee todos los CSV/Excel de una carpeta, detecta la tabla de hechos por nombre (regex 'hecho|hechos|fact') o por máximo de filas, detecta claves de unión con heurísticas (columnas comunes únicas, llaves compuestas, columnas tipo ID) y hace merges left con guarda anti-1:N; exporta `gran_tabla.*` y un reporte de uniones, y luego elimina columnas ID para producir GRAN_TABLA(SIN ID). Corresponde a la tarea de creación de la gran tabla a partir del modelo estrella.

Análisis de la lógica pandas + re + pathlib: `read_any()` por archivo/hoja, `guess_fact_name()`, `candidate_keys_for_dim()` (`nunique == len`, combinaciones únicas, columnas `id_like`), `safe_left_join()` que revierte el merge si aumenta el número de filas (1:N), `merge(how='left')` con rename de columnas con prefijo de dimensión, drop de IDs y export a CSV/Excel (`xlsxwriter/openpyxl`).

Lo que está bien construido

- Heurísticas de detección de claves razonables y defensivas: verifica unicidad y revierte joins 1:N en lugar de duplicar filas silenciosamente
- Genera un reporte de uniones (`reporte_unioniones.csv/xlsx`) que permite auditar qué dimensiones se integraron y con qué claves
- Separa la versión completa (con IDs) de la versión analítica sin IDs (`GRAN_TABLA(SIN ID)`)
- Tolerante a formatos múltiples (CSV/Excel y múltiples hojas)

Puntos de mejora (si aplican)

- Ruta hardcodeada `'/content/A123'` (placeholder del drive del autor), inexistente en el clon; y `claves_forzadas` apunta a `'MiArchivo_Dimension_municipios_de_naciminetos_R'`,

nombre que nunca coincide con los archivos reales del repo: la única clave forzada no aplica y todo queda a merced de las heurísticas

- `!pip install xlswriter` y `display()` dentro del script: magias de Colab que fallan como script estándar
- Lee todo con `dtype='object'`: las columnas numéricas salen como texto y contaminan la gran tabla (por eso los análisis posteriores re-parsan fechas y números, p. ej., `codigo_analisis_univariado.py`)
- La heurística de clave puede unir por una columna común equivocada (p. ej., `NME_CONVOCATORIA`) sin verificación semántica de que sea clave en ambas tablas
- Renombra columnas con prefijos largos tipo `'Archivo__Hoja__col'`, generando nombres difíciles de usar
- No versiona inputs ni dimensiones: el resultado depende del contenido exacto de la carpeta
- Import de `re` duplicado (líneas 11 y 13) y variable `pat_fact` definida dos veces

Sugerencias concretas

- Eliminar magias de Colab (`pip/display`) y parametrizar la carpeta de entrada
- Leer con tipos inferidos o tipar explícitamente las columnas numéricas
- Verificar semánticamente las claves (unicidad en hechos y en dimensión) y agregar validación cruzada
- Persistir el reporte de uniones como artefacto del pipeline y versionar los inputs
- Usar prefijos de columna cortos y estables

Ejecutabilidad NO. Falta la carpeta `'/content/A123'` con las tablas del modelo estrella (y los nombres forzados `'MiArchivo_...'` no existen en el repo); además `!pip install` y `display()` son incompatibles con un script estándar.

¿Produce lo que se buscaba? La lógica de ensamblaje (star schema -> tabla denormalizada) es correcta en concepto y tiene buenas salvaguardas (reversión de 1:N, reporte de uniones). El problema grave es `dtype='object'`, que degrada tipos numéricos/ordenados y obliga a parches aguas abajo, y la falta de versionado de inputs, que hace el resultado no reproducible con los datos actuales del repositorio.

Riesgo / Limitación

Problemas detectados:

- `dtype='object'` al leer Excel convierte todas las columnas en texto (pérdida de tipos numéricos)
- `claves_forzadas` apunta a un nombre inexistente (`'MiArchivo__Dimension_municipios_de_naciminetos_R'`): nunca se usa

- !pip install dentro de un archivo .py (falla fuera de notebook)
- La detección de la tabla de hechos por máximo de filas puede elegir la tabla equivocada si ninguna coincide con la regex

Valoración global Mantener como referencia del proceso de ensamblaje (buena idea de reporte de uniones). No migrar tal cual: el modelo moderno (src/modelo/dimensional.py + scripts/sprint2_duckdb.py) ya construye la gran tabla con tipos correctos y control de calidad; este script solo aporta la idea de auditar las uniones.

2.0.49 notebooks/tarea_join/codigo_join.py

Rol: legacy · **Líneas:** 26

Descripción funcional Une verticalmente (concatenación) las bases por convocatoria 2017, 2019 y 2021 (Investigadores_Reconocidos_por_convocatoria_20250829*.csv) para crear Investigadores_Consolidado.xlsx y .csv y descargarlos. Corresponde a la tarea de join (unión de las bases por convocatoria en una base consolidada).

Análisis de la lógica pandas: pd.concat([df.2017, df.2019, df.2021], ignore_index=True), to_excel() y to_csv(), y descarga con google.colab.files.download(). Operación de unión vertical (union all de registros persona x convocatoria).

Lo que está bien construido

- Código simple y claro: hace exactamente lo que la tarea pide (unión vertical de las tres convocatorias)
- Genera la base en ambos formatos (Excel y CSV) para consumo posterior

Puntos de mejora (si aplican)

- Rutas hardcodeadas de Colab '/content/Investigadores_Reconocidos_por_convocatoria_20250829*.csv', inexistentes en el clon
- from google.colab import files: solo funciona en Colab; como script local falla
- concat sin verificar que las tres bases tengan el mismo esquema (mismas columnas, mismo orden, mismos tipos): un cambio de esquema genera columnas NaN o tipos mezclados silenciosamente
- Sin deduplicación ni control de calidad: los duplicados de ID_PERSONA_PR se manejan después en arreglar_base_consultoria_7.R (acoplamiento frágil entre scripts)

- Versiones de archivo inconsistentes entre tareas: aquí 20250829, pero 2019_codigo.py usa 20250909 e inv_17_codigo_graficas.R usa 20250831: los análisis de distintas tareas NO se hicieron sobre el mismo snapshot de datos
- Sin documentación de la granularidad resultante (fila = investigador x convocatoria)

Sugerencias concretas

- Verificar igualdad de columnas y tipos antes del concat (assert o comparación de listas)
- Deduplicar con criterio documentado en el mismo script o validar en uno solo
- Parametrizar rutas y fijar una única versión de archivos fuente para todo el proyecto
- Registrar un manifiesto de versiones de inputs

Ejecutabilidad NO. Faltan los tres CSV en '/content/' (no existen en el clon) y google.colab.files no existe fuera de Colab.

¿Produce lo que se buscaba? El concat vertical es la operación correcta para unir convocatorias (no hay error metodológico en la unión en sí). El riesgo es la ausencia de control de esquema y de deduplicación, que genera inconsistencias aguas abajo (duplicados, tipos mezclados) que luego otros scripts deben parchear.

Riesgo / Limitación

Problemas detectados:

- Import de google.colab.files fuera de Colab (ImportError)
- concat sin validación de esquema: si una convocatoria cambia de columnas, se crean NaN silenciosos
- Versiones de archivo fuente distintas entre tareas (20250829 vs 20250909 vs 20250831): datos no comparables entre scripts

Valoración global Mantener como referencia histórica de la tarea de join. No migrar: src/ingesta/minciencias.py hace la ingesta consolidada moderna con validación de esquema y versionado de datos.

2.0.50 notebooks/tarea_tabla/arreglar_base_consultoria_7.R

Rol: legacy · **Líneas:** 125

Descripción funcional Depura Investigadores_Consolidado: diagnostica duplicados de ID_PERSONA.PR, deduplica conservando una fila por persona con prioridad de convocatoria (21 > 20 > 19) mediante slice(1), y audita las diferencias de edad entre registros duplicados (dif_

edad >4.1) revisando casos anómalos de forma manual con View. Corresponde a la tarea de arreglo de la base (preparación para análisis a nivel de persona).

Análisis de la lógica readxl + dplyr: duplicated() y sum(table(...) >1) para diagnóstico, group_by(ID_PERSONA_PR) + mutate(prioridad = case_when(ID_CONVOCATORIA == 21 ~ 3, ...)) + arrange(desc(prioridad)) + slice(1) + ungroup() para deduplicar, y mutate(dif_edad = EDAD_ANOS_PR - dplyr::lag(EDAD_ANOS_PR)) dentro de group_by para auditar la coherencia temporal de la edad entre convocatorias.

Lo que está bien construido

- Enfoque de calidad de datos correcto: detecta duplicados, elige el registro con una jerarquía de convocatoria explícita y verificable, y valida la unicidad después de slice(1)
- Control de calidad interesante: calcula la diferencia de edad entre registros duplicados de la misma persona (debería ser ~2-4 años entre convocatorias) y detecta inconsistencias (dif_edad >4.1)
- Decisión documentada en el código (comentarios con los casos revisados y sus diferencias)
- Reporta la proporción de filas retenidas tras la deduplicación

Puntos de mejora (si aplican)

- Ruta hardcodeada 'Downloads/Investigadores_Consolidado.xlsx', inexistente en el clon
- write.xlsx() sin cargar library(openxlsx) (solo readxl y dplyr están cargados) y sin extensión en el nombre ('Base_sin_duplicados'): error en ejecución y archivo sin formato
- La prioridad por convocatoria crea un snapshot heterogéneo: los investigadores que aparecen en varias convocatorias quedan representados con sus datos del año más reciente, mientras que los singletons conservan el único año que tienen; la edad, formación y clasificación de distintas personas provienen de años distintos (sesgo de selección de año)
- Los casos con dif_edad >4.1 se inspeccionan visualmente pero NO se decide ni se registra su tratamiento (no se excluyen, no se corrigen): auditoría incompleta
- slice(1) sin desempate: si la misma persona tiene dos registros con la misma prioridad (duplicado dentro de la misma convocatoria), se elige una fila arbitraria (no determinista)
- No valida el resultado contra el total esperado ni contra la base original más allá de nrow ratio

Sugerencias concretas

- Cargar openxlsx y escribir 'Base_sin_duplicados.xlsx' con ruta explícita
- Documentar y aplicar una política para los casos con dif_edad >4.1 (excluir, imputar o marcar)

- Añadir un orden determinista dentro de slice(1) (p. ej., arrange por año o por nivel de formación)
- Evaluar el sesgo del snapshot mixto: reportar cuántos registros provienen de cada convocatoria tras la deduplicación
- Comparar la edad imputada vs la declarada entre convocatorias como control adicional

Ejecutabilidad NO. Falta 'Downloads/Investigadores_Consolidado.xlsx' en el clon y write.xlsx() falla por no tener cargado openxlsx (además el nombre de archivo carece de extensión).

¿Produce lo que se buscaba? La deduplicación con jerarquía es una decisión válida para análisis a nivel persona, pero introduce un sesgo de selección de año (mezcla de snapshots temporales). El análisis de dif.edad es un buen control de calidad incompleto: identifica anomalías pero no las resuelve ni documenta la decisión. La base resultante solo se valida por conteo, no por contenido.

Riesgo / Limitación

Problemas detectados:

- write.xlsx() sin library(openxlsx) y con nombre sin extensión (error de ejecución y archivo sin formato)
- slice(1) sin orden determinista ante empates de prioridad (resultado no reproducible)
- Caso de auditoría (dif.edad >4.1) revisado manualmente sin decisión ni registro del tratamiento

Valoración global Mantener como referencia del proceso de limpieza y deduplicación. No migrar: la limpieza moderna está en src/ingesta/minciencias.py y src/analisis/calidad.py; si se necesita la base persona-nivel, reimplementar con política de año explícita, desempate determinista y registro de decisiones de calidad.

2.0.51 src/__init__.py

Rol: ingesta · **Líneas:** 2

Descripción funcional Marcador de paquete Python para src/. Convierte la carpeta src/ en un paquete importable (los módulos se importan como 'from ingesta import ...' vía sys.path en los scripts).

Análisis de la lógica Sin lógica: archivo vacío/de marcador (2 líneas).

Lo que está bien construido

- Correcto por convención: permite imports del paquete.

Puntos de mejora (si aplican)

- Ninguna relevante; es boilerplate estándar.

Ejecutabilidad N/A (marcador de paquete).

¿Produce lo que se buscaba? N/A.

Valoración global mantener — estándar de Python.

3 Relación con los demás criterios

Este documento se centra en el criterio 3. Los demás criterios de la rúbrica — Entendimiento del objetivo del repositorio; Datos utilizados: suficiencia y corrección; Validación de la calidad estadística; Lógica de creación del repositorio y Línea de tiempo de commits — se desarrollan a fondo en sus propios documentos y en el **documento maestro** de la auditoría (`maestro-auditoria.pdf`), que los integra todos con el detalle completo de datos, código, estadística, lógica y línea de tiempo. Esta separación evita repetir contenido: cada PDF profundiza en lo suyo.

4 Conclusiones y recomendaciones del criterio

El código está bien construido en general (modular, documentado, trazable): 10/16 scripts corren en un clon fresco y los artefactos versionados demuestran que el equipo los ejecutó con los datos completos. Los hallazgos críticos de calidad: bug del HHI en el dashboard (ValueError tragado), denominador de minorías en `comparar_dane()` (comentario vs código), `redes.py` pierde pares de 3+ afiliaciones (`split n=1`), mojibake en el CSV, CI/CD rota (módulos inexistentes), Dockerfile con placeholder, tests insuficientes (1 test) y `kaleido` no declarado. La crítica de cada archivo es constructiva: cuando un archivo está bien construido se dice explícitamente.

Referencias
